Final Report
on

CALIPER: Empirical Measurement of Container Confinement

Submitted
in partial fulfillment of
the requirements for the degree of Bachelor of Technology in Computer Engineering

by

Abhinav Ajit Madake (231070028)
Yogesh Babaji Palve (231070037)
Ritesh Aresh Saindane (231070053)
Sahil Tatyabhau Waje (231080079)

Under the Guidance of

Dr. Shraddha S Suratkar

DEPARTMENT OF COMPUTER ENGINEERING AND INFORMATION TECHNOLOGY
VEERMATA JIJABAI TECHNOLOGICAL INSTITUTE
(An Autonomous Institute Affiliated to Mumbai University)
(Central Technological Institute, Maharashtra State)
Matunga, MUMBAI - 400019

A.Y. 2026–2027

STATEMENT OF CANDIDATE

We, the undersigned candidates, hereby declare that the project work entitled **"CALIPER: Empirical Measurement of Container Confinement"** submitted in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Engineering, is a record of our original work carried out during the academic year 2026–2027.

We further declare that the contents of this report are based on our own efforts and have not been submitted to any other institute or university for the award of any degree or diploma. All sources of information used in this project have been duly acknowledged, and no part of this work has been copied or plagiarised from existing literature without proper citation. Every measurement reported here was produced by the instrument described in this report, on environments we provisioned ourselves, and the fingerprints, metric files and commands needed to reproduce each number are published in the project repository.

Name of Candidate	**Roll No.**	**Signature**
Abhinav Ajit Madake	231070028	
Yogesh Babaji Palve	231070037	
Ritesh Aresh Saindane	231070053	
Sahil Tatyabhau Waje	231080079	

Date:
Place: VJTI, Mumbai

APPROVAL SHEET

This is to certify that the project report entitled **"CALIPER: Empirical Measurement of Container Confinement"** submitted by the following candidates in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in Computer Engineering** is approved.

The project work has been carried out under the supervision of **Dr. Shraddha S Suratkar**, Department of Computer Engineering and Information Technology, Veermata Jijabai Technological Institute (VJTI), Mumbai.

The report has been examined and is hereby approved as meeting the requirements prescribed by the institute.

Name of Candidate	**Roll No.**	**Signature**
Abhinav Ajit Madake	231070028	
Yogesh Babaji Palve	231070037	
Ritesh Aresh Saindane	231070053	
Sahil Tatyabhau Waje	231080079	

Dr. Shraddha S Suratkar
B.Tech. Supervisor ***Examiner***

Examiner ***Examiner***

Place: *Veermata Jijabai Technological Institute, Mumbai.*
Date:

CERTIFICATE

This is to certify that the project entitled **"CALIPER: Empirical Measurement of Container Confinement"** submitted by the following students has been carried out under the supervision of **Dr. Shraddha S Suratkar**, Department of Computer Engineering and Information Technology, Veermata Jijabai Technological Institute (VJTI), Mumbai, in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in Computer Engineering** for the academic year 2026–2027.

The work embodied in this project report is original and has not been submitted to any other institute or university for the award of any degree or diploma.

Name of Candidate	**Roll No.**	**Signature**
Abhinav Ajit Madake	231070028	
Yogesh Babaji Palve	231070037	
Ritesh Aresh Saindane	231070053	
Sahil Tatyabhau Waje	231080079	

Dr. Shraddha S Suratkar
B.Tech. Supervisor ***Examiner***

Examiner ***Examiner***

College Seal/Stamp

Place: *Veermata Jijabai Technological Institute, Mumbai.*
Date:

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our project guide, **Dr. Shraddha S Suratkar**, Department of Computer Engineering and Information Technology, VJTI Mumbai, for continuous support, valuable guidance and encouragement throughout the course of this project. The feedback we received on the scope of the instrument, on what could honestly be claimed from its measurements and on how the results should be reported shaped both the direction and the outcome of our work.

We are also grateful to [**Head of Department**], Head of the Department of Computer Engineering and Information Technology, for providing us with the necessary facilities, resources and an environment conducive to learning and research.

We extend our thanks to all faculty members and staff of the Department of Computer Engineering and Information Technology for their help and cooperation during the development of the project.

We acknowledge the open-source projects on which this work depends and against which it was evaluated: the Linux kernel, containerd, runc, gVisor, Kata Containers, Kubernetes and the Security Profiles Operator, the Rust and Go toolchains, and the maintainers of `amicontained` and `seccomp-diff`, whose tools established the idea this project builds on. The evaluation cells and the continuous-integration cluster ran on Google Cloud under a free-trial credit.

We would also like to thank our peers and friends for their support, collaboration and motivation throughout this journey. Finally, we express our heartfelt gratitude to our families for their unwavering encouragement, patience and belief in us, which gave us the strength to complete this project successfully.

Name of Candidate	**Roll No.**	**Signature**
Abhinav Ajit Madake	231070028	
Yogesh Babaji Palve	231070037	
Ritesh Aresh Saindane	231070053	
Sahil Tatyabhau Waje	231080079	

Date:
Place: **VJTI, Mumbai**

Contents

Statement of Candidate . . . . . **i**
Approval Sheet . . . . . **ii**
Certificate . . . . . **iii**
Acknowledgement . . . . . **iv**
List of Figures . . . . . **x**
List of Tables . . . . . **x**
List of Abbreviations . . . . . **xii**
1 Synopsis **1**
1.1 Abstract 1
1.2 Problem Statement 2
1.3 Motivation in Brief 2
1.4 Objectives 2
1.5 Summary of Contributions 2
1.6 Summary of Results 3
1.7 Organisation of the Report 3
2 Background and Motivation **5**
2.1 Containers Share a Kernel 5
2.2 The Enforcement Mechanisms 5
2.2.1 Namespaces and control groups 5
2.2.2 Capabilities 5
2.2.3 seccomp-BPF 6
2.2.4 Linux Security Modules 6
2.2.5 Masked and read-only paths 6
2.2.6 Other controls 6
2.3 Where Each Mechanism Acts 6
2.4 How Kubernetes and the Runtimes Configure Confinement 7
2.4.1 Pod security context and Pod Security Standards 7
2.4.2 RuntimeDefault is defined by the runtime 7
2.4.3 RuntimeClass: runc, gVisor and Kata 8
2.5 The Motivating Case: AF_ALG and Copy Fail 8
2.6 RuntimeDefault Drift 8
2.7 Why Existing Practice Does Not Detect It 9
3 Literature Survey **10**
3.1 Paper 1: Houdini, Benchmarking Container Security Confinement 10
3.2 Paper 2: A Measurement Study on Linux Container Security 11
3.3 Paper 3: ContainerLeaks, Information Leakage in Container Clouds 11

3.4 Paper 4: Houdini's Escape, Breaking the Resource Rein of Linux Control Groups . . . 12
3.5 Paper 5: Confine, Automated System Call Policy Generation for Containers 12
3.6 Paper 6: sysfilter, Automated System Call Filtering for Commodity Software 13
3.7 Paper 7: Temporal System Call Specialization for Attack Surface Reduction 13
3.8 Paper 8: Automating Seccomp Filter Generation for Linux Applications . . 14
3.9 Paper 9: Mining Sandboxes for Linux Containers . . . 14
3.10 Paper 10: SPEAKER, Split-Phase Execution of Application Containers . . 15
3.11 Paper 11: Lic-Sec, an Enhanced AppArmor Docker Security Profile Generator 15
3.12 Paper 12: Security Namespace, Linux Security Frameworks for Containers 16
3.13 Paper 13: Security Misconfigurations in Open Source Kubernetes Manifests 16
3.14 Paper 14: Understanding the Security Implications of Kubernetes Networking 17
3.15 Paper 15: The True Cost of Containing, a gVisor Case Study . . . 17
3.16 Paper 16: Blending Containers and Virtual Machines, Firecracker and gVisor 18
3.17 Paper 17: Firecracker, Lightweight Virtualization for Serverless Applications 18
3.18 Paper 18: Quantifiable Run-Time Kernel Attack Surface Reduction . . . 19
3.19 Paper 19: Container Security, Issues, Challenges, and the Road Ahead . . . 19
3.20 Paper 20: Cloud Container Technologies, a State-of-the-Art Review . . . 20
3.21 Paper 21: The State of the Art in Container Technologies . . . 20
3.22 Paper 22: Emerging Trends, Techniques and Open Issues of Containerization 20
3.23 Paper 23: A Container Security Survey, Exploits, Attacks and Defenses . . 21
3.24 Papers 24 and 25: Two Further Surveys of Containerisation and Its Security 21
3.25 Paper 26: To Docker or Not to Docker, a Security Perspective . . . 22
3.26 Paper 27: Docker Ecosystem, Vulnerability Analysis . . . 22
3.27 Paper 28: NIST SP 800-190, Application Container Security Guide . . . 22
3.28 Tools and Practice . . . 23
3.28.1 The two direct ancestors . . . 23
3.28.2 Summary of the survey . . . 23
4 Literature Gaps . . . 25
5 Proposed Solution . . . 27
5.1 Objective . . . 27
5.2 Rationale . . . 27
5.3 Research Questions . . . 27
5.4 Design Principles . . . 27
5.5 Objectives and Outcomes . . . 28
5.6 Success Criteria . . . 28
5.7 Scope . . . 29
5.8 Work Allocation . . . 29
5.9 Schedule . . . 29
6 System Architecture . . . 31
6.1 Overview . . . 31
6.2 Components . . . 31
6.3 The Measurement Half . . . 31
6.4 The Analysis Half . . . 32
6.5 The Environment Cell . . . 33
6.6 Why Rust for the Engine and Go for the Control Plane . . . 34
6.7 The Interface and Parallel Development . . . 34

6.8 Repository Organisation . . . 34
7 Methodology I: Measurement . . . 35
7.1 The Probe Specification . . . 35
7.2 Fork Isolation . . . 36
7.2.1 Why fork and not clone . . . 36
7.2.2 The exit status is the measurement . . . 36
7.2.3 Deadlines . . . 37
7.2.4 Surviving SIGSYS . . . 37
7.2.5 No core dumps on the node . . . 37
7.2.6 Faults of the instrument . . . 37
7.3 Verdict Classification . . . 37
7.4 The Engine's Own System Calls . . . 38
7.5 Side Effects and Residual State . . . 38
7.6 Module Autoload and the Reduced Corpus . . . 39
7.7 The Corpus . . . 39
7.7.1 Socket address families (24) . . . 40
7.7.2 Netlink protocols (18) . . . 40
7.7.3 io_uring opcode reach (53) . . . 41
7.7.4 Mount filesystem types (38) . . . 41
7.7.5 Clone and unshare flags (19) . . . 41
7.7.6 Masked and read-only paths (17) . . . 41
7.7.7 Device nodes (8) . . . 42
7.7.8 Capability effects (8) . . . 42
7.7.9 The deferred set (28) . . . 42
7.7.10 Oracle review and the freeze . . . 42
7.8 Safety Review . . . 43
8 Methodology II: Analysis . . . 45
8.1 The Fingerprint Format . . . 45
8.1.1 Conventions . . . 45
8.1.2 Digest . . . 45
8.1.3 Compatibility . . . 46
8.2 Diff and Divergence Classification . . . 46
8.3 Attribution . . . 47
8.3.1 Stage 1: the errno oracle . . . 47
8.3.2 Stage 2: correlation . . . 47
8.3.3 Stage 3: elimination by the cell . . . 48
8.3.4 Confidence and the undecidable fraction . . . 48
8.3.5 What is not attempted . . . 49
8.4 The Reference Posture . . . 49
8.5 The Evaluator . . . 49
8.6 The Remediator . . . 51
8.6.1 Narrowing rather than excepting . . . 51
8.6.2 Outputs . . . 51
8.6.3 Closure verification . . . 52
8.7 Reporting . . . 52
9 Technology Stack . . . 53
9.1 Overview . . . 53

9.2 Programming Languages . . . 53
9.3 Kernel and Runtime Interfaces Used . . . 54
9.4 Infrastructure . . . 54
9.4.1 Compute Engine cells . . . 54
9.4.2 GKE cluster for continuous integration . . . 54
9.5 Continuous Integration and Release . . . 54
9.6 Version Control and Collaboration . . . 54
9.7 Development Environment . . . 55
10 Experimental Setup . . . 56
10.1 The Environment Matrix . . . 56
10.2 Provisioning . . . 57
10.3 Measurement Procedure . . . 57
10.4 Ground-Truth Environments . . . 58
10.5 Remediation, Residual and Runtime Runs . . . 58
10.6 Continuous Evaluation on GKE . . . 58
10.7 Reproducibility . . . 59
11 Results and Analysis . . . 60
11.1 Verdicts per Cell . . . 60
11.1.1 Verdicts by family . . . 61
11.2 Divergence across Cells . . . 61
11.3 Expectation and Claim Violations . . . 63
11.3.1 Undeclared exposures by family . . . 64
11.3.2 The three claim violations . . . 64
11.4 The AF_ALG Case . . . 64
11.5 Attribution Accuracy . . . 65
11.5.1 The undecidable fraction in production . . . 66
11.6 Remediation Closure . . . 67
11.7 Residual State and Runtime Cost . . . 68
11.8 Comparison with Ancestor Tools . . . 68
11.9 Summary of Results . . . 70
11.9.1 Answers to the research questions . . . 70
12 Responsible Use and Disclosure . . . 71
12.1 Responsible Use . . . 71
12.2 Disclosure Review: Method . . . 71
12.3 Findings . . . 71
12.3.1 gVisor . . . 71
12.3.2 Kata Containers . . . 72
12.4 Where the Commitment Was Not Met . . . 73
13 Discussion . . . 74
13.1 What the Compression Gave Up . . . 74
13.2 Limitations . . . 74
13.3 Threats to Validity . . . 75
13.4 Lessons Learned . . . 76
14 Conclusion and Future Work . . . 77
14.1 Conclusion . . . 77
14.2 Future Work . . . 77
References . . . 79

A Reproducing the Results **84**
B Project Record **86**
B.1 Timeline 86
B.2 Milestones 86
B.3 Statistics 87
C The Complete Probe Index **88**

List of Figures

2.1 The order in which confinement mechanisms act on a typical system call, and the errors each can produce. The exact order is per system call and is recorded with every probe. 7
6.1 Architecture of CALIPER. The measurement half runs inside the measured container; the control plane consumes fingerprints. A remediation is verified by measuring again under the generated profile. 31
7.1 The lifecycle of one probe. The parent issues only `fork`, `pidfd_open`, `ppoll`, `wait4`, `close` and, on a timeout, `kill`. 36
8.1 Excerpt of the cell 2 fingerprint (`fixtures/cells/cell2.json`). 46
8.2 The order in which the classifier tests explanations for a divergence (`control/classifier.go`). 47
10.1 The six cells and the axis each moves relative to the baseline, cell 2. 56
11.1 Verdict distribution per run (values in Table 11.1). Cell 1's single not-applicable probe is omitted. 60
11.2 Divergence per cell pair by class (values in Table 11.3). Without the classifier, all 1,057 divergent probes would read as policy differences; 606 of them are explained by architecture, kernel version or RuntimeClass. 62
11.3 Undecidable fraction by cell, with the ground truth and the proposal's 10 per cent threshold for comparison (values in Table 11.8). 66
11.4 Runtime of the full and reduced corpus per cell, against the 60-second and 5-second budgets (dashed). Values in Table 11.10. 69
B.1 Main periods of work (from merge dates). The week-17 gate, with the demonstration, is 2026-11-29. 86
B.2 Commits on `main` per week, 116 in all (the first week includes the three commits of 2026-09-04). 86

List of Tables

1.1 Headline results against the success criteria of the proposal (§11.2). 3
2.1 Changes to the RuntimeDefault seccomp profiles relevant to this project. . 9
3.1 Existing tool categories compared with CALIPER. 24
5.1 Objectives of the proposal (§4) and their outcomes. 28

5.2 Success criteria fixed in the proposal (§11.2). 29
5.3 Workstreams and responsibilities. 30
5.4 Freeze gates. Work began on 2026-09-07; gate names are the proposal's week numbers, not calendar weeks. 30
6.1 Components and their responsibilities. 32
6.2 Fields of the environment cell and their sources. 33
6.3 Repository layout. 34
7.1 Fields of a probe definition (`spec/probe.md`). 35
7.2 How the child's end is read by the parent. 37
7.3 The six residual classes (`engine/crates/engine/src/residual.rs`). . . . 39
7.4 The committed corpus by family, with the isolation of each family's oracles. 40
7.5 The capability-effect probes. 42
7.6 The deferred set (`engine/crates/corpus/src/deferred.rs`). 43
7.7 Probes whose success grants the child something, and what the child does with it. 43
8.1 Top-level fields of a fingerprint, format version 1. 45
8.2 Stage 1 of attribution: the oracle's reading. 48
8.3 Tiers of posture v1. 49
8.4 The 15 assertions of posture v1 and their citations (16 in all). "untrusted" abbreviates `untrusted-multi-tenant`. 50
8.5 Closure rules of the remediator. 51
9.1 Technology stack. 53
9.2 Continuous integration and release workflows. 55
10.1 Environment cells. Machines: cell 1 `c4a-standard-2`; cells 2 to 5 `e2-standard-2`; cell 6 `n2-standard-2` with nested virtualisation; GKE `e2-medium`. 56
10.2 Single-cause environments for attribution accuracy (`fixtures/ground-truth/`). 58
11.1 Verdicts of the full corpus. Under the hardened profile the three `mount.fs.*` probes were recorded as unmeasured: the filter killed their set-up (creating the run's mount directory), which the harness reports as a `setup-killed` fault rather than a verdict. 60
11.2 Verdicts by family and cell (permitted/denied/unimplemented). *`capability.sys_rawio` is not applicable on arm64. 61
11.3 Divergent probes per cell pair and class (`fixtures/metrics/divergence-metrics.json`). 62
11.4 Findings at the `restricted` tier, and expectation violations at `untrusted-multi-tenant` (`fixtures/metrics/finding-metrics.json`). . 63
11.5 Undeclared exposures at `restricted` by family. Path includes the control probe on every cell; device includes the `/dev/null` control; socket and netlink include their out-of-range oracle probes where permitted. 64
11.6 `socket.family.af_alg` across recordings. 65
11.7 Attribution against single-cause environments (`fixtures/metrics/attribution-accuracy.json`). 65

11.8 Undecidable fraction on production cells, full corpus under RuntimeDefault, corpus 0.3.3 (GKE: 0.3.2). 66
11.9 Closure verified by re-probing (`fixtures/metrics/closure-metrics.json`). 67
11.10 Residual state and runtime (`fixtures/metrics/residual-metrics.json`). Classes: residual classes the cell exposes. Modules: loaded by the full run. 68
11.11 Same-target comparison on cell 2 (`fixtures/ancestors/cell2/`). 69
11.12 Every success criterion of the proposal (§11.2) with its measured value. . . 70
12.1 Disclosure review of runtime behaviour observed in the evaluation. 72
B.1 Milestones and their issues. The backlog was seeded from the proposal by `scripts/seed-backlog.sh`. 87
C.1 The complete probe index (`corpus/index.json`). G: guaranteed errno; A: absence errno (encoded absence after a slash); †: may autoload a module. . 88

List of Abbreviations

AF_ALG	Address family of the Linux kernel crypto user-space API (socket family 38)
API	Application Programming Interface
BPF / eBPF	Berkeley Packet Filter / extended BPF; seccomp filters are classic BPF programs
CAP_*	A Linux capability, for example CAP_SYS_ADMIN or CAP_NET_BIND_SERVICE
CI	Continuous Integration
CISA	Cybersecurity and Infrastructure Security Agency (United States)
COS	Container-Optimized OS, the node operating system of Google Kubernetes Engine
CRI	Container Runtime Interface, the gRPC interface between the kubelet and a runtime
CVE	Common Vulnerabilities and Exposures identifier
DAC	Discretionary Access Control (file owner, group and mode bits)
errno	The error number a failed system call returns to user space
GCE	Google Compute Engine (virtual machines)
GKE	Google Kubernetes Engine
GHCR	GitHub Container Registry
IPC	Inter-Process Communication (System V or POSIX)
JSON	JavaScript Object Notation
KEV	Known Exploited Vulnerabilities catalogue (CISA)
LSM	Linux Security Module, for example AppArmor or SELinux
LTO	Link-Time Optimisation
MAC	Mandatory Access Control
musl	A small C standard library used to build a static probe binary

NIST	National Institute of Standards and Technology
OCI	Open Container Initiative (image and runtime specifications)
OIDC	OpenID Connect, used here for keyless image signing
PID	Process Identifier
PSA / PSS	Pod Security Admission / Pod Security Standards (Kubernetes)
PR	Pull Request
RBAC	Role-Based Access Control (Kubernetes)
RuntimeClass	Kubernetes object selecting the runtime handler for a pod (runc, gVisor, Kata)
RuntimeDefault	The seccomp (and AppArmor) profile the container runtime ships as its default
SA	Service Account
SARIF	Static Analysis Results Interchange Format
seccomp	Secure computing mode; seccomp-BPF filters system calls by number and arguments
SELinux	Security-Enhanced Linux
SHA-256	Secure Hash Algorithm, 256-bit digest
SIGSYS	The signal delivered when a seccomp filter's action kills the caller
SPO	Security Profiles Operator (Kubernetes SIGs project)
UID / GID	User Identifier / Group Identifier
userns	User namespace
VM	Virtual Machine
VMM	Virtual Machine Monitor
YAML	YAML Ain't Markup Language

Verdicts and finding categories used throughout

permitted	The probed call reached the kernel and was not refused, or returned the error its oracle guarantees
denied	The call was refused before reaching the oracle's point (an errno other than the guaranteed one)
unimplemented	The kernel of this cell does not provide the entry point (declared absence errno)
killed	The probe's child was terminated by SIGSYS, the outcome of a seccomp kill action
timed-out	The probe exceeded its risk-class timeout and was killed by the engine
not-applicable	The entry point does not exist on this architecture; decided before anything runs
expectation violation	Reachable, and the reference posture asserts it should not be
claim violation	The manifest asserts a protection that the measurement contradicts
undeclared exposure	Reachable, and neither the manifest nor the posture addresses it

Chapter 1
Synopsis

1.1 Abstract

A container's security posture is declared in a manifest and enforced by the kernel, and the two are not the same thing. The manifest names a seccomp profile, a capability set and a Pod Security Standard. What the kernel actually permits depends additionally on the container runtime and its version, the node's distribution, kernel build and loaded modules, the processor architecture, the active Linux Security Module and the RuntimeClass, none of which the manifest records. Two clusters running an identical manifest can therefore expose materially different kernel attack surface, and nothing in a deployment pipeline reports the difference. The motivating case is CVE-2026-31431 ("Copy Fail"), a kernel privilege escalation reached through the AF_ALG socket family, which the RuntimeDefault seccomp profiles of containerd and Moby did not deny and which Pod Security Standards at the Restricted level did not address.

CALIPER is an instrument that measures enforced confinement directly. It runs as an ordinary, unprivileged container and executes a corpus of 185 argument-level reachability probes in eight families (socket address families, netlink protocols, io_uring opcodes, mount filesystem types, clone and unshare flags, masked and read-only paths, device nodes and capability effects). Each probe runs in its own forked child, so a probe killed by a seccomp filter or hung in the kernel is recorded as a measurement rather than ending the run. The result is a confinement fingerprint keyed by the *environment cell* that produced it. A Go control plane diffs fingerprints and classifies every divergence as architecture-, kernel-version-, runtime-class- or policy-explained; attributes denials to the mechanism that produced them using errno oracles, capability and security-module correlation and cell elimination; evaluates a fingerprint against the declared manifest and against a versioned, cited reference posture; and generates the minimal seccomp change that closes what it finds, verifying each closure by probing again.

The instrument was evaluated on six environment cells provisioned as Compute Engine virtual machines (arm64 and amd64, AppArmor and SELinux, containerd 1.7 and 2.2, runc, gVisor and Kata Containers) and on a Google Kubernetes Engine cluster that measures every pull request. Every success criterion set in the project proposal was met: policy-explained divergence in 12 of 15 cell pairs; expectation violations in at least three cells; the AF_ALG case reproduced with an unmodified probe image and shown present on one node and absent on another under the same manifest; 220 of 220 attributed denials correct with 1.3 per cent undecidable on a constructed ground truth; 75 of 75 seccomp-attributable findings closed and verified; no residual objects after any run; and a slowest full run of 8.6 seconds against a 60-second budget. Two results are weaker than those figures suggest and are stated openly: on the six production cells between 21.6 and 85.0 per cent of denials remain undecidable, and the three-cell expectation criterion at the `restricted` tier depends on a single finding of doubtful weight. The evaluation also observed two genuine runtime defects, one in gVisor and one in Kata Containers. Both were public upstream before this report, and neither was reported by this project.
Keywords: container security, Linux kernel, seccomp, confinement measurement, Kubernetes, attack surface, gVisor, Kata Containers, errno oracle.

1.2 Problem Statement

Given a container, on a particular node, at a particular moment, *what can it actually reach in the kernel*, and why does that differ from what an identically configured container reaches elsewhere? Existing tools answer neighbouring questions. Configuration scanners report what a manifest declares; runtime monitors report what a workload did, which is a subset of what it could have done; profile generators report what an application needs, not what its environment permits. None measures enforced confinement, and none separates a genuine policy difference from a difference that the processor architecture, the kernel version or a sandboxed runtime explains. The project's problem is to build an instrument that does both, from inside an unprivileged container, without damaging the node it measures.

1.3 Motivation in Brief

The AF_ALG case shows the failure concretely. The vulnerability is reached through a socket address family rather than a distinctive system call, so a filter at system-call-name granularity cannot express it. Published testing found that workloads which satisfied every conventional hardening check were exploitable, and establishing that fact required hand-built test images and a hand-written profile. Whether a given node is exposed depends on the runtime version installed on it: containerd added an AF_ALG deny to its default profile in May 2026, and a node running an earlier version enforces a weaker policy than an identically configured node running a later one. The same happened with io_uring in 2023. The declared posture is therefore an assertion, and in current practice it is unverified. Chapter 2 develops the background and the motivating case in full.

1.4 Objectives

The proposal set eight objectives, all of which were delivered (Table 5.1, Chapter 5):

1. a probe engine with fork isolation, bounded and accounted side effects and precise verdicts;
2. an argument-granular probe corpus with a committed set of eight families and a specified deferred set;
3. denial attribution based principally on errno oracles, with a reported confidence;
4. a stable, environment-keyed fingerprint format and a four-class divergence classifier;
5. a versioned, cited reference posture;
6. an evaluator that separates claim violations, expectation violations and undeclared exposures;
7. a remediation generator emitting minimal argument-level seccomp policy, with closure verified by re-probing;
8. an evaluation across at least six environment cells that reproduces the AF_ALG case unaided.

1.5 Summary of Contributions

- **A measurement engine** written in Rust as a static musl binary in a scratch image. It forks a child per probe, waits on a process descriptor with a hard deadline, survives seccomp kill actions, snapshots six classes of residual kernel state around every probe and the loaded module set around the whole run, and is checked in CI against a committed baseline of its own system calls.

- **A corpus of 185 probes**, each authored with an errno oracle that states what error the kernel is structurally guaranteed to return if the call reaches it, which enforcement mechanisms that oracle isolates, the kernel dependency of the entry point, its capability requirement, its applicability and its side effects. A further 28 probes in nine families are specified in the same format and deliberately not run.
- **A divergence classifier** that assigns each difference between two fingerprints exactly one of four classes, so that architecture, kernel-version and sandbox effects are never reported as policy.
- **An attributor** that combines errno oracles, capability and LSM correlation and elimination by the cell's own observable state, and reports what it cannot decide as a measured fraction rather than assuming it is zero.
- **A reference posture** (v1) of four tiers and fifteen assertions, each with a rationale and at least one citation, released with the instrument.
- **An evaluator and remediator** that produce a Localhost seccomp profile and a Security Profiles Operator resource from the runtime's actual default profile, and count a finding as closed only when a fresh measurement shows it denied.
- **A reproducible evaluation**: six provisioned cells with pinned package versions, a GKE cluster evaluating every pull request, a signed multi-architecture release (v1.0.0), and committed fingerprints from which every number in Chapter 11 is regenerated by a single command.

1.6 Summary of Results

Table 1.1 lists the headline measurements against the proposal's success criteria. Each is discussed, with its caveats, in Chapter 11.

Criterion	Threshold	Measured	Met
Policy-explained divergence	≥ 4 cell pairs	12 of 15 pairs; architecture and runtime class separated in every pair	yes
Expectation violations	≥ 3 cells	3 cells at `restricted` (one rests on a single doubtful finding); 6 cells at `untrusted-multi-tenant`	yes
AF_ALG reproduction	binary	reproduced with the standard Job and image	yes
Attribution accuracy	$\geq 90\%$	220 of 220 attributed denials correct	yes
Undecidable fraction	$\leq 10\%$	1.3% on the ground truth; 21.6–85.0% on production cells	yes
Seccomp closure	100%	75 of 75 verified; other mechanisms 0 of 63	yes
Residual state	0 objects	0 unreclaimed, 0 leaks on every cell	yes
Runtime	60 s full, 5 s reduced	8.6 s and 2.9 s (gVisor, slowest)	yes

Table 1.1: Headline results against the success criteria of the proposal (§11.2).

1.7 Organisation of the Report

Chapter 2 gives the technical background on Linux container confinement and develops the motivating case. Chapter 3 surveys the research literature and existing tools paper by

paper, and Chapter 4 distils the gaps they leave. Chapter 5 states the proposed solution, its objectives, design principles and success criteria. Chapter 6 presents the architecture. Chapters 7 and 8 describe the methodology: the measurement side (probe specification, engine, residual accounting and corpus) and the analysis side (attribution, fingerprint, classifier, posture, evaluator, remediator and reporting). Chapter 9 lists the technology stack, and Chapter 10 the experimental setup. Chapter 11 reports and analyses every result. Chapter 12 records the responsible-use and disclosure review, Chapter 13 discusses what the compressed schedule gave up, the limitations and the threats to validity, and Chapter 14 concludes. The appendices give the reproduction commands, the project record and the complete probe index.

Chapter 2
Background and Motivation

This chapter describes the Linux mechanisms that together confine a container, how Kubernetes and the container runtimes configure them, and why the configuration a workload author writes does not determine what the kernel enforces. It closes with the motivating case and the problem that follows from it.

2.1 Containers Share a Kernel

A container is not a virtual machine. It is an ordinary group of processes on the host kernel, given a restricted view of the system by namespaces, restricted resources by control groups, and restricted kernel access by a stack of independent enforcement mechanisms. Every system call the workload makes is served by the same kernel that serves every other tenant on the node. A vulnerability in any kernel code path that the workload can reach is therefore a potential escape to the host, and the security of a container is, to a first approximation, the size and content of the kernel surface it can reach. Lin et al. showed with 88 working exploits that the kernel mechanisms which restrict reachable surface (capabilities, seccomp and mandatory access control) prevent more privilege escalations than the isolation mechanisms (namespaces and cgroups) do [1]. Measuring that reachable surface is the subject of this project.

2.2 The Enforcement Mechanisms

2.2.1 Namespaces and control groups

Linux provides eight namespace types: mount, PID, network, IPC, UTS, user, cgroup and time. Each gives the process its own instance of a global resource. A container runtime creates a fresh set for each pod (Kubernetes shares the network and IPC namespaces among a pod's containers). Creating a namespace from inside a container is itself a privileged operation: `clone(2)` and `unshare(2)` with a `CLONE_NEW*` flag require CAP_SYS_ADMIN, except for a new user namespace, which an unprivileged process may create where the kernel and the distribution allow it, and inside which it holds a full capability set. Control groups (cgroups v2 on every cell measured here) limit and account resources; they confine resource use, not kernel reachability, and Gao et al. showed that their accounting can be escaped by work the kernel performs on a process's behalf outside its group [2].

2.2.2 Capabilities

Root's privileges are split into roughly forty capabilities, each gating a family of operations: CAP_SYS_ADMIN for mounts and most namespace creation, CAP_NET_ADMIN for network configuration, CAP_SYS_MODULE for module loading, CAP_NET_BIND_SERVICE for ports below 1024, and so on [3]. A process carries four sets that matter here: *effective* (checked by the kernel), *permitted*, *bounding* (the ceiling across `execve`) and *ambient*. All four are readable without privilege from `/proc/self/status`. A hardened workload drops every capability. A capability check usually sits deep inside the system call, after argument validation and after the security-module hook, which matters for attribution (Section 2.3).

2.2.3 seccomp-BPF

seccomp in filter mode attaches one or more classic BPF programs to a process. At every system call entry, before the kernel does any work, each program inspects the architecture, the system call number and the six argument registers, and returns an action: `ALLOW`; `ERRNO` with a chosen error, which makes the call fail immediately; `KILL_PROCESS` or `KILL_THREAD`, which terminate the caller with SIGSYS; `TRAP`; `LOG`; or `USER_NOTIF`, which defers the decision to a supervisor [4]. Filters are inherited across `fork` and `execve` and cannot be removed. Two properties shape this project. First, a filter sees register values only: it can compare a socket family or a flag word, but it cannot dereference a pointer, so it cannot inspect a pathname, a filesystem type string or a structure passed by address. Second, the filter's contents are not readable by the confined process; `/proc/self/status` exposes only the `Seccomp` mode (0 none, 1 strict, 2 filter) and the `Seccomp_filters` count. Reading a filter needs `PTRACE_SECCOMP_GET_FILTER` and CAP_SYS_ADMIN.

2.2.4 Linux Security Modules

AppArmor and SELinux are mandatory access control modules that place hooks at security-relevant points inside the kernel: socket creation, file open, mount, ptrace and many more. The container runtime loads a profile (AppArmor, by path) or assigns a context (SELinux, by type and category) to the container. A process can read its own label from `/proc/self/attr/current` (and the AppArmor-specific `/proc/self/attr/apparmor/current`) and can tell whether the profile is enforcing. Yama and kernel lockdown are further, label-less modules that can refuse ptrace and raw hardware access respectively. Sun et al. observed that because these frameworks are global to the kernel, containers cannot apply policies of their own [5]; for this project the consequence is that the LSM in force is a property of the node, not of the workload.

2.2.5 Masked and read-only paths

The OCI runtime specification lets a runtime mask paths (`maskedPaths`) and mount others read-only (`readonlyPaths`) [6]. runc implements a mask of a file as a bind mount of `/dev/null` over it and a mask of a directory as an empty read-only tmpfs. The kubelet requests a fixed default set for every container: masked `/proc/acpi`, `/proc/asound`, `/proc/kcore`, `/proc/keys`, `/proc/latency_stats`, `/proc/timer_list`, `/proc/timer_stats`, `/proc/sched_debug`, `/proc/scsi`, `/sys/firmware` and `/sys/devices/virtual/powercap`; read-only `/proc/bus`, `/proc/fs`, `/proc/irq`, `/proc/sys` and `/proc/sysrq-trigger`. Gao et al. demonstrated that unmasked procfs and sysfs files leak host-wide information to containers [7]; the masked set is the runtime's partial answer.

2.2.6 Other controls

`no_new_privs` prevents a process from gaining privilege through set-user-ID binaries or file capabilities. A read-only root filesystem and a non-root UID reduce what a compromised process can modify. Device access is controlled by the device cgroup (or its eBPF equivalent in cgroups v2) and by the device nodes the runtime creates. Each of these is configured independently and enforced at a different point in the kernel.

2.3 Where Each Mechanism Acts

The mechanisms act at different points along a system call's path, and the order is fixed by the kernel's code. Figure 2.1 shows the order for a typical call. seccomp acts first, at system call entry. The kernel then validates arguments and resolves descriptors; only

then do the LSM hook and, usually later still, the capability check run. A call that is refused returns an errno, and `EPERM` is the conventional answer of all four mechanisms. The kernel does not say which one produced it.

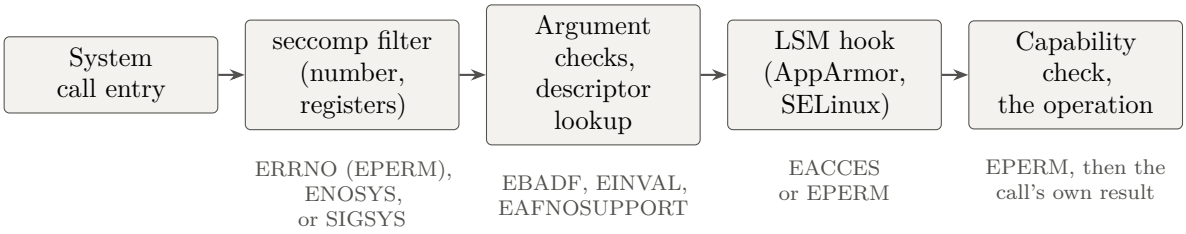


Figure 2.1: The order in which confinement mechanisms act on a typical system call, and the errors each can produce. The exact order is per system call and is recorded with every probe.

This ordering is what makes attribution possible without privilege. If a call is built so that, once it reaches the kernel, the argument checks are guaranteed to fail with a specific error (an invalid descriptor gives `EBADF`), then receiving `EPERM` instead proves the call was stopped *before* the argument checks, which only a seccomp filter can do. Chapter 8 develops this into the errno-oracle technique. The ordering is not uniform: the hook for `socket(2)` runs before the family is looked up, and from Linux 6.15 `io_uring_setup(2)` calls an LSM hook before checking its parameters, so the same reasoning separates seccomp from a capability but not from an LSM for those calls, and does so on some kernels and not on others.

2.4 How Kubernetes and the Runtimes Configure Confinement

2.4.1 Pod security context and Pod Security Standards

A Kubernetes pod declares its confinement in its security context: the UID, `allowPrivilegeEscalation`, added and dropped capabilities, a `seccompProfile` of type `RuntimeDefault`, `Localhost` or `Unconfined`, an AppArmor profile, a read-only root filesystem and, optionally, `hostUsers: false` for a user namespace. Pod Security Standards define three levels that admission enforces per namespace [8]. *Privileged* is unrestricted. *Baseline* forbids known privilege escalations: privileged containers, host namespaces, host paths and added capabilities beyond a small set. *Restricted* additionally requires a non-root user, dropping `ALL` capabilities, no privilege escalation and a seccomp profile of `RuntimeDefault` or `Localhost`. Restricted is the strongest level Kubernetes defines.

2.4.2 RuntimeDefault is defined by the runtime

`RuntimeDefault` does not name a profile; it names whatever the runtime on the node ships as its default. For containerd [9] that is the profile compiled into the containerd binary; for Moby it is Moby's `default.json`. Both are allowlists with a default action of `ERRNO(EPERM)`, conditional rules for `clone`, `personality` and `socket`, and an explicit `ENOSYS` for `clone3` so that C libraries fall back to `clone`, whose flags a filter can inspect. The AppArmor default (`cri-containerd.apparmor.d` for containerd) is likewise compiled into the runtime. The contents change between releases, and a node's runtime version is chosen by its operating system image, not by the workload author.

2.4.3 RuntimeClass: runc, gVisor and Kata

A RuntimeClass selects the low-level runtime handler for a pod. *runc* configures the mechanisms above on the host kernel. *gVisor* (`runsc`) interposes a user-space kernel, the Sentry, which implements the Linux system call interface itself; the Sentry runs under its own tight seccomp filter, reaches the host through a small set of system calls, and has file access mediated by a separate Gofer process. The workload's system calls never reach the host kernel directly, many interfaces are not implemented, and the version the workload sees is synthetic [10, 11]. *Kata Containers* runs each pod in a lightweight virtual machine with its own guest kernel; an agent inside the guest applies the OCI specification there [12]. Anjali et al. found that, despite moving functionality out of the host kernel, gVisor and Firecracker-style microVMs exercise substantially more host kernel code than native containers [13]. A measurement under either sandbox is a measurement of the sandbox's interface, and comparing it with a runc measurement without saying so would report the sandbox as policy.

2.5 The Motivating Case: AF_ALG and Copy Fail

CVE-2026-31431, tracked as Copy Fail, is a logic flaw in the `algif_aead` module, which implements authenticated encryption for the kernel crypto user API reached through the AF_ALG socket family. Combined with `splice(2)`, it writes four bytes into the page cache of any readable file, enough to modify a set-user-ID binary in memory without changing it on disk. CISA added it to the Known Exploited Vulnerabilities catalogue on 1 May 2026 [14, 15], and vendors described it as a root privilege escalation across cloud environments [16, 17]. Three properties of the case motivate this project.

1. **It is reached through an argument, not a system call.** Any container that may call `socket(2)` can attempt it; what matters is the value of the first argument. Policy generated or reviewed at system-call-name granularity cannot express the difference between a workload that may create TCP sockets and one that may reach the kernel crypto API.
2. **Conventional hardening did not prevent it.** Published testing found that the RuntimeDefault seccomp profiles of both containerd and Moby allowed AF_ALG and that Pod Security Standards at Restricted did not address it, so workloads that satisfied every conventional check were exploitable. Establishing that required hand-built test images and a hand-written profile [18].
3. **Exploitability depends on the node.** The flaw is in a module. Whether `algif_aead` is loaded, or can be autoloaded by an unprivileged socket call, is a property of the distribution's kernel configuration and module policy, which the workload author neither controls nor observes. The kernel loads the module for an unregistered socket family on request, without a capability check on the caller.

2.6 RuntimeDefault Drift

The AF_ALG case is one instance of a general problem: the protection RuntimeDefault provides is a property of the runtime binary on the node and changes over time. Table 2.1 records the two changes that matter for this project. io_uring, which performs file and network operations from a submission queue without issuing the corresponding system calls and so sidesteps system-call filtering, was removed from both default profiles in November 2023; the change reached containerd only in 2.0 and was not backported to 1.7. AF_ALG followed in May 2026. Ubuntu 24.04 ships containerd 1.7.12 in its release

pocket and 2.2 in its updates pocket, so an unpatched and a patched host of the same release sit on either side of the io_uring change. Two nodes running an identical, admitted manifest can therefore enforce different policies, and nothing in the deployment pipeline reports the difference.

Interface	Change	Merged	First release carrying it
io_uring	Moby PR 46762: block `io_uring_*` in the default profile [19]	November 2023	Moby 25.0, January 2024
io_uring	containerd PR 9320: disallow io_uring in RuntimeDefault [20]	November 2023	containerd 2.0, November 2024; not backported to 1.7
AF_ALG	Moby PR 52501: block AF_ALG sockets (CVE-2026-31431) [21]	May 2026	Moby 29.4.2
AF_ALG	containerd PR 13327: deny AF_ALG in RuntimeDefault [22]	May 2026	containerd 2.2.4; backported to 1.7.32

Table 2.1: Changes to the RuntimeDefault seccomp profiles relevant to this project.

2.7 Why Existing Practice Does Not Detect It

Configuration review compares a manifest with a benchmark. In the AF_ALG case the manifest was correct by every benchmark, and the enforcement was weaker than practitioners believed; a configuration scanner reports no finding. Runtime monitoring observes the workload's system calls as it makes them, and an attack surface that a benign workload never exercises is invisible to it. Profile generation traces an application and emits the system calls it used, which answers what the application needs and not what the environment permits, and does so at system-call-name granularity. A measurement from inside the container, at argument granularity, is the missing piece, and it needs four further things to be useful: an identity for the environment it was taken in, a way to separate policy from architecture and sandbox effects, a way to say which mechanism produced each denial, and a model of what ought not to be reachable. Those are the components of CALIPER.

Chapter 3
Literature Survey

This chapter reviews the published work closest to the project's question, paper by paper, followed by the tools practitioners use today. For each paper it gives the publication details, the problem the paper addresses, its proposed solution, its results, the gap or limitation relevant here, and its relevance to CALIPER. The 28 papers fall into five groups: measurement and benchmarking of container confinement (Sections 3.1–3.4), system call policy generation (Sections 3.5–3.10), access control and manifest analysis (Sections 3.11–3.14), sandboxed runtimes and kernel attack surface (Sections 3.15–3.18), and surveys and guidance on container security (Sections 3.19–3.27). With the three journal extensions discussed inside those sections, the chapter cites fifteen journal or transactions papers; the rest appeared at reputed conferences.

3.1 Paper 1: Houdini, Benchmarking Container Security Confinement

Publication Details:

- **Authors:** Huzaifa Patel, David Barrera, Anil Somayaji
- **Venue:** 2025 22nd Annual International Conference on Privacy, Security and Trust (PST), pp. 1–10, August 2025
- **DOI:** 10.1109/PST65910.2025.11268879

Introduction: Container workloads are a standard part of cloud infrastructure, yet there is no easy way to verify whether a given container configuration provides even a basic level of confinement. Without confinement a single vulnerable application can be used to compromise a whole cluster [23].

Proposed Solution: Houdini is a security test suite for container confinement. It runs a set of tests against a container configuration and is optimised for comparing confinement technologies with one another, while remaining usable against deployable containers.

Results: The paper presents the motivation, design and implementation, and initial results of running the suite on a set of Docker container configurations, positioning Houdini as a benchmark framework for evaluating confinement technologies.

Gap / Limitation: The evaluation covers Docker configurations; the paper does not key its results to the full environment (architecture, kernel, runtime version, LSM, RuntimeClass), does not separate policy differences from architecture or sandbox differences, does not say which mechanism produced a refusal, and does not generate or verify remediation. We found no released tool.

Relevance: Houdini is the closest published statement of the problem and the reason this project presents itself as an engineering contribution rather than a new idea. CALIPER differs in granularity (arguments rather than tests), in identity (the environment cell), and in carrying the result through classification, attribution, evaluation against a cited posture, and verified remediation.

3.2 Paper 2: A Measurement Study on Linux Container Security

Publication Details:

- **Authors:** Xin Lin, Lingguang Lei, Yuewu Wang, Jiwu Jing, Kun Sun, Quan Zhou
- **Venue:** Proceedings of the 34th Annual Computer Security Applications Conference (ACSAC '18), pp. 418–429, December 2018
- **DOI:** 10.1145/3274694.3274720

Introduction: It is widely agreed that containers are less secure than virtual machines because they share the kernel, but there had been no concrete, systematic evaluation using real exploits [1].

Proposed Solution: The authors collected 223 exploits effective on container platforms, classified them with a two-dimensional attack taxonomy, and evaluated the default Linux container configuration against 88 typical exploits.

Results: 50 of the 88 exploits (56.82 per cent) succeeded from inside a container with the default configuration. Kernel security mechanisms (capabilities, seccomp and mandatory access control) prevented more privilege escalations than the isolation mechanisms (namespaces and cgroups), but their interdependence produced a "short board effect". The 11 exploits that broke isolation followed a common four-step attack model, against which the authors proposed a defence.

Gap / Limitation: The study measures one default configuration with exploits, which answers "is this exploit blocked" rather than "what is reachable". It cannot be run safely against a production node, and it does not compare environments.

Relevance: The paper's central finding, that the reachable-surface mechanisms matter most, justifies measuring reachability. CALIPER measures the same mechanisms without exploit code and records which of them refused each probe.

3.3 Paper 3: ContainerLeaks, Information Leakage in Container Clouds

Publication Details:

- **Authors:** Xing Gao, Zhongshu Gu, Mehmet Kayaalp, Dimitrios Pendarakis, Haining Wang
- **Venue:** 47th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN), pp. 237–248, June 2017
- **DOI:** 10.1109/DSN.2017.49

Introduction: Because the kernel's resource isolation is incompletely implemented, containers sharing a kernel in a multi-tenant cloud can observe system-wide host information [7].

Proposed Solution: The authors identified information leakage channels reachable from inside containers, showed how the leaked information eases advanced attacks such as co-residence and synergistic power attacks, analysed the root causes, and proposed a two-stage defence.

Results: The channels exposed a spectrum of host information without proper partitioning, and the defence was effective at trivial performance overhead. An extended journal version in *IEEE Transactions on Dependable and Secure Computing* (vol. 18, no. 1, pp. 174–191, 2021) shows that the leaked information can be used to infer private data, detect and verify co-residence, and build covert channels [24].

Gap / Limitation: The work identifies channels on the platforms studied; it does

not provide a way for an operator to check, on a given node, whether the paths that the runtime is supposed to mask actually are masked.

Relevance: The masked-path family of the CALIPER corpus measures exactly this. In the evaluation, Kata Containers left two directories of the kubelet's masked set reachable, a real defect (Chapter 12).

3.4 Paper 4: Houdini's Escape, Breaking the Resource Rein of Linux Control Groups

Publication Details:

- **Authors:** Xing Gao, Zhongshu Gu, Zhengfa Li, Hani Jamjoom, Cong Wang
- **Venue:** Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security (CCS '19), pp. 1073–1086, November 2019
- **DOI:** 10.1145/3319535.3354227

Introduction: cgroups partition processes into groups and apply resource controllers, and child processes inherit their parent's group. The authors show that inherited confinement does not guarantee consistent and fair accounting [2].

Proposed Solution: A set of strategies that generate out-of-band workloads by de-associating work from the originating process group, so that the resources it consumes are not charged to the right cgroup, demonstrated in five case studies within Docker.

Results: On a local testbed and an Amazon EC2 dedicated server, a container consumed up to 200 times its CPU limit and reduced the computing and I/O performance of co-resident containers by up to 95 per cent.

Gap / Limitation: The attacks concern resource accounting rather than kernel reachability, and they depend on kernel paths that a workload can trigger; the paper does not provide a way to check which of those paths a given environment exposes.

Relevance: The paper shows that confinement failures arise from kernel paths a container can reach indirectly. CALIPER's posture includes entry families (io_uring, user namespaces, raw netlink) on the same reasoning, and its residual accounting deliberately tracks cgroup directories among the six classes of state a probe may leave behind.

3.5 Paper 5: Confine, Automated System Call Policy Generation for Containers

Publication Details:

- **Authors:** Seyedhamed Ghavamnia, Tapti Palit, Azzedine Benameur, Michalis Polychronakis
- **Venue:** 23rd International Symposium on Research in Attacks, Intrusions and Defenses (RAID 2020), USENIX Association, October 2020; extended as "Confine: Fine-grained system call filtering for container attack surface reduction" by M. Rostamipoor, S. Ghavamnia and M. Polychronakis, *Computers & Security*, vol. 132, 103325, 2023
- **DOI (extended version):** 10.1016/j.cose.2023.103325

Introduction: Default container seccomp profiles are generic: they allow every system call any reasonable workload might need, leaving most containers with far more kernel surface than they use [25, 26].

Proposed Solution: Confine derives a restrictive system call policy for a container automatically, by statically analysing the binaries and libraries in the container image, including the C library, to identify the system calls the container can issue, and emits a

seccomp profile that denies the rest.

Results: Applied to a large set of popular Docker Hub images, the generated policies disabled a substantial share of the system call interface for most images and would have neutralised a number of previously disclosed kernel vulnerabilities.

Gap / Limitation: Confine answers what an application *needs*. It does not measure what the environment *permits*, and its policies are at system-call granularity (the 2023 extension adds argument-level filtering for selected calls). It does not account for capabilities, LSM policy, masked paths or runtime differences.

Relevance: Confine is the strongest example of the profile-generation approach that this project explicitly places out of scope. CALIPER's remediator is minimal with respect to a reference posture and the runtime's own default profile, not with respect to an application's needs.

3.6 Paper 6: sysfilter, Automated System Call Filtering for Commodity Software

Publication Details:

- **Authors:** Nicholas DeMarinis, Kent Williams-King, Di Jin, Rodrigo Fonseca, Vasileios P. Kemerlis
- **Venue:** 23rd International Symposium on Research in Attacks, Intrusions and Defenses (RAID 2020), USENIX Association, pp. 459–474, October 2020
- **URL:** `https://www.usenix.org/conference/raid2020/presentation/demarinis`

Introduction: An attacker who compromises an application can abuse every operating-system service the process may call, and the kernel exposes its whole system call set to every process [27].

Proposed Solution: sysfilter is a binary-analysis framework for x86-64 Linux that builds the set of system calls a program can make through static analysis and restricts the process to that set.

Results: Evaluated on 411 real C and C++ binaries with about 38,500 tests, sysfilter imposed minimal or negligible run-time slowdown, and a study of about 30,000 Debian applications characterised their system call profiles.

Gap / Limitation: As with Confine, the analysis concerns the application; it says nothing about how a given environment's policy differs from another's, and it filters by system call number.

Relevance: sysfilter and Confine both show that system-call-granularity policy is the norm. The AF_ALG case is invisible at that granularity, which is why CALIPER's corpus is argument-level and its remediation uses argument comparisons.

3.7 Paper 7: Temporal System Call Specialization for Attack Surface Reduction

Publication Details:

- **Authors:** Seyedhamed Ghavamnia, Tapti Palit, Shachee Mishra, Michalis Polychronakis
- **Venue:** 29th USENIX Security Symposium (USENIX Security '20), pp. 1749–1766, August 2020
- **URL:** `https://www.usenix.org/conference/usenixsecurity20/presentation/ghavamnia`

Introduction: Server applications need more system calls during initialisation than while serving requests, so a single policy for the whole lifetime is broader than necessary [28].

Proposed Solution: Temporal specialisation limits the system calls available to a process according to its phase of execution, using new static call-graph analyses per phase, implemented on LLVM.

Results: On six server applications it disabled 51 per cent more security-critical system calls than library-level specialisation and neutralised 13 more Linux kernel vulnerabilities that could lead to privilege escalation.

Gap / Limitation: The technique requires source-level analysis of the application and again targets application needs. The same group later proposed C2C (ACM CCS 2022), which derives system call filters from an application's configuration options [29]; it, too, tailors policy to the application and assumes the filter is enforced as written.

Relevance: The paper quantifies kernel surface in terms of security-critical system calls and the kernel vulnerabilities they reach, the same unit of argument CALIPER's reference posture uses when it cites a CVE for each asserted entry family.

3.8 Paper 8: Automating Seccomp Filter Generation for Linux Applications

Publication Details:

- **Authors:** Claudio Canella, Mario Werner, Daniel Gruss, Michael Schwarz
- **Venue:** Proceedings of the 2021 Cloud Computing Security Workshop (CCSW '21), pp. 139–151, November 2021
- **DOI:** 10.1145/3474123.3486762

Introduction: seccomp filters reduce the impact of exploits, but writing them by hand for each application is laborious, and earlier automated approaches make unnecessary assumptions or need very long analysis [30].

Proposed Solution: Chestnut generates strict filters in two phases: static extraction of used system calls through a compiler pass and a binary analyser, and an optional dynamic refinement.

Results: On 18 applications Chestnut blocked on average 302 system calls (86.5 per cent) with the compiler and 288 (82.5 per cent) with binary analysis, was up to 73 times faster than prior compiler-based work, blocked `exec` in 50 and 77.7 per cent of applications respectively, and blocked exploitation of more than 61 per cent of 175 kernel CVEs reached through system calls.

Gap / Limitation: The approach is application-centric, works at system call granularity and assumes the filter is enforced as written; it does not verify enforcement in the deployed environment.

Relevance: Chestnut shows how much surface a default profile leaves open. CALIPER's closure verification addresses the assumption Chestnut leaves implicit: a finding counts as closed only when a fresh measurement under the generated profile shows the probe denied.

3.9 Paper 9: Mining Sandboxes for Linux Containers

Publication Details:

- **Authors:** Zhiyuan Wan, David Lo, Xin Xia, Liang Cai, Shanping Li

- **Venue:** 2017 IEEE International Conference on Software Testing, Verification and Validation (ICST), pp. 92–102, March 2017
- **DOI:** 10.1109/ICST.2017.16

Introduction: Attacks against containers often use kernel exploits through the system call interface [31].

Proposed Solution: The authors explore a container's behaviour with automatic testing, record the system calls accessed, and use that set as the container's sandbox, restricting system calls not seen during testing.

Results: Mining took less than eleven minutes per container; the mined sandboxes did not impair regular functionality and imposed low overhead.

Gap / Limitation: The approach was later extended into a journal article on practical and effective sandboxing for Linux containers (*Empirical Software Engineering*, vol. 24, no. 6, pp. 4034–4070, 2019) [32]. Dynamic mining is bounded by test coverage, works at system call granularity, and describes the workload rather than the environment.

Relevance: The paper is an early instance of trace-based profile generation, the category to which the Security Profiles Operator's recorder and Inspektor Gadget belong (Section 3.28).

3.10 Paper 10: SPEAKER, Split-Phase Execution of Application Containers

Publication Details:

- **Authors:** Lingguang Lei, Jianhua Sun, Kun Sun, Chris Shenefiel, Rui Ma, Yuewu Wang, Qi Li
- **Venue:** Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA 2017), Lecture Notes in Computer Science, Springer, pp. 230–251, 2017
- **DOI:** 10.1007/978-3-319-60876-1_11

Introduction: A container's system call needs differ between the phase in which it starts up and the phase in which it serves its workload [33].

Proposed Solution: SPEAKER separates a container's execution into booting and running phases and applies a different, narrower system call set in the running phase.

Results: The paper reports that the running phase needs considerably fewer system calls than the default profile allows, so splitting the phases reduces the reachable kernel surface at small cost.

Gap / Limitation: Like the other generators, SPEAKER tailors policy to the workload and assumes enforcement; it does not measure what a deployed environment enforces.

Relevance: SPEAKER, like Paper 7, shows that the useful unit of policy is narrower than a whole system call set. CALIPER's measurements are at argument granularity for the same reason.

3.11 Paper 11: Lic-Sec, an Enhanced AppArmor Docker Security Profile Generator

Publication Details:

- **Authors:** Hui Zhu, Christian Gehrmann
- **Venue:** Journal of Information Security and Applications, vol. 61, 102924, September 2021

- **DOI:** 10.1016/j.jisa.2021.102924

Introduction: Mandatory access control can protect containers without manual configuration if profiles are generated automatically [34].

Proposed Solution: Lic-Sec combines Docker-sec (a 2018 profile generator) with a modified LiCShield (2015) to generate AppArmor profiles for Docker containers.

Results: Against an exploit database of 40 exploits effective on Docker, selected from the latest 400 on Exploit-DB, Lic-Sec protected against all privilege escalation attacks for which Docker-sec and LiCShield failed.

Gap / Limitation: The evaluation is exploit-based on one runtime; the generated policy's enforcement across distributions (AppArmor versus SELinux) is outside its scope.

Relevance: The paper shows that the LSM can be decisive. CALIPER's cell 3 replaces AppArmor with SELinux to measure the effect of that axis, and its attributor reads the process's own label to decide whether an LSM can have produced a denial.

3.12 Paper 12: Security Namespace, Linux Security Frameworks for Containers

Publication Details:

- **Authors:** Yuqiong Sun, David Safford, Mimi Zohar, Dimitrios Pendarakis, Zhongshu Gu, Trent Jaeger
- **Venue:** 27th USENIX Security Symposium (USENIX Security '18), August 2018
- **URL:** `https://www.usenix.org/conference/usenixsecurity18/presentation/sun`

Introduction: Because the kernel is shared, containers cannot apply local policies for integrity measurement or mandatory access control, unlike virtual machines [5].

Proposed Solution: Security namespaces, a kernel abstraction that lets a container define and apply its own security policy to its processes, with a routing mechanism that dispatches each operation to every container whose security it may affect, so that one container's decision cannot compromise the host or other containers.

Results: Prototypes for integrity measurement and mandatory access control mitigated in-container security problems with less than 0.7 per cent additional system call latency and almost identical application throughput.

Gap / Limitation: The proposal changes the kernel and was not merged upstream; the global, node-level nature of LSM policy remains.

Relevance: Because LSM policy is global, it is a property of the node and belongs in the fingerprint's environment cell. CALIPER records the active LSM, the label and the enforcing state for that reason.

3.13 Paper 13: Security Misconfigurations in Open Source Kubernetes Manifests

Publication Details:

- **Authors:** Akond Rahman, Shazibul Islam Shamim, Dibyendu Brinto Bose, Rahul Pandita
- **Venue:** ACM Transactions on Software Engineering and Methodology, vol. 32, no. 4, pp. 1–36, 2023
- **DOI:** 10.1145/3579639

Introduction: Kubernetes is the de facto orchestrator, so security misconfigurations

in its manifests can cause large-scale breaches [35].

Proposed Solution: An empirical study of 2,039 Kubernetes manifests mined from 92 open-source repositories, and a static analysis tool, SLI-KUBE, that quantifies how often each misconfiguration occurs.

Results: The authors identify 11 categories of security misconfiguration, among them absent resource limits, an absent `securityContext` and activation of `hostIPC`.

Gap / Limitation: Linting reads the declaration. In the AF_ALG case the manifest was correctly hardened by every rule of this kind, and the exposure came from the runtime version on the node.

Relevance: The two approaches are complementary: lint the manifest, then measure what the node actually enforces for it. CALIPER's evaluator uses the manifest only for claims it can check against a measurement.

3.14 Paper 14: Understanding the Security Implications of Kubernetes Networking

Publication Details:

- **Authors:** Francesco Minna, Agathe Blaise, Filippo Rebecchi, Balakrishnan Chandrasekaran, Fabio Massacci
- **Venue:** IEEE Security & Privacy, vol. 19, no. 5, pp. 46–56, 2021
- **DOI:** 10.1109/MSEC.2021.3094726

Introduction: Orchestrators such as Kubernetes make it easy to deploy microservice applications, but their network abstractions behave differently from traditional networks [36].

Proposed Solution: An analysis of Kubernetes networking from a security standpoint, showing how its abstractions can enable attacks that a traditional network-security mental model does not anticipate.

Results: The authors argue that the abstractions pave the way for "unexpected attacks" and that cluster network security must be reasoned about in Kubernetes' own terms.

Gap / Limitation: The work concerns the network layer, not the kernel interface a workload can reach.

Relevance: It makes the same point at a different layer: the abstraction an operator configures is not a reliable picture of what is enforced, so it has to be examined directly.

3.15 Paper 15: The True Cost of Containing, a gVisor Case Study

Publication Details:

- **Authors:** Ethan G. Young, Pengfei Zhu, Tyler Caraza-Harter, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau
- **Venue:** 11th USENIX Workshop on Hot Topics in Cloud Computing (HotCloud '19), July 2019
- **URL:** `https://www.usenix.org/conference/hotcloud19/presentation/young`

Introduction: gVisor is a security-oriented container engine that integrates with Docker and implements the kernel interface in user space [10].

Proposed Solution: A performance analysis of gVisor covering network throughput through its in-Sentry network stack, file opens through the Gofer, start-up, memory

efficiency and system call overheads.

Results: The analysis identified substantial overheads in several of these areas and drew implications for the design of hypervisor-based container engines.

Gap / Limitation: The study is about performance; it does not characterise which kernel interfaces gVisor exposes or how its confinement differs from runc's.

Relevance: The slowest CALIPER run is on gVisor (8.6 seconds against 0.1 to 0.6 seconds elsewhere), consistent with the system call overheads the paper reports. CALIPER characterises the other half: which entry points gVisor implements, and where its behaviour differs from Linux.

3.16 Paper 16: Blending Containers and Virtual Machines, Firecracker and gVisor

Publication Details:

- **Authors:** Anjali, Tyler Caraza-Harter, Michael M. Swift
- **Venue:** Proceedings of the 16th ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments (VEE '20), pp. 101–113, March 2020
- **DOI:** 10.1145/3381052.3381315

Introduction: Serverless providers choose among native processes, containers and lightweight isolation platforms such as gVisor and Firecracker, which move functionality out of the host kernel to different degrees [13].

Proposed Solution: A comparative study of LXC containers, gVisor and Firecracker microVMs that measures how much host kernel functionality each uses, with microbenchmarks targeting kernel subsystems.

Results: Despite moving functionality out of the kernel, both Firecracker and gVisor execute substantially more host kernel code than native Linux; gVisor and Linux containers execute substantially the same code, with different frequency.

Gap / Limitation: The study measures host kernel code exercised by benchmarks, not the interface a workload can reach inside the sandbox.

Relevance: The result warns that "sandboxed" is not a measurement. CALIPER treats the sandbox as a property of the cell, classifies differences that the sandbox explains as runtime-class divergence, and still evaluates what the workload can reach.

3.17 Paper 17: Firecracker, Lightweight Virtualization for Serverless Applications

Publication Details:

- **Authors:** Alexandru Agache, Marc Brooker, Andreea Florescu, Alexandra Iordache, Anthony Liguori, Rolf Neugebauer, Phil Piwonka, Diana-Maria Popa
- **Venue:** 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI '20), pp. 419–434, February 2020
- **URL:** `https://www.usenix.org/conference/nsdi20/presentation/agache`

Introduction: Serverless platforms need isolation as strong as virtual machines with overhead close to containers [37].

Proposed Solution: Firecracker, an open-source virtual machine monitor specialised for serverless and container workloads, deployed in AWS Lambda and Fargate.

Results: The paper describes how specialising for serverless shaped the design and reports experience from migrating Lambda customers to it, where it supports millions of

production workloads.

Gap / Limitation: A VM-based sandbox moves the boundary to a guest kernel, whose configuration (and the agent that applies container policy inside it) becomes the thing to verify.

Relevance: Kata Containers, measured as CALIPER's cell 6, follows the same model with a guest kernel per pod. The evaluation found that its agent did not apply part of the requested policy inside the guest (Chapter 12).

3.18 Paper 18: Quantifiable Run-Time Kernel Attack Surface Reduction

Publication Details:

- **Authors:** Anil Kurmus, Sergej Dechand, Rüdiger Kapitza
- **Venue:** Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA 2014), Lecture Notes in Computer Science, Springer, pp. 212–234, 2014
- **DOI:** 10.1007/978-3-319-08509-8_12

Introduction: The kernel's attack surface is the code an unprivileged process can cause to execute, and reducing it requires a way to quantify it [38].

Proposed Solution: The paper measures kernel attack surface in terms of the kernel code reachable by applications and restricts, at run time, the kernel functionality an application may use, quantifying the reduction achieved.

Results: The authors report a substantial, quantified reduction of the reachable kernel surface for the workloads studied.

Gap / Limitation: The approach instruments the kernel and targets a known workload; it is not a portable measurement an unprivileged container can make of its own environment.

Relevance: The paper establishes reachable kernel surface as a measurable quantity. CALIPER measures a coarse, portable proxy for it from user space: the verdict of each entry point at argument granularity.

3.19 Paper 19: Container Security, Issues, Challenges, and the Road Ahead

Publication Details:

- **Authors:** Sari Sultan, Imtiaz Ahmad, Tassos Dimitriou
- **Venue:** IEEE Access, vol. 7, pp. 52976–52996, 2019
- **DOI:** 10.1109/ACCESS.2019.2911732

Introduction: Security is the main concern and adoption barrier for containers in industry surveys [39].

Proposed Solution: A literature survey organised around four use cases: protecting a container from the applications inside it, inter-container protection, protecting the host from containers, and protecting containers from a malicious or semi-honest host.

Results: The first three use cases rely on software mechanisms in the kernel (namespaces, cgroups, capabilities, seccomp) and LSMs; the fourth relies on hardware such as TPMs and Intel SGX. The survey closes with open research problems.

Gap / Limitation: The survey catalogues mechanisms; it does not address how an operator verifies that the mechanisms are in force as configured.

Relevance: CALIPER addresses the third use case, protecting the host from a

container, and measures the software mechanisms the survey lists as its foundation.

3.20 Paper 20: Cloud Container Technologies, a State-of-the-Art Review

Publication Details:

- **Authors:** Claus Pahl, Antonio Brogi, Jacopo Soldani, Pooyan Jamshidi
- **Venue:** IEEE Transactions on Cloud Computing, vol. 7, no. 3, pp. 677–692, 2019
- **DOI:** 10.1109/TCC.2017.2702586

Introduction: Containers have become the common way to virtualise applications in the cloud, and orchestrating clusters of them is a central problem, but no secondary study had consolidated the research [40].

Proposed Solution: A systematic mapping study of 46 selected studies on containers and their orchestration, classified and compared with a characterisation framework.

Results: A discussion of agreed and emerging concerns in container orchestration, positioned against cloud platforms, microservices and continuous development.

Gap / Limitation: Security is one concern among several, and the study maps research rather than examining deployed environments.

Relevance: It shows how mature orchestration research is and, by contrast, how little of it verifies what a node enforces once a workload is scheduled.

3.21 Paper 21: The State of the Art in Container Technologies

Publication Details:

- **Authors:** Emiliano Casalicchio, Stefano Iannucci
- **Venue:** Concurrency and Computation: Practice and Experience, vol. 32, no. 17, Art. no. e5668, 2020
- **DOI:** 10.1002/cpe.5668

Introduction: Containers are used for distributed applications on cloud, edge, fog and Internet-of-Things platforms, and many research challenges remain open [41].

Proposed Solution: An extensive literature review identifying the challenges of adopting containers in high-performance computing, big-data analytics and geo-distributed applications, and analysing the state-of-the-art solution for each.

Results: From 97 papers, performance, orchestration and cyber-security emerge as the main issues; cyber-security covers container isolation, the confidentiality of containerised data and network security.

Gap / Limitation: Isolation is reviewed as a research topic; no method is given to establish, for a particular node, how well it is isolated.

Relevance: The review names container isolation among the main open problems, which CALIPER approaches empirically.

3.22 Paper 22: Emerging Trends, Techniques and Open Issues of Containerization

Publication Details:

- **Authors:** Junzo Watada, Arunava Roy, Ruturaj Kadikar, Hoang Pham, Bing Xu
- **Venue:** IEEE Access, vol. 7, pp. 152443–152472, 2019
- **DOI:** 10.1109/ACCESS.2019.2945930

Introduction: Containerisation is lightweight, portable and efficient, yet its industrial adoption faces challenges in networking, persistent storage, multi-cloud support, management interfaces and security [42].

Proposed Solution: A review of the technicalities involved in containerisation and of its popularity dynamics, together with the potential problems and open issues.

Results: A consolidated overview of trends, techniques and open issues for developers and researchers.

Gap / Limitation: Security is listed among the open issues without a method to assess it.

Relevance: The review motivates practical tooling that can assess a container platform's security as deployed.

3.23 Paper 23: A Container Security Survey, Exploits, Attacks and Defenses

Publication Details:

- **Authors:** Omar Jarkas, Ryan Ko, Naipeng Dong, Redowan Mahmud
- **Venue:** ACM Computing Surveys, vol. 57, no. 7, pp. 1–36, 2025
- **DOI:** 10.1145/3715001

Introduction: The shared Linux kernel and the weaker isolation of containers compared with virtual machines make a single kernel vulnerability a threat to every container on the host, while much academic work on container security is theoretical [43].

Proposed Solution: A taxonomy that integrates academic knowledge with industry insight and real-world vulnerabilities into an actionable framework for container security.

Results: More than 200 container-related vulnerabilities are analysed and categorised into 47 exploit types across 11 attack vectors.

Gap / Limitation: The taxonomy catalogues what has gone wrong; it does not tell an operator which of those entry points a given node exposes now.

Relevance: The paper supports the premise of this project: because one reachable kernel flaw can compromise the whole host, the reachable kernel surface of each node is worth measuring.

3.24 Papers 24 and 25: Two Further Surveys of Containerisation and Its Security

Publication Details:

- **Authors:** Aditya Bhardwaj, C. Rama Krishna
- **Venue:** Arabian Journal for Science and Engineering, vol. 46, no. 9, pp. 8585–8601, 2021
- **DOI:** 10.1007/s13369-021-05553-3

Scope: A survey of virtualisation in cloud computing that follows the move from hypervisor-based virtual machines to containerisation and compares the two approaches [44].

Publication Details:

- **Authors:** Devi Priya V S, Sibi Chakkaravarthy Sethuraman, Muhammad Khurram Khan
- **Venue:** Computers & Security, vol. 135, Art. no. 103490, 2023
- **DOI:** 10.1016/j.cose.2023.103490

Scope: A survey of container security organised by precaution levels and mitigation strategies, with directions for further research [45].

Gap / Limitation: Both surveys describe isolation and its mitigations as design choices; neither offers a measurement of what a deployed node enforces.

Relevance: Together with the surveys in Sections 3.19 and 3.23, they show that the literature treats container confinement as a configuration question. This project treats it as a measurement question.

3.25 Paper 26: To Docker or Not to Docker, a Security Perspective

Publication Details:

- **Authors:** Theo Combe, Antony Martin, Roberto Di Pietro
- **Venue:** IEEE Cloud Computing, vol. 3, no. 5, pp. 54–62, September 2016
- **DOI:** 10.1109/MCC.2016.100

Introduction: Containers offer flexibility and near-native performance, and Docker leads the market [46].

Proposed Solution: An overview of the container ecosystem with an adversary model and a discussion of the Docker environment's security implications through realistic use cases.

Results: The authors identify vulnerabilities affecting Docker usage, many arising from configuration and from the surrounding ecosystem rather than from the isolation mechanisms themselves.

Gap / Limitation: The analysis is qualitative and predates the Kubernetes Pod Security Standards and the current runtime landscape.

Relevance: The paper's emphasis on configuration as a source of weakness motivates checking the configuration that is actually in force, which is what a measurement does.

3.26 Paper 27: Docker Ecosystem, Vulnerability Analysis

Publication Details:

- **Authors:** Antony Martin, Simone Raponi, Théo Combe, Roberto Di Pietro
- **Venue:** Computer Communications, vol. 122, pp. 30–43, 2018
- **DOI:** 10.1016/j.comcom.2018.03.011

Introduction: A follow-up by the authors of the paper in Section 3.25 on the security of Docker taken as a whole ecosystem rather than as an isolated container engine [47].

Proposed Solution: A vulnerability analysis of the Docker ecosystem, organising where weaknesses arise across the platform and its surrounding components.

Results: A structured account of the ecosystem's vulnerabilities, extending the earlier qualitative analysis.

Gap / Limitation: The analysis concerns the ecosystem's design and known vulnerabilities; it does not provide a way to check a particular node's enforced confinement.

Relevance: The paper treats confinement as one part of a wider attack surface. CALIPER covers the part that changes from node to node without the workload changing.

3.27 Paper 28: NIST SP 800-190, Application Container Security Guide

Publication Details:

- **Authors:** Murugiah Souppaya, John Morello, Karen Scarfone
- **Venue:** NIST Special Publication 800-190, National Institute of Standards and Technology, September 2017
- **DOI:** 10.6028/NIST.SP.800-190

Introduction: Guidance for organisations adopting application containers on the risks of the container technology stack [48].

Proposed Solution: The guide identifies risks across images, registries, orchestrators, containers and host operating systems, and recommends countermeasures, among them reducing the host's attack surface, using container-specific host operating systems and enforcing runtime confinement.

Results: A widely cited reference framework for container security programmes.

Gap / Limitation: Recommendations are stated as configuration objectives; verifying their effect on a particular node is left to the operator.

Relevance: The reference posture plays the role of such guidance, but in a form a measurement can be checked against automatically, with a citation for every assertion.

3.28 Tools and Practice

Practitioners rely on four kinds of tool. Table 3.1 summarises what each answers and why none answers the project's question.

3.28.1 The two direct ancestors

`amicontained` established the idea of introspecting confinement from inside a container. It reports the container runtime, the namespaces in use, the capability sets, the AppArmor profile and a list of blocked system calls found by trying each one. Its last release was v0.4.9 in November 2019 and its last commit in September 2020. `seccomp-diff` extracts and disassembles the seccomp-BPF filter of a running process and diffs two filters. It needs host privilege to attach to the process, it is confined to seccomp, and it reasons about the filter rather than about reachability, so it cannot account for capabilities, LSM policy, mount masking, loaded modules or architecture. Both were run against the same target as CALIPER on cell 2; Section 11.8 reports the comparison.

3.28.2 Summary of the survey

The research literature measures container security with exploits, leakage channels and accounting attacks (Papers 2 to 4), generates system call policy for applications (Papers 5 to 10), extends mandatory access control and lints manifests (Papers 11 to 14), studies sandboxed runtimes for performance (Papers 15 to 17), quantifies kernel attack surface with kernel instrumentation (Paper 18), and surveys container security as a set of mechanisms and configurations (Papers 19 to 28). The one work that sets out to benchmark confinement as such (Paper 1) does so per configuration, without environment identity, attribution or remediation. Chapter 4 lists the gaps that follow.

Category	**Examples**	**What it answers, and what it misses**	**Granularity**
Configuration scanners	kube-bench, Kubescape, Trivy [49, 50, 51]	What the manifest and cluster configuration declare, against a benchmark. In the AF_ALG case the declaration was correct and the scanner reports nothing.	manifest fields
Runtime monitors	Falco, Tetragon [52, 53]	What the workload did, as it does it. Surface that is never exercised is invisible.	events
Profile generators	oci-seccomp-bpf-hook, Security Profiles Operator recorder, Inspektor Gadget [54, 55, 56]	What the application needs, from a trace. Says nothing about what the environment permits.	system call names
Introspection ancestors	amicontained v0.4.9 [57]; seccomp-diff [58]	Runtime, namespaces, capabilities and a coarse list of blocked calls (amicontained); the decoded filter of a running process, with host privilege (seccomp-diff). Neither covers masked paths, mounts, modules, LSM effect or argument-level reachability.	system calls; filter rules
CALIPER	this work	What the kernel permits this container, now, per argument; why it differs from another environment; which mechanism refused it; whether that matches a cited expectation; the minimal fix, verified.	arguments

Table 3.1: Existing tool categories compared with CALIPER.

Chapter 4
Literature Gaps

The survey in Chapter 3 leaves the following gaps. Each is stated as an observation about the existing work and the need it leaves, followed by the part of CALIPER that addresses it.

- **No measurement of enforced confinement.**
  Observation: Scanners read declarations, monitors observe behaviour, and generators derive what an application needs. Surveys describe the mechanisms [40, 41, 42, 39, 43, 45], and the research literature evaluates confinement with exploits against chosen configurations [1, 23].
  Need: A measurement, safe to run on a production node, of what the kernel actually permits a given container.
  Addressed by: the probe engine and corpus (Chapter 7).
- **System-call granularity hides the cases that matter.**
  Observation: Policy generators [25, 27, 28, 29, 30, 32, 33], the ancestor tools and most published evaluations work at the granularity of system call names or numbers. The AF_ALG case, raw netlink protocols, namespace flags and io_uring opcodes are all distinctions within a single system call.
  Need: Probes and remediation at argument granularity.
  Addressed by: the argument-level corpus and argument-comparison seccomp rules (Sections 7.7 and 8.6).
- **Results are not keyed to the environment.**
  Observation: Existing evaluations report per configuration or per platform [1, 23, 46, 47]. Architecture, kernel build and loaded modules, runtime version, LSM and RuntimeClass all change what is reachable, and none of them appears in a manifest.
  Need: A result format whose identity is the environment, with each field's source recorded.
  Addressed by: the environment cell and fingerprint format (Section 8.1).
- **Differences between environments are not explained.**
  Observation: A naive comparison of an arm64 and an amd64 node, or of a runc and a gVisor node, reports every difference as a weakness; studies of sandboxed runtimes compare cost and host-kernel use rather than the interface they expose [10, 13]. No published tool separates policy differences from architecture, kernel-version or sandbox differences.
  Need: A classifier that assigns each divergence exactly one explanation.
  Addressed by: the four-class divergence classifier (Section 8.2).
- **Denials are not attributed to a mechanism.**
  Observation: Inside a container an `EPERM` could have come from seccomp, a capability check, an LSM, a user namespace limit or a masked mount, and the kernel does not say which. Reading the seccomp filter needs privilege an unprivileged container lacks, and the benchmark closest to this work reports whether a test escaped, not which mechanism refused an individual call [23]. Without attribution there is no remediation, because the fix differs per mechanism.

Need: Unprivileged attribution with an honest confidence, and the undecidable remainder reported as a measured quantity.
Addressed by: errno oracles, correlation and cell elimination (Section 8.3).

- **No model of what ought to be unreachable.**
  Observation: Manifest analysis checks declarations [35], Pod Security Standards [8] stop at Restricted and say nothing about entry families such as AF_ALG or io_uring, and RuntimeDefault promises only whatever the runtime ships. Comparing a measurement with the manifest alone would have missed the motivating case, because the manifest and the measurement agreed.
  Need: A versioned, cited reference posture, stronger than the tiers it is named after, against which expectation violations are reported separately from claim violations.
  Addressed by: posture v1 and the evaluator (Sections 8.4 and 8.5).
- **Generated policy is not verified.**
  Observation: Profile generators assume that the profile they emit is the one enforced [26, 30, 29]. On sandboxed runtimes it may not be applied at all, and on managed clusters a Localhost profile may not be deliverable.
  Need: Remediation minimal with respect to the runtime's actual default and the posture, emitted in a distributable form, and counted as closed only after a fresh measurement.
  Addressed by: the remediator and closure verification (Section 8.6).
- **The instrument itself is a risk.**
  Observation: Exploit-based evaluation [1] cannot be run on a production node, and probing a kernel interface can change the host (module autoload) or leave objects behind.
  Need: No exploit code, enumeration interfaces in preference to execution, fork isolation, measured residual state, declared side effects, a reduced corpus for shared nodes and a signed image that monitors can allowlist.
  Addressed by: the safety design (Sections 7.2, 7.5 and 7.8).
- **Sandboxed runtimes are trusted rather than measured.**
  Observation: gVisor and Kata are adopted for stronger isolation [10, 13, 37], but their compatibility layers reimplement or reapply parts of the container specification, and how faithfully they do so is rarely checked from the workload's side.
  Need: The same measurement applied inside the sandbox, with its results kept apart from policy.
  Addressed by: cells 5 and 6 and the runtime-class divergence class; the evaluation found one genuine defect in each runtime (Chapter 12).
- **Results are rarely reproducible.**
  Observation: Published confinement evaluations depend on machines and package states that cannot be rebuilt.
  Need: Provisioning pinned to exact package versions, committed measurements, and one command per reported number.
  Addressed by: the Compute Engine cells, committed fixtures and metric generators (Chapter 10 and Appendix A).

Chapter 5
Proposed Solution

5.1 Objective

The project builds an instrument that answers, by measurement, what a container can reach in the kernel on a particular node, explains every difference between two such measurements, attributes each denial to the mechanism that produced it where that can be decided without privilege, evaluates the measurement against both the manifest and a cited model of what ought to be unreachable, and closes what it finds with the narrowest seccomp change, verified by measuring again. The instrument runs as an ordinary container with the same confinement as the workload it stands in for: it never needs, and never asks for, more privilege than that workload has.

5.2 Rationale

The claim is an engineering one. Measuring confinement from inside a container is not a new idea (`amicontained`, Houdini). What did not exist is an instrument that does it at argument granularity, keys the result to the environment, separates policy from architecture and sandbox effects, attributes denials, and carries the result through to a verified remediation. The proposal therefore treats the divergence classifier, the reference posture and the evaluator as the contribution, and the breadth of the corpus as the substrate; breadth was the first thing to cut when the schedule required it.

5.3 Research Questions

RQ1 Can enforced confinement be measured at argument granularity from inside an unprivileged container, safely and quickly enough to run on every pull request?

RQ2 Across environment cells that differ in one axis at a time, how much of the measured divergence is policy, and how much is explained by architecture, kernel version or RuntimeClass?

RQ3 What fraction of denials can be attributed to their mechanism without privilege, and how accurately?

RQ4 Does evaluation against a cited reference posture find exposures that evaluation against the manifest misses, as the AF_ALG case suggests?

RQ5 Can the exposures found be closed with minimal seccomp changes, and does a fresh measurement confirm the closure on every runtime?

5.4 Design Principles

- **Measure, do not infer.** Every verdict is the kernel's answer to one call. The engine stores no interpretation; attribution and classification are computed later from the raw verdicts and the probe's declared metadata, and anything that cannot be decided is reported as undecidable.
- **Identity is the environment cell.** Architecture, kernel version and loaded modules, distribution, runtime and runtime version, LSM and RuntimeClass are first-class fields of every result, each with its source.

- **Reachability, never exploitation.** No probe contains exploit code or uses what a successful call grants. Enumeration interfaces are preferred over execution; io_uring opcodes are enumerated with `IORING_REGISTER_PROBE`, and no submission entry is ever submitted.
- **The instrument survives its findings.** Each probe runs in a forked child, so a seccomp kill or a hang is a recorded verdict rather than a lost run.
- **One interface between the halves.** The Rust measurement side and the Go analysis side meet only at the fingerprint format, which lets four people work in parallel against recorded fingerprints.
- **Report what was given up.** The corpus is split into a committed and a deferred set, and every fingerprint lists the deferred probes as unmeasured, so the gap is visible in every result.

5.5 Objectives and Outcomes

Table 5.1 lists the eight objectives of the proposal and what was delivered against each.

#	Objective	Outcome
1	Probe engine with fork isolation, accounted side effects and precise verdicts	Delivered. Every probe runs in a forked child with a 1-second deadline; six verdicts; SIGSYS survival demonstrated with 122 kills in one run; six residual classes snapshotted per probe; module autoload reported.
2	Argument-granular corpus	Committed set delivered: 185 probes in 8 families, frozen 2026-09-27. Deferred set specified (28 probes, 9 families) and not run.
3	Denial attribution by errno oracle	Delivered, with capability and LSM correlation and cell elimination; three confidence grades; the undecidable fraction is reported as measured.
4	Fingerprint format and divergence classifier	Format version 1 frozen 2026-10-19 (PR #141); four-class classifier; corpus skew reported apart.
5	Versioned, cited reference posture	Posture v1: four tiers, 15 assertions, 16 citations, released with the instrument.
6	Evaluation against manifest and posture	Delivered: claim violations, expectation violations and undeclared exposures, in JSON and text. SARIF deferred.
7	Minimal remediation with closure verification	Localhost seccomp profile and Security Profiles Operator resource, built from the runtime's actual RuntimeDefault; every closure verified by re-probing.
8	Evaluation across six cells	Six Compute Engine cells plus a GKE cluster; AF_ALG case reproduced with the standard image.

Table 5.1: Objectives of the proposal (§4) and their outcomes.

5.6 Success Criteria

The proposal fixed measurable criteria before any code was written (Table 5.2). They are reported, each with its measured value and caveats, in Chapter 11.

Metric	Definition	Success threshold
Divergence across cells	Probes whose verdict differs between two cells, per divergence class	Policy divergence in ≥ 4 pairs; architecture and runtime class separated in every pair
Expectation and claim violations	Findings by category and cell	≥ 1 expectation violation in ≥ 3 cells
AF_ALG reproduction	Detection without bespoke images or profiles	Binary
Attribution accuracy	Against environments in which each denial has one known cause	$\geq 90\%$ of attributed denials correct; $\leq$ 10% of all denials undecidable
Remediation closure	Findings closed, verified by re-probing	100% for seccomp-attributed; others reported
Residual state	Unreclaimed objects after a full run	Zero; autoloaded modules reported apart
Runtime cost	Wall time of a run	Full corpus < 60 s; reduced corpus < 5 s
Ancestor comparison	Findings of amicontained and seccomp-diff on identical targets	Reported, no threshold

Table 5.2: Success criteria fixed in the proposal (§11.2).

5.7 Scope

In scope: measurement of kernel reachability from inside a container on Linux; classification, attribution and evaluation of the measurements; seccomp remediation and its verification; delivery as a Kubernetes Job and DaemonSet, in CI, and on self-managed cells.

Out of scope: continuous runtime monitoring and intrusion detection; generation of profiles from application traces; exploitation of any entry point measured; userspace package vulnerability scanning; non-Linux platforms. The instrument measures and reports. It does not enforce, and it is not a substitute for the mechanisms whose effect it measures.

5.8 Work Allocation

The work was cut along the fingerprint interface into four workstreams so that no member was blocked on another (Table 5.3). Workstreams B and D exchanged their work on 2026-09-14, after the week-3 gate; the letters stayed with the work.

5.9 Schedule

The proposal compressed a two-semester plan into seventeen weeks, starting on Monday 2026-09-07, by reducing scope rather than the critical path. Table 5.4 gives the freeze

WS	Member	Responsibility
A	Abhinav Ajit Madake	Probe engine, fork isolation, safety, probe specification format, the attribution engine, and half of the corpus families
B	Ritesh Aresh Saindane	Corpus families, errno-oracle discipline across the corpus, capability and LSM correlation, the reference posture
C	Yogesh Babaji Palve	Fingerprint serialisation, diff engine and classifier, manifest parsing, evaluator, reporting, and the capability-effect family
D	Sahil Tatyabhau Waje	Delivery: Kubernetes packaging, CI integration, the environment matrix, remediation and closure verification; integrator

Table 5.3: Workstreams and responsibilities.

gates as planned and as met. The gates keep the proposal's week numbers as names; their due dates were brought forward, so the last gate, W17, falls on 2026-11-29, twelve calendar weeks after the start. Week 16 was held empty as declared slack. The system was to be independently defensible from the week-9 gate: measuring, classifying and diffing confinement across environments, even without evaluation or remediation.

Gate	Content	Due	Met
W2	Probe specification and fingerprint format agreed	2026-09-11	2026-09-11
W3	Engine and ten-probe reference corpus end to end	2026-09-18	2026-09-14 (four days early)
W8	Corpus frozen	2026-09-27	2026-09-27, 185 probes
W9	Fingerprint format frozen	2026-10-11	2026-10-19 (PR #141)
W12	Attribution, evaluator and remediation	2026-10-25	2026-10-22 to 2026-10-26
W15	Evaluation complete	2026-11-15	2026-10-26 to 2026-10-27 (accuracy, residual, closure)
W17	Release, report and demonstration	2026-11-29	v1.0.0 released 2026-10-29

Table 5.4: Freeze gates. Work began on 2026-09-07; gate names are the proposal's week numbers, not calendar weeks.

Chapter 6
System Architecture

6.1 Overview

CALIPER has two halves that meet at one interface. The *measurement* half is a single static Rust binary, `caliper-probe`, that runs inside the container being measured and emits a raw measurement. The *analysis* half is a Go control plane that completes the measurement into a fingerprint and then diffs, classifies, attributes, evaluates, reports and remediates. The fingerprint format (Section 8.1) is the only thing the two halves share. Figure 6.1 shows the components and the flow of data between them, including the re-probe that verifies a remediation.

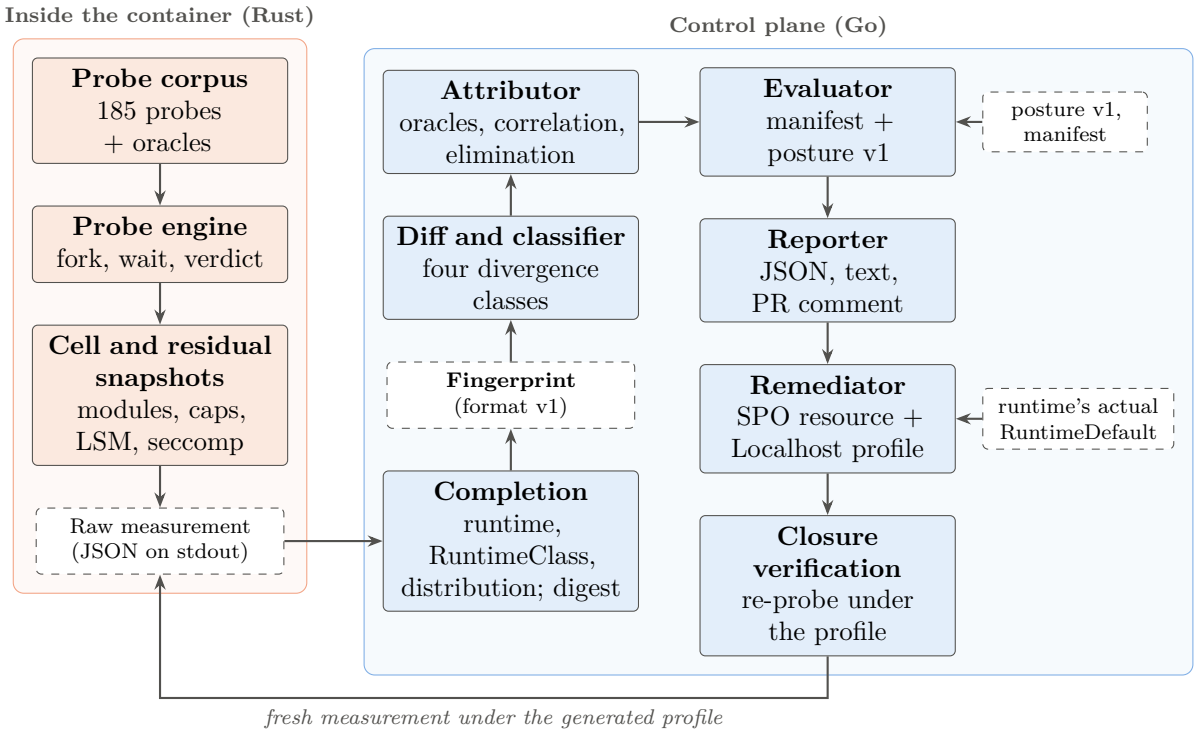


Figure 6.1: Architecture of CALIPER. The measurement half runs inside the measured container; the control plane consumes fingerprints. A remediation is verified by measuring again under the generated profile.

6.2 Components

Table 6.1 lists each component with its language, its location in the repository and its responsibility. The measurement half is about 9,900 lines of Rust and the analysis half about 8,900 lines of Go, tests included.

6.3 The Measurement Half

The probe binary is the only code that runs inside the measured container. It is a statically linked musl executable in a `scratch` image: no shell, no C library on disk, nothing that could issue system calls of its own. On `--run` it snapshots the environment cell and

Component	Location	Responsibility
Probe engine	`engine/crates/engine` (`harness.rs`, `verdict.rs`, `cell.rs`, `residual.rs`, `effects.rs`)	Fork isolation, deadlines, verdict classification, environment-cell snapshot, residual accounting, janitor
Probe corpus	`engine/crates/corpus` (one module per family, `deferred.rs`)	Probe definitions with oracle, kernel dependency, capability, applicability and side effects
Probe binary	`engine/crates/probe`	Command line: `--run`, `--reduced`, `--dump-corpus`, `--noop`
Fingerprint and diff	`control/diff.go`	Decoding, completion, digest, diff over the shared probe set, corpus skew
Classifier	`control/classifier.go`	Assigns each divergence one of four classes
Attributor	`control/attribution.go`	Mechanism and confidence for each result
Manifest parser	`control/manifest.go`, `kubernetes.go`	Workload declarations; cell metadata from Node, Pod and RuntimeClass objects
Evaluator	`control/evaluator.go`, `posture.go`	Expectation, claim and undeclared findings against posture v1
Reporter	`control/report.go`, `cmd/caliper-ci-report`	JSON and text reports; the pull-request comment
Remediator	`control/posture_remediation.go`, `cmd/caliper-remediate`	SPO `SeccompProfile` and OCI Localhost profile
Pipeline	`control/cmd/caliper-pipeline`, `deploy/gce/run-pipeline.sh`	Measure, evaluate, remediate, re-probe, verify
Metric generators	`control/cmd/caliper-*-metrics`, `caliper-attribution-accuracy`	Regenerate every reported metric from committed fingerprints
Delivery	`deploy/k8s`, `deploy/gce`, `.github/workflows`	Job and DaemonSet; cell provisioning and measurement; CI and release

Table 6.1: Components and their responsibilities.

the loaded module set, runs every probe of the selected corpus set in its own forked child, snapshots residual state around each probe, runs the end-of-run janitor, snapshots the module set again and writes one JSON document to standard output. It interprets nothing: a verdict is computed from the child's exit status and the probe's declared metadata, and every further reading is the control plane's. Chapter 7 describes it in full.

6.4 The Analysis Half

The control plane is a Go module with one library package and a command per task. It never measures; it consumes fingerprints. The stages are:

1. **Completion.** The probe cannot observe the runtime, runtime version, distribution or RuntimeClass. The control plane adds them, from the Kubernetes Node, Pod and RuntimeClass objects when it runs in a cluster, or from the operator outside one, records the source of each value, and computes the SHA-256 digest of the completed document.

2. **Diff and classification.** Two fingerprints are compared over the probes present in both; each difference is assigned exactly one of four classes.
3. **Attribution.** Each denial is assigned a mechanism and a confidence, or reported as undecidable with its candidate mechanisms.
4. **Evaluation.** Each permitted probe is categorised against the workload's manifest and a posture tier.
5. **Reporting.** JSON and text reports carry the findings, divergences, attribution confidence, the undecidable fraction and the authorisation notice.
6. **Remediation and verification.** Expectation violations become a narrowed seccomp profile; a fresh measurement under that profile decides which findings are closed.

Chapter 8 describes each stage.

6.5 The Environment Cell

The fingerprint's identity is the environment cell, not the workload. Table 6.2 lists the cell's fields and who records each. Only what an unprivileged process can observe is recorded by the probe; the rest is supplied by the control plane with its source, because "containerd 2.2 from the Node object" and "containerd 2.2 because the operator said so" are different evidence.

Field	Recorded by	Source
Architecture	probe	`uname(2)` machine field
Kernel release	probe	`uname(2)`; a sandbox's synthetic version is kept as its claim
Loaded modules	probe	`/proc/modules`, read before the first probe and after the last
Capability sets	probe	effective, permitted, bounding and ambient masks from `/proc/self/status`
LSM, label, enforcing	probe	`/proc/self/attr/<lsm>/current` for each module tried (the generic `/proc/self/attr/current` for SELinux); `/sys/fs/selinux/enforce`
seccomp mode, filters	probe	`Seccomp` and `Seccomp_filters` lines of `/proc/self/status`
User-namespace limits	probe	`user.max_user_namespaces`, `kernel.unprivileged_userns_clone`, AppArmor's restriction sysctl, and whether the process is in the initial user namespace (`/proc/self/uid_map`)
Distribution	control plane	Node object (`osImage`) or operator
Runtime and version	control plane	Node object (`containerRuntimeVersion`) or operator
RuntimeClass	control plane	pod spec and RuntimeClass handler, or operator

Table 6.2: Fields of the environment cell and their sources.

A RuntimeClass is handled in the same spirit as architecture. gVisor implements a subset of the system call interface and reports a synthetic kernel version, and Kata boots a guest kernel of its own, so a verdict that differs between a runc cell and a sandboxed cell is explained by the sandbox before any policy is consulted. A difference in LSM or in runtime version, by contrast, is a difference in the policy the kernel enforces and is

classified as policy.

6.6 Why Rust for the Engine and Go for the Control Plane

The engine has three requirements that a managed runtime cannot meet. It must fork before every probe, and a runtime that has started threads cannot fork safely. It must not issue system calls of its own that pollute the measurement, while a garbage collector and a scheduler issue them in the background. And it must survive a child killed by a seccomp `KILL` action, which requires precise control of what the parent and child each execute. Rust compiled against musl into a static binary meets all three; CI checks the engine's own system calls against a committed baseline (Section 7.4). The control plane has none of these constraints and needs the container ecosystem's libraries (Kubernetes client, GitHub API, Security Profiles Operator types), which are written in Go.

6.7 The Interface and Parallel Development

Because the fingerprint format is the only interface, the four workstreams could work in parallel from week 2. The first format was agreed on 2026-09-11; recorded fingerprints from Lima virtual machines were committed as fixtures in week 5, and the diff engine, classifier, evaluator and reporter were developed and tested against them long before the corpus was complete. Format version 1 was frozen on 2026-10-19. A consumer refuses a format version it does not know rather than reading it on a best-effort basis, and optional fields added within version 1 are ignored by older consumers, so a fixture recorded in week 3 still parses in week 12.

6.8 Repository Organisation

Path	Contents
`spec/`	Frozen interfaces: `probe.md` (probe specification) and `fingerprint.md` (format version 1), with a worked example
`engine/`	Rust workspace (crates `engine`, `corpus`, `probe`), Dockerfile, system-call baselines, seccomp fixtures
`corpus/`	`index.json` (213 probes: 185 committed, 28 deferred), regenerated by CI from `--dump-corpus`; revision table
`posture/`	`v1/posture.yaml` and `v1/RATIONALE.md`; changelog
`control/`	Go control plane, its commands and test data
`deploy/`	Kubernetes Job and DaemonSet; Compute Engine cell scripts; the end-to-end pipeline
`fixtures/`	Committed fingerprints (cells, ground truth, pipeline runs, runtime runs, ancestors) and metric files
`docs/`	Proposal, oracle design and sweep, per-issue evaluation records, safety review
`.github/workflows/`	`ci.yml`, `cluster-evaluation.yml`, `release.yml`

Table 6.3: Repository layout.

Chapter 7
Methodology I: Measurement

This chapter describes how a measurement is made: the probe specification, the engine that executes probes, the accounting of side effects, and the corpus. Chapter 8 describes what is done with the measurement.

7.1 The Probe Specification

A probe attempts one kernel operation at argument granularity and records what the kernel returned. The specification (`spec/probe.md`) was frozen at the week-2 gate on 2026-09-11. Every probe is authored complete with the fields in Table 7.1; the proposal is explicit that attribution metadata is not added in a later phase.

Field	Content
Identity	A stable identifier (`socket.family.af_alg`), the entry family, and a one-line description of the exact call
Operation	The system call and the arguments under test
Errno oracle	`guarantees`: the errno the kernel is structurally guaranteed to return if the call reaches it, or none; `isolates`: which mechanisms an `EPERM` before that point can be (`seccomp`, `seccomp-or-lsm` or `undecidable`); `reason`: the kernel code path that justifies both
Capability requirement	The operation's documented capability, for correlation with the recorded capability sets
Kernel dependency	`since` (first kernel release), `absent_errno` (what absence returns), `module`, `present_if` (a path whose existence proves the kernel builds it), `also_absent` with a protocol module, `encoded_absent` (an enumeration bit encoded as an errno), `lsm_hook_since` (the release from which an LSM hook precedes the oracle's point), and whether user-namespace limits can refuse it
Architecture applicability	`all`, or the architectures on which the entry point exists
Side effects	Residual classes the probe may leave (`mounts`, `net-namespaces`, ...) and whether it can trigger a module autoload
Risk class	Whether it requires fork isolation, and its timeout (1,000 ms for every committed probe)

Table 7.1: Fields of a probe definition (`spec/probe.md`).

Four decisions were taken at the freeze and constrain everything after it.

1. **Probes are Rust definitions compiled into the engine**, not a data file read at run time. Opening and parsing a corpus file would be system calls of the instrument's own, made before the module snapshot; a data file beside a static binary is a second artefact that can drift; and a declarative corpus would be a second interface across the Rust–Go boundary. The engine emits the corpus as JSON on request (`--dump-corpus`); that output, `corpus/index.json`, is what the control plane and the freeze-gate checks read.
2. **A probe that exceeds its timeout is recorded `timed-out`; hung and slow are**

not distinguished. From outside the child, a probe blocked forever and one that would have returned just after the deadline are the same observation. The timeout travels in the fingerprint so that a verdict can be read against it.

3. **The committed/deferred split lives in the corpus index**, not in the specification, so promoting a deferred probe is an index entry and an implementation, never a re-specification.
4. **Rollback is the child's, not a hook.** The child removes what it created before it exits; the engine measures whether it did. A rollback hook run by the parent would run under the same filters (failing the same way, or killing the parent), or under different ones (succeeding where the probe could not and hiding the asymmetry the corpus exists to measure).

7.2 Fork Isolation

Every probe runs in a child created with musl's `fork()`, whether or not its risk class says it needs isolation: a probe that declares it does not need rollback still must not be able to end the run. Figure 7.1 shows one probe's lifecycle.

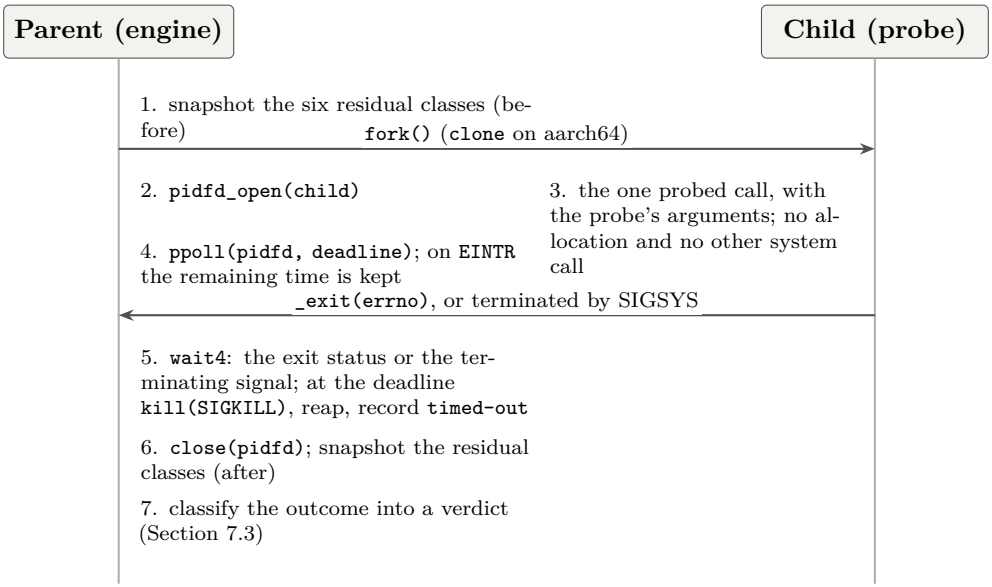


Figure 7.1: The lifecycle of one probe. The parent issues only `fork`, `pidfd_open`, `ppoll`, `wait4`, `close` and, on a timeout, `kill`.

7.2.1 Why fork and not clone

An early version created children with a raw `clone` system call. musl caches the thread identifier and delivers `abort()` and `raise()` to it with `tkill`, so a raw-cloned child that aborted killed the *parent*. `fork()` fixes the identifier up in the child, at the cost of `set_tid_address` and `rt_sigprocmask` there before the probe, both already in the engine's start-up set.

7.2.2 The exit status is the measurement

The child reports through its exit status (Table 7.2). Linux errno values on x86_64 and aarch64 stop at 133, so an 8-bit exit status carries every errno, with two values reserved.

How the child ended	Meaning
`_exit(0)`	The operation succeeded
`_exit(n)`, $1 \leq n \leq 133$	The operation failed with errno n
`_exit(253)`	The probe panicked (a panic hook exits with 253 rather than unwinding)
`_exit(254)`	An errno that does not fit
Terminated by SIGSYS	A seccomp `KILL_PROCESS` or `KILL_THREAD` action (or `TRAP` with no handler)
Terminated by another signal	The probe crashed: a fault of the instrument, never a verdict
Still running at the deadline	Killed with SIGKILL and recorded `timed-out`

Table 7.2: How the child's end is read by the parent.

7.2.3 Deadlines

The parent opens a process descriptor on the child and waits on it with the raw `ppoll` system call, bounded by the probe's timeout. Using the raw system call means that on `EINTR` the kernel hands back the remaining time and the same deadline continues. A test shows the same 200 ms sleep recorded `timed-out` under a 50 ms deadline and `permitted` under a 2 s one.

7.2.4 Surviving SIGSYS

A probe that trips a filter whose action is `KILL_PROCESS` terminates the process that issued it. Because that process is the child, the parent sees SIGSYS in the wait status, records `killed` and continues. The engine's test installs a real `KILL_PROCESS` filter in the test thread, so that the parent runs under the same filter as the probe, as in deployment; trips it; checks the verdict; and runs two more probes under the same filter, one of which trips it again. In the evaluation, cell 2 measured under a hand-hardened profile recorded 122 `killed` verdicts in a single run that completed normally (Section 11.1).

7.2.5 No core dumps on the node

At start-up the engine marks itself non-dumpable (`PR_SET_DUMPABLE = 0`), which every child inherits. A child killed by SIGSYS would otherwise dump core, and inside a container `core_pattern` (which is not namespaced) commonly pipes the core to the host's crash handler: a side effect on the node and a stall of a second or more per kill. `RLIMIT_CORE = 0` does not prevent piped cores; the dumpable flag does.

7.2.6 Faults of the instrument

A harness stage marker distinguishes a probe that failed from a harness that failed: a failure while setting up a child is recorded as `setup-failed` or `setup-killed`, and the probe goes to the fingerprint's `unmeasured` list with that reason rather than receiving a verdict. A run measured with such defects exits with status 3 (`--exit-0-on-defects` keeps a Kubernetes Job from retrying, since the defects are in the document).

7.3 Verdict Classification

`verdict.rs` turns how the child ended into one of six verdicts and the raw errno. The rules, applied in order, are:

1. If the probe's architecture applicability excludes this machine: **not-applicable**, decided before anything runs.
2. If the child returned 0: **permitted**, errno 0.
3. If the child returned the errno its oracle guarantees: **permitted**, with that errno. The call reached the point the oracle was built to reach.
4. If the child returned the errno that absence of the entry point produces, and the kernel dependency says this kernel may lack it (version below `since`, `present_if` path missing, module absent, or the release unparsable): **unimplemented**.
5. Any other errno: **denied**. In particular `ENOSYS` from a system call the kernel implements is a filter answering in the kernel's voice, and is **denied**.
6. If the child was terminated by SIGSYS: **killed**.
7. If the deadline expired: **timed-out**.
8. Any other signal, or a harness fault: not a verdict; the probe is recorded as unmeasured with the reason.

Two rules are deliberate. A child that died of SIGABRT or SIGSEGV is the probe crashing, which must not read as a hardened node: a buggy probe on a permissive node would otherwise look like confinement. And the engine interprets nothing beyond these rules; which mechanism produced a denial is computed by the control plane.

7.4 The Engine's Own System Calls

A probe measures what the kernel permits; the engine must not add system calls of its own, and the parent runs under the same filter as the probe, so its start-up set must survive RuntimeDefault and a hand-hardened profile. `engine/baseline/<arch>.txt` records that set, produced by tracing `caliper-probe --noop`: twelve system calls on aarch64 and thirteen on x86_64, all from Rust's standard library, musl start-up and the engine's initialisation. CI traces the binary and diffs the observed set against the file, so a dependency that starts issuing system calls fails the build rather than the measurement. `nix`, `libc` and `serde` are the only dependencies that touch the system.

`engine/fixtures/seccomp/hardened.json` is a hand-hardened profile: an exact allowlist with `KILL_PROCESS` as the default action, in four groups: the start-up set; the harness set (fork, wait, kill, write, exit); runc's own pre-exec set, found empirically; and the run path's set (`uname` and the procfs and sysfs reads of the cell and residual snapshots). The probed system calls are deliberately absent (except `clone3`, which the harness keeps), so under this profile every probe outside the list has its child killed by SIGSYS. A test applies the engine's groups in a forked child immediately before `execve`, for `--noop` and `--run`, with a negative control that drops one start-up system call and checks the child dies.

7.5 Side Effects and Residual State

Fork isolation is the rollback for every descriptor-shaped effect: sockets, io_uring rings and device descriptors are released by the kernel when the child exits. The classes of state that outlive a process are measured directly, around every probe, rather than by comparing the whole host, which normal system activity would swamp. Table 7.3 lists the six classes and how each is counted. A class the environment does not expose is recorded as null, never as zero.

Snapshots are taken after the child is reaped and before anything is removed, so a probe that could not remove what it created (killed first, or creation permitted and

Class	How it is counted
Mounts	Lines of `/proc/self/mountinfo`
Keyrings	Lines of `/proc/keys`; where that file is masked (as under RuntimeDefault), the `nkeys` of the process's fsuid in `/proc/key-users`
Cgroups	Directories below `/sys/fs/cgroup`
System V IPC	Entries in `/proc/sysvipc/shm`, `sem` and `msg`
POSIX IPC	Entries in `/dev/shm` and `/dev/mqueue`
Network namespaces	Distinct `ns/net` links among visible processes

Table 7.3: The six residual classes (`engine/crates/engine/src/residual.rs`).

removal blocked) leaves an object that is counted, attributed to that probe and checked against its declaration. Two quantities are reported: *unreclaimed objects*, the growth of each class over the whole run, and *leaks*, per-probe changes, so that an object one probe leaves and a later one removes still fails. Every probe declares its residual classes; a probe with no declaration is refused by a corpus test that gates every merge.

The structural fallback is an end-of-run *janitor*. After the final snapshot it removes what is recognisably the instrument's (POSIX objects under a reserved name prefix, System V objects in a reserved key range, the run's mount directory `/tmp/caliper-<run>-mount`), each removal in its own forked child under the same filters, and records every attempt in the fingerprint. Its output is a finding about the corpus, never a silent fix. Per-probe namespaces would have been cleaner and are not available: a fresh namespace needs CAP_SYS_ADMIN or a user namespace, RuntimeDefault denies both, and a user namespace would change what the kernel permits.

7.6 Module Autoload and the Reduced Corpus

One side effect is neither descriptor-shaped nor reclaimable. A `socket(2)` call on an address family the kernel has not registered makes the kernel request the module for it, as does an unregistered netlink protocol, a device node whose driver is a module, or a filesystem type; the kernel does this with no capability check on the caller, and the container cannot unload the module afterwards. The engine records the loaded module set before any probe runs, so that the environment cell describes the node and not the instrument's footprint, records it again after the run, and reports the difference as `module_delta.loaded_by_run`. Thirty-two committed probes declare a possible autoload (19 socket, 8 netlink, 3 device and 2 mount). The *reduced* corpus is the same revision without them; it is the set CI runs on every pull request, and the one to run on a shared node.

7.7 The Corpus

The committed corpus has 185 probes in eight families (Table 7.4). It was frozen at 185 probes on 2026-09-27. Later revisions corrected oracles, kernel dependencies and side-effect handling without adding or removing a probe; between the revision used for the cell recordings (0.3.0) and the released one (0.3.3), every verdict and errno on all six cells is unchanged. Where an enumeration interface can answer the question it is used instead of executing the operation.

Family	Probes	Oracle isolates	What a probe does
socket	24	1 seccomp, 19 seccomp-or-lsm, 4 undecidable	`socket(family, type, 0)` only; AF_ALG is opened and never bound
netlink	18	18 seccomp-or-lsm	`socket(AF_NETLINK, SOCK_RAW, protocol)` only
io_uring	53	53 seccomp	setup with zero entries, register on an invalid descriptor, opcode enumeration on a one-entry ring
mount	38	1 seccomp, 37 undecidable	34 read `/proc/filesystems`; one mounts onto a missing path; three mount on the run's own directory and unmount at once
clone	19	2 seccomp, 4 seccomp-or-lsm, 13 undecidable	`clone` or `unshare` with one namespace flag; invalid flags; `clone3` with a short size
path	17	17 seccomp-or-lsm	`openat(O_PATH)` and `fstatfs`: masked, read-only or reachable
device	8	8 undecidable	read-only non-blocking open, then close
capability	8	8 undecidable	one capability-gated call each, built to fail after the check
total	185	57 / 58 / 70	corpus revision 0.3.3

Table 7.4: The committed corpus by family, with the isolation of each family's oracles.

7.7.1 Socket address families (24)

Each probe calls `socket(AF_x, type, 0)` and exits. The families are `AF_INET`, `AF_INET6`, `AF_UNIX`, `AF_ALG`, `AF_PACKET`, `AF_KEY`, `AF_BLUETOOTH`, `AF_CAN`, `AF_TIPC`, `AF_RDS`, `AF_XDP`, `AF_RXRPC`, `AF_PHONET`, `AF_IEEE802154`, `AF_NFC`, `AF_VSOCK`, `AF_KCM`, `AF_QIPCRTR`, `AF_SMC`, `AF_MCTP`, `AF_AX25`, `AF_APPLETALK` and `AF_X25`, plus the oracle probe `socket.family.out_of_range`, which calls `socket(1000, SOCK_STREAM, 0)`. The kernel rejects a family at or beyond `NPROTO` with `EAFNOSUPPORT` before `security_socket_create` runs, so only a filter at system call entry can answer `EPERM` first: that oracle isolates seccomp. For a real family the LSM hook runs before the family's create function, so `EPERM` is seccomp or an LSM; four families (`AF_PACKET`, `AF_KEY`, `AF_XDP`, `AF_PHONET`) check a capability themselves and are undecidable. `EAFNOSUPPORT` from a real family is its absence, recorded as the kernel dependency.

7.7.2 Netlink protocols (18)

Each probe calls `socket(AF_NETLINK, SOCK_RAW, protocol)` for one of `route`, `usersock`, `sock_diag`, `nflog`, `xfrm`, `selinux`, `iscsi`, `audit`, `fib_lookup`, `connector`, `netfilter`, `kobject_uevent`, `generic`, `scsitransport`, `rdma`, `crypto` and `smc`, and the oracle probe with protocol 32, for which `netlink_create` guarantees `EPROTONOSUPPORT`. That check runs after the LSM hook and the family lookup, so an `EPERM` there is seccomp or an LSM. Creating a netlink socket checks no capability (CAP_NET_ADMIN and the audit capabilities are checked per message), so every protocol probe is `seccomp-or-lsm`. No message is ever sent.

7.7.3 io_uring opcode reach (53)

Two probes test the system calls and 51 enumerate opcodes. `io_uring.setup.zero_entries` calls `io_uring_setup(0, &params)`, which is guaranteed `EINVAL` without creating a ring. `io_uring.register.probe` calls `io_uring_register(INT_MAX, IORING_REGISTER_PROBE, NULL, 0)`: a descriptor above the `nr_open` ceiling guarantees `EBADF`, and descriptor resolution precedes every hook, so this oracle isolates seccomp. (`-1` was used first; from 6.13 it selects the "blind" register path, which answers `EINVAL`, a defect found and fixed during development.) The opcode probes create a one-entry ring and read `IORING_REGISTER_PROBE`'s table; an opcode whose `IO_URING_OP_SUPPORTED` bit is clear is encoded as `ENODATA`, an errno `io_uring_register` never produces for a ring it has just accepted. No submission queue entry is ever submitted. From Linux 6.15 `io_uring_setup` calls `security_uring_allowed()` before checking its parameters, recorded as `lsm_hook_since: [6, 15]`, so on newer kernels a setup or opcode `EPERM` is seccomp or an LSM. The 51 opcodes run from `NOP`, `READV` and `WRITEV` to `FUTEX_WAITV`, `FIXED_FD_INSTALL`, `FTRUNCATE`, `BIND` and `LISTEN`.

7.7.4 Mount filesystem types (38)

Thirty-four probes read `/proc/filesystems` and report `ENODEV`, the errno `get_fs_type` answers for an unregistered type, when a type is not listed: `proc`, `sysfs`, `tmpfs`, `devtmpfs`, `devpts`, `overlay`, `fuse`, `fusectl`, `cgroup`, `cgroup2`, `bpf`, `debugfs`, `tracefs`, `configfs`, `securityfs`, `efivarfs`, `pstore`, `binfmt_misc`, `hugetlbfs`, `mqueue`, `autofs`, `ecryptfs`, `squashfs`, `iso9660`, `vfat`, `ext4`, `btrfs`, `xfs`, `nfs`, `nfs4`, `cifs`, `9p`, `virtiofs` and `ntfs3`. They measure what the kernel would mount if a mount were allowed. `mount.reach` calls `mount(NULL, "/caliper-no-such-dir", "proc", 0, NULL)`, guaranteed `ENOENT` once the call reaches path lookup. Three probes (`mount.fs.proc`, `mount.fs.overlay`, `mount.fs.fuse`) attempt a real mount of the type on the run's own directory under `/tmp` and unmount at once; the FUSE probe passes no descriptor, so it is guaranteed `EINVAL` after the capability check.

7.7.5 Clone and unshare flags (19)

`unshare` with each of `CLONE_NEWUSER`, `NEWNS`, `NEWNET`, `NEWPID`, `NEWIPC`, `NEWUTS`, `NEWCGROUP`, `NEWTIME` and `NEWUSER|NEWNET` (9); `clone` with the same flags except `NEWTIME` (8); `unshare.flags.invalid`, which passes `0x1` and is guaranteed `EINVAL` before any namespace is created; and `clone3.args.short`, which passes a size below `CLONE_ARGS_SIZE_VER0` and is guaranteed `EINVAL`. The cloned child calls `exit_group` at once. Namespace creation checks capabilities inside the operation, so those probes are undecidable without the capability correlate; the user-namespace probes are `seccomp-or-lsm` because no capability is required.

7.7.6 Masked and read-only paths (17)

A mask's answer to `open` is success and to `read` is zero bytes, while the real object often answers `EACCES` to an unprivileged caller, so recording those answers untouched would report the mask as permitted and the real file as denied, the inverse of the confinement. The kernel's honest answer to "is this the object or the mask" is `fstatfs` reporting tmpfs where procfs belongs. Each path probe therefore opens the path with `O_PATH` (no permission check, no read), calls `fstatfs`, and returns `ENODATA` if the filesystem is not the object's, `EROFS` if the mount is read-only (`ST_RDONLY`), and success otherwise. Neither

errno can be produced by the kernel for these calls, which is the one bounded exception to recording the kernel's answer untouched. The paths are the kubelet's eleven masked paths and five read-only paths (Chapter 2), plus `path.control.proc_self_status` as a control.

7.7.7 Device nodes (8)

A read-only, non-blocking `openat` and `close` of `/dev/fuse`, `/dev/net/tun`, `/dev/kvm`, `/dev/kmsg`, `/dev/mem`, `/dev/port`, `/dev/null` (control) and `/dev/sda`. Nothing is read or written. `ENOENT` means the runtime did not create the node; `EACCES` and `EPERM` are undecidable because DAC, the device cgroup, an LSM and a driver's capability check can all answer.

7.7.8 Capability effects (8)

Each probe makes one capability-gated call, chosen so that the privileged path is harmless and the unprivileged path distinctive (Table 7.5). These probes are deliberately `undecidable`: the missing-capability result is the observation, and it is correlated with the recorded capability sets rather than read as a filter.

Probe	Capability	Call and why it is harmless
`capability.sys_time`	`CAP_SYS_TIME`	`settimeofday(NULL, NULL)`: the check precedes a no-op update
`capability.sys_boot`	`CAP_SYS_BOOT`	`reboot(0, 0, CAD_OFF, NULL)` with bad magic: `EINVAL` after the check
`capability.sys_module`	`CAP_SYS_MODULE`	`finit_module(-1, "", 0)`: `EBADF` after the check
`capability.mknod`	`CAP_MKNOD`	`mknodat` of a `/dev/null`-equivalent node in `/tmp`, removed by the child
`capability.sys_ptrace`	`CAP_SYS_PTRACE`	`process_vm_readv` of the non-dumpable parent with an invalid remote address: `EFAULT` after the access check
`capability.net_bind_service`	`CAP_NET_BIND_SERVICE`	bind an IPv4 socket to port 1: `EACCES` without the capability; no `listen`
`capability.sys_nice`	`CAP_SYS_NICE`	`setpriority(PRIO_PROCESS, 0, -1)` on the child itself
`capability.sys_rawio`	`CAP_SYS_RAWIO`	`iopl(3)`, x86_64 only; no port I/O follows

Table 7.5: The capability-effect probes.

7.7.9 The deferred set (28)

Twenty-eight probes in nine families are specified in the same format, each with its oracle and kernel ordering, and are never run (Table 7.6). Every fingerprint lists them as unmeasured, so the gap is visible in every result. Promoting one is an implementation and an index entry, not a redesign.

7.7.10 Oracle review and the freeze

Before the freeze every oracle was reviewed against the kernel function named in its `reason` and the man-page errno contract, family by family, and a sweep of the generated index (`docs/oracle-sweep.md`) checked three properties across all 185 probes: no probe has an empty oracle, no oracle guarantees `EPERM`, and no oracle that claims to isolate seccomp has its guaranteed answer ordered after an LSM or capability hook. Eleven

Family	Probes	Strategy
ioctl	3	`ioctl(-1, CMD, ...)`: `EBADF` before `security_file_ioctl` isolates filtering on the request code (`FIOCLEX`, `TIOCSTI`, `TIOCLINUX`)
bpf	5	a non-zero byte past `union bpf_attr` answers `E2BIG` before `security_bpf`; program types are undecidable against their capability
keyctl	4	a serial that cannot exist answers `ENOKEY` before any key permission hook; nothing is created
prctl	3	`security_task_prctl` precedes the option switch, so every `EINVAL` oracle is seccomp-or-LSM
`perf_event_open`	3	bad flags (`EINVAL`) and bad size (`E2BIG`) precede the hook; a CPU-wide event is gated by `perf_event_paranoid`
userfaultfd	3	an unknown flag beside `UFFD_USER_MODE_ONLY` answers `EINVAL` before any hook
`open_by_handle_at`	2	`name_to_handle_at` rejects unknown flags before path lookup
module	2	`init_module` and `delete_module` check CAP_SYS_MODULE first; undecidable
`seccomp_notify`	3	`seccomp(2)` has no LSM hook, so `EFAULT` from a NULL program isolates seccomp

Table 7.6: The deferred set (`engine/crates/corpus/src/deferred.rs`).

probes carry a guaranteed errno; 57 isolate seccomp, 58 isolate seccomp or an LSM, and 70 are undecidable from the errno alone.

7.8 Safety Review

The corpus was reviewed before release for exploit code and for operations whose success confers privilege (issue #50). It contains no payload, no shellcode, no crafted structure for a known vulnerability and no sequence beyond the single call under test. Most probes are built so that success yields nothing usable: the call fails after the check under test with the error the oracle guarantees, or changes nothing. Five can hand the child something when the cell permits them, and none uses it (Table 7.7). Each grant is one the cell already gave the workload, and each ends with the child.

Probe	On success the child holds	Then
`capability.sys_rawio`	I/O privilege level 3 (`iopl(3)`), x86 only	exits without port I/O
`capability.sys_nice`	a nice value one lower, on itself	exits
`capability.net_bind_service`	a socket bound to TCP port 1 in its network namespace	exits without `listen`
`device.open.*`	a read-only descriptor, for example on `/dev/mem`	closes it unread
`clone.*`, `unshare.*` (user namespace)	capabilities inside a new, empty user namespace	exits

Table 7.7: Probes whose success grants the child something, and what the child does with it.

The exception is module autoload. A full run under RuntimeDefault on cell 2 or cell 4 loads 31 modules, among them `af_alg`, `af_key`, `xfrm_user`, `bluetooth`, `can`, `rxrpc`, `smc` and `kcm`, which stay loaded on the host for every tenant until someone with host privilege

removes them. That widens the node's reachable surface for everyone, the opposite of the instrument's purpose. It is declared on each of the 32 probes, reported in every fingerprint and counted apart from residual state, and the reduced corpus, which loads none, is the one to run on a shared node. The proposal's statement that no probe performs an operation whose success confers privilege should be read with these cases in view.

Chapter 8
Methodology II: Analysis

This chapter describes the control plane: the fingerprint format, the divergence classifier, the attributor, the reference posture, the evaluator, the remediator and its closure verification, and the reports.

8.1 The Fingerprint Format

A fingerprint is one measurement run. Format version 1 (`spec/fingerprint.md`) was agreed in week 2 and frozen on 2026-10-19 by PR #141. Table 8.1 lists its top-level fields and Figure 8.1 shows an excerpt of the cell 2 fingerprint.

Field	Content
`format_version`	Integer, incremented only by a breaking change. A consumer refuses a version it does not know
`corpus_revision`	The corpus that produced the run, distinct from the format version
`corpus_set`	`"full"` or `"reduced"`; a probe the reduced set omits is absent from `results`, not a divergence
`digest`	`sha256:` over the canonical document with this field omitted; identity and deduplication, not authenticity
`cell`	The environment cell (Table 6.2); control-plane fields carry `{value, source}`
`results`	One entry per measured probe: `probe_id`, `verdict`, raw `errno`, and, once attributed, an `attribution` object
`unmeasured`	Probes not run, each with its reason: `deferred`, a harness fault, or a crash with its signal
`module_delta`	`before`, `after` and `loaded_by_run` module lists
`residual`	Observability of each class, counts before and after, leaks, unverifiable classes, the run-level audit and the janitor's actions
`elapsed_ms`	Wall time of the run (optional, added within version 1)

Table 8.1: Top-level fields of a fingerprint, format version 1.

8.1.1 Conventions

Object keys are `snake_case`; enumerated values are `kebab-case` (`"timed-out"`, `"policy-explained"`); an enumerated value that carries data is a one-key object whose key is the value. The errno is always the raw integer, never a name. Booleans are stated, not implied by absence: a missing key means the producer did not know, `false` means it knew and the answer was no. Every control-plane field records its source.

8.1.2 Digest

The producer computes the digest once: it omits `digest` from the complete document, encodes it as UTF-8 JSON with object keys sorted, no insignificant whitespace and no HTML escaping, hashes the bytes with SHA-256, and prefixes the lowercase hexadecimal result with `sha256:`. A consumer preserves the supplied digest and never recomputes

```
{ "format_version": 1, "corpus_revision": "0.3.0", "corpus_set": "full",
  "digest": "sha256:977558ac4482df1fec0d87674f56cf44516fb5706938c7d0bdef7843a4604030",
  "cell": { "architecture": "x86_64",
            "kernel": { "release": "7.0.0-1014-gcp", "version": {"major": 7, "minor": 0},
                        "sandbox_claimed": false, "modules": ["8021q", "aesni_intel", ...] },
            "distribution":    {"value": "Ubuntu 24.04", "source": "operator"},
            "runtime":         {"value": "containerd",   "source": "operator"},
            "runtime_version": {"value": "2.2.1",        "source": "operator"},
            "runtime_class":   {"value": "runc",         "source": "operator"},
            "lsm": "apparmor", "lsm_label": "cri-containerd.apparmor.d (enforce)",
            "lsm_enforcing": true,
            "capabilities": {"effective": "0000000000000000", "bounding": "0000000000000000", ...},
            "user_namespaces": {"max": 28082, "unprivileged_clone": 1, "apparmor_restricted": 1} },
  "results": [ {"probe_id": "socket.family.af_alg",    "verdict": "permitted", "errno": 0},
               {"probe_id": "io_uring.register.probe", "verdict": "denied",    "errno": 1},
               {"probe_id": "path.masked.proc_kcore",  "verdict": "denied",    "errno": 61}, ... ],
  "unmeasured": [ {"probe_id": "ioctl.request.fioclex", "why": "deferred"}, ... ],
  "module_delta": { "before": [...], "after": [...], "loaded_by_run": ["af_alg", "af_key", ...] },
  "residual": { "before": {"mounts": 25, "keyrings": null, "cgroups": 0, ...},
                "after":  {"mounts": 25, "keyrings": null, "cgroups": 0, ...}, "leaks": [] } }
```


Figure 8.1: Excerpt of the cell 2 fingerprint (`fixtures/cells/cell2.json`).

it, because a consumer that ignores optional fields added later could not reproduce the original bytes. Fingerprints are not signed: the proposal requires signing for the probe *image*, so that operators can allowlist a known instrument, and nothing in the threat model makes a forged fingerprint the attack.

8.1.3 Compatibility

Within a format version, fields may be added and must be optional; no field is removed, renamed or retyped; no enumeration value is removed; and unknown fields are ignored. Anything else is a new format version. Two fingerprints from different corpus revisions are compared over the probe identifiers present in both, and a probe present in only one is reported in a separate *corpus-skew* section and never classified: instrument drift must not be reported as a property of the node.

8.2 Diff and Divergence Classification

`Diff` decodes two fingerprints, refuses an unknown format version, and lists every probe in the shared set whose verdict differs. `Classify` then assigns each divergence exactly one class, testing the explanations in a fixed order so that a difference is never reported as policy when something else explains it (Figure 8.2).

The kernel-version test is the subtle one. A probe whose dependency names a `present_if` path or a `since` release is kernel-explained only when the other side is `permitted`: a kernel lacking the entry point on one side and a filter denying it on the other is still a policy difference on the second side, which is how a corrected classifier stopped hiding policy behind kernel differences (issue #88). A probe whose dependency names a module is kernel-explained when the module is loaded after the run on one side and not the other; otherwise the kernel releases are compared. A difference in LSM or in runtime version reaches the fourth class: it is a difference in the policy the kernel actually enforces. The classifier's verdict per pair is therefore a statement about *why* two nodes differ, which is the project's stated contribution.

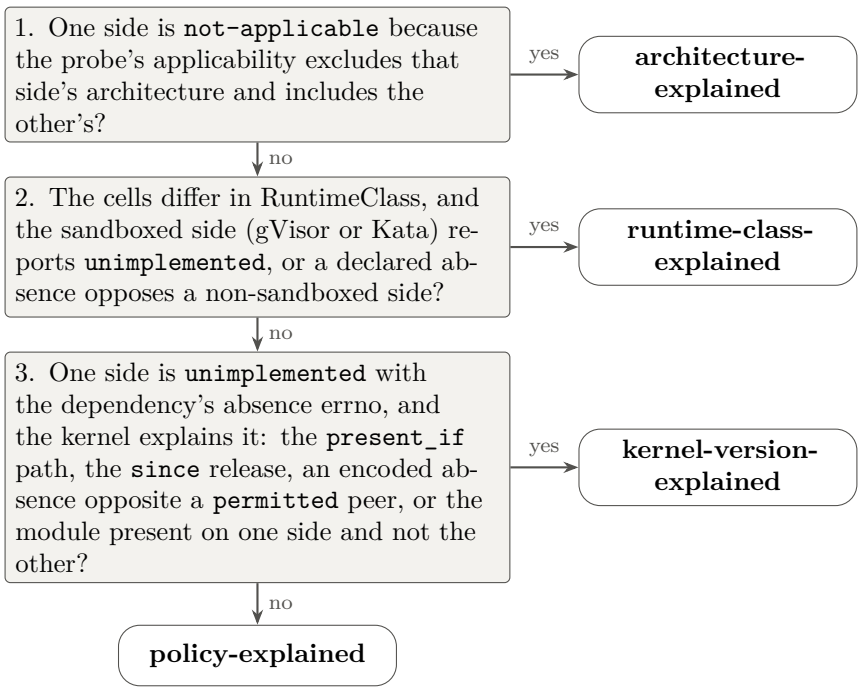


Figure 8.2: The order in which the classifier tests explanations for a divergence (`control/classifier.go`).

8.3 Attribution

The attributor (`control/attribution.go`) gives every result an attribution: a mechanism (`seccomp`, `lsm`, `capability`, `mount` or `kernel-limit`) or none, a confidence, an `undecidable` flag with the candidate mechanisms, and a reason in words. It works in three stages: the errno oracle, correlation with the process's own recorded state, and elimination by the cell. It never reads host state.

8.3.1 Stage 1: the errno oracle

Table 8.2 gives the reading of each verdict and errno against the probe's oracle.

Before the reading is taken, the probe's `lsm_hook_since` is applied to the cell's kernel release: on that release or later (or when the release cannot be parsed), an oracle that isolates seccomp is downgraded to `seccomp-or-lsm`. The same io_uring denial is therefore decided on a 5.14 kernel and ambiguous on a 7.0 kernel, which is exactly what the kernel's code implies.

8.3.2 Stage 2: correlation

Correlation narrows a denial using facts the process recorded about itself:

- **Capability.** For an `EPERM` on a capability-gated probe, if the capability is absent from the effective set but present in the bounding set, the missing capability is the cause (low confidence). If it is present in the effective set, seccomp and an LSM hook remain, and the capability as well when the process is not in the initial user namespace (under `hostUsers: false` a held capability need not hold against the node's objects). If it is absent from both sets, seccomp and the capability remain. For CAP_SYS_PTRACE and CAP_SYS_RAWIO the LSM is always added, because Yama and lockdown can refuse them whatever the major LSM's label says.
- **`EACCES`.** On a capability-gated call, `EACCES` is decided only when CAP_DAC_OVERRIDE is held in the initial user namespace; otherwise a

Result	Reading	Confidence
`killed`	seccomp: only a `KILL` action produces SIGSYS	high
`permitted`, `unimplemented`, `not-applicable`	not a denial	high
`timed-out`	undecidable: hung and slow are not distinguished	low
`denied` `ENODATA` / `EROFS` (path family)	mount: the probe's encoding of a mask or a read-only mount	high
`denied` `ENOSYS`	seccomp: the kernel implements the call, so `ENOSYS` is a filter	high with a guaranteed errno, else medium
`denied` `EPERM`, oracle isolates `seccomp`	seccomp: nothing before the oracle's point answers `EPERM` but a filter at entry	high with a guaranteed errno, else medium
`denied` `EPERM`, `seccomp-or-lsm`	undecidable between seccomp and an LSM	low
`denied` `EPERM`, `undecidable`, capability-gated	undecidable between seccomp and the missing capability	low
`denied`, any other errno	undecidable unless correlation decides it	low

Table 8.2: Stage 1 of attribution: the oracle's reading.

file-permission check, which is not one of the mechanisms the attributor names, could have produced it. On a `seccomp-or-lsm` probe under a confining, enforcing AppArmor or SELinux label, `EACCES` after the LSM hook is attributed to the LSM.

- **User-namespace limits.** `ENOSPC` from a user-namespace probe with `user.max_user_namespaces` at zero is the kernel's own limit; an `EPERM` while a restricting sysctl is set adds `kernel-limit` to the candidates.

8.3.3 Stage 3: elimination by the cell

The cell can rule candidates out. seccomp mode 0 means no filter is installed. A non-enforcing (or absent) LSM refuses nothing, except through Yama and lockdown, which have no label, and except AppArmor's user-namespace restriction, which applies to unconfined tasks too. A capability in the effective set of the initial user namespace is not missing. When one candidate survives it is the attribution, at medium confidence; when none or several survive, the denial stays undecidable with the remaining candidates.

8.3.4 Confidence and the undecidable fraction

High is a positive proof (a guaranteed errno replaced by `EPERM`, a SIGSYS, an encoded mask). *Medium* is a structural reading without a guaranteed errno, or a single survivor of elimination. *Low* is correlation or an unresolved denial. The *undecidable fraction* is the number of undecidable denials over all denials, killed results included. It is reported in every report as a measured quantity rather than assumed to be zero, and reachability, the primary measurement, does not depend on it.

8.3.5 What is not attempted

Reading the filter itself needs `PTRACE_SECCOMP_GET_FILTER` and CAP_SYS_ADMIN. The privileged helper that would use it was outlined in the proposal and was neither specified nor built. Denials that the three stages cannot decide are reported as undecidable.

8.4 The Reference Posture

The AF_ALG case is not a claim violation: Pod Security Standards at Restricted assert nothing about AF_ALG, and RuntimeDefault's contract is whatever the runtime ships. Measured against the manifest alone the case produces no finding, because the declaration and the measurement agree and both are weaker than practitioners believe. What failed was an *expectation*. The reference posture is the model of that expectation: for each tier it asserts which kernel entry families a workload should not reach, and each assertion carries a rationale and a citation so that it can be audited rather than accepted on authority. Posture v1 (`posture/v1/posture.yaml`) is released with the instrument. Its tiers are in Table 8.3 and its assertions in Table 8.4; assertions inherit downward, so every assertion at `restricted` also applies at `untrusted-multi-tenant`.

Tier	Describes	Why it is stronger than the PSS level it is named after
`privileged`	Infrastructure needing node access: CNI plugins, storage drivers, node agents	It is the PSS Privileged floor and adds nothing, so a workload labelled privileged unnecessarily can be identified
`baseline`	Ordinary services with no adversarial input: internal APIs, batch workers	Adds entry families a service should never reach, even when RuntimeDefault permits them
`restricted`	Services handling untrusted requests: public APIs, ingress, image processing	Asserts a smaller kernel surface independently of RuntimeDefault, so the expectation is stable across runtime versions
`untrusted-multi-tenant`	Code the operator does not trust, sharing a node: CI jobs running pull requests, serverless functions, customer plugins	Kubernetes has no level above Restricted; this tier adds the full isolation model: no kernel crypto API, module loading, perf events, raw netlink, io_uring, unmasked procfs

Table 8.3: Tiers of posture v1.

io_uring is asserted at both `restricted` and `untrusted-multi-tenant`; with inheritance the second is redundant, so the 15 assertions state 14 distinct expectations. Five assertions concern deferred families. They are kept deliberately: the evaluator reports them as asserted but not measured, so that the posture states what a tier requires independently of what the corpus currently measures. The posture is deliberately stronger than the tiers it is named after, and that gap is the point of it.

8.5 The Evaluator

`EvaluateFindings` joins a fingerprint with one workload's manifest and a posture tier, and places every `permitted` result in one or more categories:

Tier	Family	Probes	Citation
baseline	mount	`mount.reach`; `mount.fs.*`: proc, overlay, fuse	mount(2) [3]
baseline	device	`device.open.*`: fuse, kvm, mem	OCI runtime spec, devices [6]
baseline	path	`path.masked.proc_kcore`	OCI runtime spec, maskedPaths [6]
restricted	io_uring	whole family	io_uring documentation [59]
restricted	capability	`sys_module`, `sys_boot`, `sys_time`, `sys_rawio`, `sys_ptrace`	capabilities(7) [3]
untrusted	socket	`socket.family.af_alg`	CVE-2026-31431 [14], CVE-2017-2636 [60]
untrusted	netlink	`netfilter`, `xfrm`, `crypto`, `rdma`	CVE-2024-1086 [61]
untrusted	io_uring	whole family	io_uring documentation [59]
untrusted	path	`proc_kcore`, `proc_keys`	OCI runtime spec, maskedPaths [6]
untrusted	clone	`unshare.flags.newuser`, `clone.flags.newuser`	CVE-2022-0185 [62]
untrusted	`perf_event_open`	deferred family	CVE-2013-2094 [63]
untrusted	bpf	deferred family	CVE-2017-16995 [64]
untrusted	keyctl	deferred family	CVE-2016-0728 [65]
untrusted	userfaultfd	deferred family	userfaultfd documentation [66]
untrusted	`open_by_handle_at`	deferred family	the “Shocker” escape [67]

Table 8.4: The 15 assertions of posture v1 and their citations (16 in all). “untrusted” abbreviates `untrusted-multi-tenant`.

Expectation violation An assertion at the evaluated tier or below names the probe's family and either lists the probe or covers the whole family. The finding carries the assertion's rationale and citations, and, for probes with a module dependency, whether the module was loaded before the run and whether the run loaded it.

Claim violation The manifest declares the probe's capability unavailable, either by dropping it (or `ALL`) explicitly or through PSS Restricted's requirement to drop all capabilities, and the probe was permitted. Two guards keep this honest: only probes whose oracle is `undecidable` (whose answer is the capability check itself, with nothing earlier that could pass for it) are eligible, and CAP_NET_BIND_SERVICE is excluded because PSS Restricted allows adding it. RuntimeDefault and Localhost seccomp profiles are deliberately not treated as claims: their contents cannot be inferred from the manifest.

Undeclared exposure Reachable, and neither the posture tier nor the manifest addresses it. An input to corpus and posture development.

CI evaluates at `restricted` with `deploy/k8s/job.yaml` as the manifest; the closure experiment evaluates at `untrusted-multi-tenant`.

8.6 The Remediator

`GenerateRemediation` takes the evaluated expectation violations, the runtime's *actual* RuntimeDefault profile (captured from containerd on the node, not assumed), and a target (runtime architecture, libseccomp architecture, capabilities and kernel release, used to resolve the profile's architecture-, capability- and kernel-gated rules for that node). It keeps the profile's default action and allowlist and narrows only what the findings require. Table 8.5 lists the closure for each probe.

Finding	Closure
`socket.family.af_alg`	`socket`: family 38 no longer allowed
`netlink.protocol.*`: netfilter, xfrm, crypto, rdma	`socket(AF_NETLINK, *, p)` no longer allowed for $p \in \{12, 6, 21, 20\}$
`clone.flags.newuser`, `unshare.flags.newuser`	`clone` / `unshare` allowed only when the `CLONE_NEWUSER` bit is clear; `clone3` answers `ENOSYS` so the flag cannot be passed through a pointer
`io_uring.*`	remove `io_uring_setup`, `io_uring_enter` and `io_uring_register` from the allowlist (opcodes are behind a pointer)
`capability.sys_*`: time, boot, module, rawio, ptrace	remove the gated calls (for example `reboot`, `kexec_load`, `kexec_file_load`)
`path.*`, `device.*`, `mount.*`	not closable: the deciding argument is a pathname or a filesystem type behind a pointer

Table 8.5: Closure rules of the remediator.

8.6.1 Narrowing rather than excepting

libseccomp combines comparisons on the same argument within one rule with a logical OR, so an `ERRNO` exception placed beside a broad `ALLOW` rule for `socket` cannot close a family. The remediator therefore replaces a broad `socket` allow with one masked-equality allow per family that remains permitted, using a 32-bit mask so that high register bits cannot bypass the family test, and does the same for netlink protocols. For `clone` and `unshare` it adds a masked-equality condition to the existing allow so that it matches only when the user-namespace bit is clear. It never adds an allow rule the runtime's default did not have. Probes of one family that share a rule (every io_uring probe) are closed by the first rule; the rest are recorded as closed by it.

8.6.2 Outputs

The primary output is a Security Profiles Operator [55] `SeccompProfile` custom resource, which the operator distributes to every node and which solves the delivery problem on managed clusters, where a Localhost profile file cannot be placed in the kubelet's seccomp root without a custom node image. The secondary output is the equivalent OCI Localhost profile. The profile's name includes a hash of its rules, so different remediations never overwrite one another.

8.6.3 Closure verification

`caliper-pipeline` (driven by `deploy/gce/run-pipeline.sh`) measures the cell under RuntimeDefault from a fresh boot, captures the RuntimeDefault containerd applied, evaluates, generates the remediation, has SPO install the resource where a cluster with the operator is available, and measures again from a fresh boot under the generated Localhost profile. Each finding is then closed (the re-probe shows it denied or killed), `not-closable` (the generator could not express it), or `unenforced` (the re-probe ran with seccomp mode 0, so no profile could reach it). The generator's own claim is never counted.

8.7 Reporting

`BuildReport` combines findings, classified divergences and attribution. Each fingerprint is labelled by its position and the first 12 hexadecimal digits of its digest. `WriteJSONReport` and `WriteTextReport` render the same report for machines and people; both include the authorisation notice, the attribution confidence counts and the undecidable fraction. For expectation violations with a module dependency, the text report prints the module's state, for example `af_alg: not loaded before run, loaded by run`. In CI, `caliper-ci-report` posts the report as a pull-request comment and updates it in place on later pushes. SARIF output was deferred.

Chapter 9
Technology Stack

9.1 Overview

The technology choices follow from the two halves of the architecture. The measurement half must add nothing to the kernel surface it measures, so it is a single static binary with no runtime and no shared libraries. The analysis half needs the container ecosystem's libraries and is written in the language they are written in. Table 9.1 summarises the stack; the sections below give the reasons.

Layer	Technology
Probe engine	Rust 1.93 (pinned in `rust-toolchain.toml`); targets `x86_64-` and `aarch64-unknown-linux-musl`; linker `rust-lld`
Engine crates	`nix` and `libc` (system calls), `serde` and `serde_json` (fingerprint), `seccompiler` (test fixtures only)
Probe image	two-stage Dockerfile: Debian bookworm to build, `scratch` to ship; the image contains the binary and nothing else
Control plane	Go 1.26; `k8s.io/client-go` v0.32.4; `go-github` v68; `gopkg.in/yaml.v3`
Kernel interfaces	seccomp-BPF, capabilities, namespaces, cgroups v2, procfs and sysfs, `pidfd_open`, io_uring registration, netlink, AppArmor and SELinux attribute interfaces
Container runtimes	containerd 1.7.12, 2.2.1 and 2.2.7; runc 1.1.12 and 1.3.4; gVisor release-20261005.0; Kata Containers 3.14.0; `crictl`
Orchestration	Kubernetes Job and DaemonSet; Pod Security Admission; RuntimeClass; Security Profiles Operator v1.1.1 with cert-manager v1.21.2 (acceptance test)
Cloud	Google Compute Engine (six cells), Google Kubernetes Engine (CI target), region `asia-south1`, zone `asia-south1-a`
CI/CD	GitHub Actions (`ci.yml`, `cluster-evaluation.yml`, `release.yml`); actions pinned by commit SHA
Release	GitHub Container Registry; Docker Buildx `imagetools`; Sigstore cosign v3.1.3 [68], keyless via the workflow's OIDC identity
Comparison tools	`amicontained` v0.4.9; `seccomp-diff` at commit `68d4f45`
Development	Lima virtual machines on Apple silicon (arm64 with Apple's Virtualization framework; amd64 under QEMU) from `scripts/dev-vm.yaml`, with Docker, `strace` and a Rust toolchain
Documentation	Markdown in the repository; this report in LaTeX (built with Tectonic)

Table 9.1: Technology stack.

9.2 Programming Languages

- **Rust (probe engine and corpus).** The engine must fork before each probe, must not issue system calls of its own, and must survive a child killed by SIGSYS. Rust gives direct control of system calls through `nix` and `libc` without a runtime, and compiles against musl into a fully static binary. The release profile sets `panic =`

`"abort"` (a probe child must never unwind), link-time optimisation and stripping. Probe definitions are Rust code, so the oracle's reasoning about kernel code paths sits beside the call it describes.

- **Go (control plane).** Fingerprint handling, diffing, evaluation, remediation and the Kubernetes and GitHub integrations use `client-go` and `go-github`, and the remediator emits Security Profiles Operator resources whose types are defined in Go. The JSON shared with the engine uses `encoding/json`.
- **Shell (delivery).** Cell provisioning, measurement and the pipeline driver are Bash scripts checked with `shellcheck` in CI.

9.3 Kernel and Runtime Interfaces Used

The engine uses only interfaces available to an unprivileged process: `fork`, `pidfd_open`, `ppoll` and `wait4` for the harness; `uname` for identity; reads of `/proc/self/status`, `/proc/self/attr/*`, `/proc/modules`, `/proc/self/uid_map` and a handful of sysctls for the cell; `/proc/self/mountinfo`, `/proc/keys` or `/proc/key-users`, `/sys/fs/cgroup`, `/proc/sysvipc/*`, `/dev/shm`, `/dev/mqueue` and `/proc/*/ns/net` for residual state; and, for probes, exactly the call under test. The cell scripts use `crictl` to create pods through containerd's CRI the way the kubelet does, so a measurement on a cell has the same confinement as a pod scheduled there.

9.4 Infrastructure

9.4.1 Compute Engine cells

The six evaluation cells are virtual machines in a dedicated Google Cloud project, created and managed by `deploy/gce/cells.sh`. Compute Engine supplied what developer hardware could not: native amd64 machines and nested virtualisation for the Kata cell, an arm64 machine type (`c4a-standard-2`) running the same Ubuntu kernel build as the amd64 cells, and the ability to rebuild any cell from its provisioning script. Machines are stopped between measurements; the cost of the evaluation, including the GKE cluster, was covered by a free-trial credit.

9.4.2 GKE cluster for continuous integration

A zonal GKE cluster with one `e2-medium` node on Container-Optimized OS (containerd 2.2.7, kernel 6.12) measures every pull request. The measurement namespace enforces the `restricted` Pod Security level, and a service account with a namespaced role that can only create, watch, read logs from and delete Jobs and pods is the only credential the workflow holds.

9.5 Continuous Integration and Release

9.6 Version Control and Collaboration

The repository [69] is public on GitHub under the Apache License 2.0, chosen for its express patent grant and because it is the licence of the projects CALIPER measures and integrates with. `main` is protected: changes arrive by pull request with an approving review from the code owner, the strict status checks of all CI jobs, linear history and resolved conversations. Work was tracked as GitHub issues, one milestone per freeze gate, with the workstream letter in every title and the person as assignee. Secret scanning with push protection, Dependabot alerts and private vulnerability reporting are enabled. At

Workflow / job	Runner	Checks
`ci.yml`: scripts	ubuntu-24.04	`shellcheck` on every script; reference posture validation (every assertion has a rationale and a citation, every probe id exists); Kubernetes manifest validation
`ci.yml`: engine	ubuntu-24.04	static musl build and tests; the frozen corpus index matches `--dump-corpus`; no dynamic loader in the binary; system-call baseline; the probe image builds from `scratch`; the reduced corpus omits every autoloading probe; the reduced corpus runs within its 5-second budget
`ci.yml`: engine-arm64	ubuntu-24.04-arm	the same build, index, loader and baseline checks natively on arm64
`ci.yml`: control	ubuntu-24.04	`go build`, `go vet` and `go test`, including every metric regeneration test
`cluster-evaluation.yml`	ubuntu-24.04	builds the pull request's probe image, runs the reduced corpus as a Job on the GKE cluster, evaluates at `restricted`, and posts or updates the report comment; skipped for forks
`release.yml`	native amd64 and arm64	on a `v*` tag: native image builds, a joined multi-architecture image, the image's `--version` checked against the tag, keyless cosign signature of the digest, and a GitHub release carrying the posture, its rationale and the corpus index

Table 9.2: Continuous integration and release workflows.

the time of writing `main` has 116 commits from 87 merged pull requests, and 71 of 73 issues are closed; the two open issues are this report and the demonstration.

9.7 Development Environment

Development happened on Apple-silicon laptops. `scripts/dev-vm.yaml` defines a Lima virtual machine with Docker, `strace` and a Rust toolchain that mounts the repository at its host path; the engine was cross-compiled on the host and run in the VM, and an emulated amd64 VM provided a real x86_64 kernel and runc for the x86_64 baseline and hardened profile. A snapshot of the VM was the rollback after any probe experiment. Go code was built and tested in a `golang:1.26` container inside the VM, so that every developer ran the same toolchain as CI.

Chapter 10

Experimental Setup

10.1 The Environment Matrix

The unit of evaluation is the environment cell: the tuple of architecture, runtime and version, kernel, distribution and loaded modules, LSM and RuntimeClass. Cells are counted honestly: kind, minikube and k3s on one host are one cell, not three, because all three are containerd on the same kernel. Cell 2 is the baseline, and every other cell moves as few axes as possible relative to it, so that a divergence is attributable to one thing (Figure 10.1, Table 10.1).

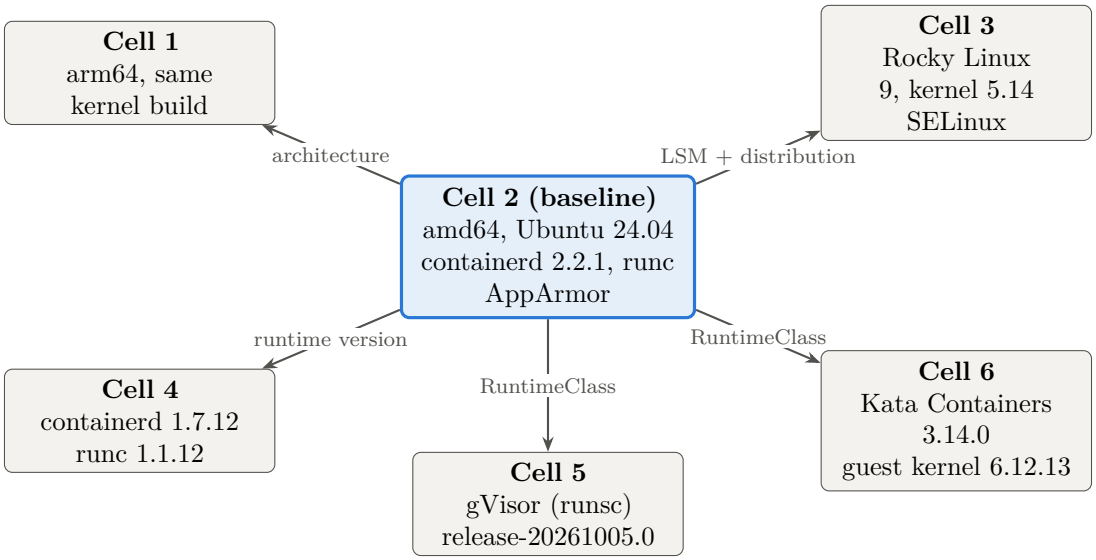


Figure 10.1: The six cells and the axis each moves relative to the baseline, cell 2.

Cell	Arch	Distribution	Kernel	Runtime	LSM
1	arm64	Ubuntu 24.04	7.0.0-1014-gcp	containerd 2.2.1, runc 1.3.4	AppArmor
2	amd64	Ubuntu 24.04	7.0.0-1014-gcp	containerd 2.2.1, runc 1.3.4	AppArmor
3	amd64	Rocky Linux 9	5.14.0-687.56.1.el9_8	containerd 2.2.1, runc 1.3.4	SELinux
4	amd64	Ubuntu 24.04	7.0.0-1014-gcp	containerd 1.7.12, runc 1.1.12	AppArmor
5	amd64	Ubuntu 24.04	host 7.0.0-1014-gcp; sandbox reports 4.19	containerd 2.2.1, gVisor release-20261005.0	none in sandbox
6	amd64	Ubuntu 24.04	guest 6.12.13	containerd 2.2.1, Kata 3.14.0	none in guest
GKE	amd64	Container-Optimized OS	6.12.105+	containerd 2.2.7, runc	AppArmor

Table 10.1: Environment cells. Machines: cell 1 `c4a-standard-2`; cells 2 to 5 `e2-standard-2`; cell 6 `n2-standard-2` with nested virtualisation; GKE `e2-medium`.

Cells 4, 5 and 6 each move exactly one axis. Cell 3 moves the LSM and the distribution together, because SELinux is not a supported Ubuntu configuration; divergence in that

pair is attributed to policy only after the kernel-version class has been excluded probe by probe. Cell 1 was expected to move the kernel build with the architecture, but on Compute Engine the arm64 Ubuntu image runs the same kernel release as the amd64 one, so cell 1 moves the architecture alone, which is stronger than the proposal expected. The containerd versions are pinned rather than left to the host's patch state: Ubuntu 24.04 ships 1.7.12 in its release pocket and 2.2 in updates, and both predate the AF_ALG deny, so cells 2 and 4 together carry the AF_ALG reproduction while sitting on either side of the io_uring change.

10.2 Provisioning

Each cell is created by `cells.sh create cellN`, which boots the VM from a pinned image and runs the cell's provisioning script in `deploy/gce/provision/`. Provisioning is designed so that a cell rebuilt later is the same cell:

- Ubuntu cells read the package archive through the snapshot `20261016T120000Z`; the kernel, AppArmor, libseccomp and the runtime are held, and unattended upgrades are disabled. Cell 3 version-locks its runtime, policy and kernel.
- Every download (gVisor, Kata, `crictl`) is checked against a pinned hash.
- Provisioning fails unless the containerd on `PATH`, the running daemon and runc are the pinned versions, and unless the cell's RuntimeClass handler is registered.
- The packages that define the cell are recorded in `/etc/caliper/packages` and committed beside each fingerprint.
- Cells 5 and 6 register two handlers each: the default (`runsc`, `kata`) and a variant that applies seccomp inside the sandbox (`runsc-seccomp`, which sets gVisor's `oci-seccomp`; `kata-seccomp`, which sets `disable_guest_seccomp = false`).

10.3 Measurement Procedure

`cells.sh measure cellN <image>` reboots the cell first, so that modules an earlier run autoloaded are not recorded as the node's, and then runs `deploy/gce/measure.sh` on it. That script builds the pod the way the kubelet builds the standard Job (`deploy/k8s/job.yaml`) and runs it through containerd's CRI with `crictl`:

- its own network and PID namespaces;
- UID 65534, all capabilities dropped, `no_new_privs`, a read-only root filesystem and a writable `/tmp`;
- RuntimeDefault seccomp, and the runtime's default AppArmor profile where AppArmor is enabled, or SELinux labelling and a relabelled `/tmp` where SELinux is enforcing;
- the kubelet's default masked and read-only paths, taken from the kubelet's source, because an earlier version of the request omitted them and left `/proc` unmasked (a defect found in review and corrected before any recorded measurement);
- the RuntimeClass handler for cells 5 and 6.

The script fails if the probe fails or does not finish, and prints the cell's packages and the configuration containerd actually applied to standard error. The raw output is then completed by `caliper-complete` with the distribution, runtime, runtime version and RuntimeClass as operator-sourced facts, and digested. The released v1.0.0 image (amd64 and arm64) is pinned by digest in `cells.sh`. The recordings in `fixtures/cells/` were made earlier with images built locally from commit `529048c`, and every run's `.packages` file records the exact image used.

10.4 Ground-Truth Environments

Attribution accuracy needs denials whose cause is known. `measure.sh` has ground-truth modes, each a run on cell 2 from a fresh boot with every control off (UID 0, all capabilities, seccomp and AppArmor unconfined, no masked or read-only paths) except the one the mode names (Table 10.2). A control run with no control in force gives the kernel's own denials: seven device nodes the runtime does not create (`ENOENT`) and `capability.sys_ptrace`'s `EFAULT` on its capable path. The same probe denied with the same errno is left out of every environment.

Environment	Only control in force	Fingerprint
control	none	`gt-none.json`
seccomp	containerd's RuntimeDefault seccomp profile	`gt-seccomp.json`
seccomp-hardened	the hand-hardened profile as a Localhost profile	`gt-seccomp-hardened.json`
apparmor	containerd's RuntimeDefault AppArmor profile, enforcing	`gt-apparmor.json`
capability	all capabilities dropped, UID 65534; Ubuntu's AppArmor user-namespace restriction lifted	`gt-capability.json`
paths	the kubelet's masked and read-only paths	`gt-paths.json`
userns	`user.max_user_namespaces=0`, the AppArmor restriction lifted for the run	`gt-userns.json`

Table 10.2: Single-cause environments for attribution accuracy (`fixtures/ground-truth/`).

10.5 Remediation, Residual and Runtime Runs

- **Closure.** `run-pipeline.sh cellN` with `CALIPER_POSTURE_TIER=untrusted-multi-tenant` measures under RuntimeDefault, captures containerd's applied profile, evaluates, generates the remediation, and re-measures from a fresh boot under the generated Localhost profile; outputs in `fixtures/pipeline/<cell>[-<handler>]/`. The tier is `untrusted-multi-tenant` because at `restricted` cells 1 to 3 have nothing to close.
- **Residual and runtime.** The full and the reduced corpus on cells 1 to 6, each from a fresh boot, and the full corpus on cell 2 with every control off, so that every object-creating probe actually creates its object; the runs are in `fixtures/runtime/`.
- **SPO acceptance.** A temporary two-node GKE cluster with cert-manager v1.21.2 and the Security Profiles Operator v1.1.1 installed cell 2's generated `SeccompProfile`, which reached the `Installed` state on both nodes on 2026-10-24; the cluster was deleted afterwards.
- **Ancestors.** On cell 2, CALIPER and `amicontained` inspected the same long-lived container created through the CRI with the standard security context; after a restart, `seccomp-diff` attached from the host to a fresh BusyBox container with the same settings and extracted its filter.

10.6 Continuous Evaluation on GKE

Every pull request is measured on the GKE cluster by `cluster-evaluation.yml`: the workflow builds the pull request's probe image, applies it as the standard Job in a namespace that enforces `restricted`, waits for completion, reads the fingerprint from the pod's

log, completes it with the Node and RuntimeClass objects, evaluates at `restricted`, and posts the report. The probe's own output may be preceded by warnings; the reporter skips to the first valid JSON document. Forked pull requests are skipped so that cluster credentials never reach untrusted code.

10.7 Reproducibility

Every fingerprint, ground-truth run, pipeline run and runtime run is committed under `fixtures/`, each with a `.packages` file recording the probe image digest, the cell's packages and what containerd applied. Every metric in Chapter 11 is regenerated from those fixtures by a command in `control/cmd`, and a test compares each generator's output with the committed metric file, so a change that alters a reported number fails CI (Appendix A).

Chapter 11

Results and Analysis

This chapter reports every success criterion of the proposal with its measured value, and states where a result is weaker than its headline figure. Every number is generated from fingerprints committed to the repository (Appendix A). Charts are paired with the tables that carry their exact values.

11.1 Verdicts per Cell

Table 11.1 and Figure 11.1 give the verdicts of the full corpus under RuntimeDefault on each cell, on the GKE node, and on cell 2 under the hand-hardened profile. No probe timed out on any run, and no probe was killed except under the hardened profile.

Run	Permitted	Denied	Unimpl.	Killed	N/A	Total
Cell 1 (arm64)	53	106	25	0	1	185
Cell 2 (baseline)	53	107	25	0	0	185
Cell 3 (SELinux, 5.14)	44	105	36	0	0	185
Cell 4 (containerd 1.7)	105	55	25	0	0	185
Cell 5 (gVisor)	37	29	119	0	0	185
Cell 6 (Kata)	101	37	47	0	0	185
GKE (COS, containerd 2.2.7)	44	106	35	0	0	185
Cell 2, hardened profile	27	20	13	122	0	182

Table 11.1: Verdicts of the full corpus. Under the hardened profile the three `mount.fs.*` probes were recorded as unmeasured: the filter killed their set-up (creating the run's mount directory), which the harness reports as a `setup-killed` fault rather than a verdict.

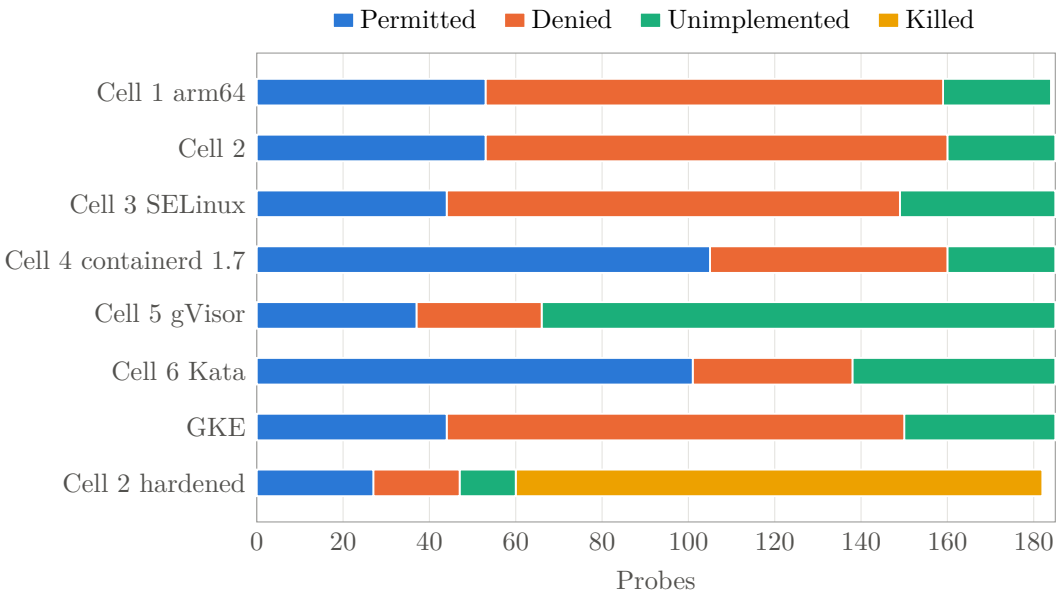


Figure 11.1: Verdict distribution per run (values in Table 11.1). Cell 1's single not-applicable probe is omitted.

Three observations follow directly. Cells 1 and 2 are nearly identical: same kernel build, same runtime, different architecture. Cell 4 permits 52 more probes than cell 2 because containerd 1.7.12's RuntimeDefault still allows io_uring. Cell 5 reports 119 entry points unimplemented, because gVisor implements a subset of Linux, and cell 6 permits 101 probes because Kata applies no seccomp inside its guest by default. The hardened run on cell 2 is the survivability test at scale: 122 children were killed by SIGSYS and the run completed normally, with every kill recorded as a verdict.

11.1.1 Verdicts by family

Table 11.2 breaks the six cells down by family, as permitted/denied/unimplemented.

Family	n	Cell 1	Cell 2	Cell 3	Cell 4	Cell 5	Cell 6
socket	24	10/5/9	10/5/9	5/4/15	10/5/9	4/1/19	6/1/17
netlink	18	15/0/3	15/0/3	14/0/4	15/0/3	3/2/13	11/0/7
io_uring	53	0/53/0	0/53/0	0/53/0	53/0/0	0/0/53	53/0/0
mount	38	25/4/9	25/4/9	22/4/12	25/4/9	10/4/24	20/3/15
clone	19	0/19/0	0/19/0	0/19/0	0/19/0	8/11/0	6/13/0
path	17	1/12/4	1/12/4	1/11/5	1/12/4	7/0/10	3/6/8
device	8	1/7/0	1/7/0	1/7/0	1/7/0	3/5/0	1/7/0
capability	8	1/6/0*	1/7/0	1/7/0	0/8/0	2/6/0	1/7/0

Table 11.2: Verdicts by family and cell (permitted/denied/unimplemented). *`capability.sys_rawio` is not applicable on arm64.

The table shows where each cell's confinement lives. On the runc cells every clone and unshare probe is denied (RuntimeDefault and the dropped capabilities), every path probe but the control is masked, read-only or absent, and the socket and netlink families are mostly permitted, because RuntimeDefault filters `socket` only by a few families. Under gVisor the path family is permitted (7) or absent (10) and none is masked; under Kata three path probes are permitted. Both are explained in Chapter 12.

11.2 Divergence across Cells

Criterion: policy-explained divergence in at least four cell pairs, with architecture- and runtime-class-explained divergence separated from it in every pair.

Table 11.3 gives the number of divergent probes in each of the 15 pairs, by class, and Figure 11.2 plots them. Policy-explained divergence is present in 12 pairs, so the first half of the criterion is met.

The pairs behave as the matrix was designed to make them behave:

- **Cells 1 and 2** differ in one probe, `capability.sys_rawio`, which exists only on x86; it is classified architecture-explained, as it is in every pair with cell 1.
- **Cells 2 and 4** differ in 54 probes: the 53 io_uring probes that containerd 1.7's RuntimeDefault allows, and `capability.net_bind_service`, which containerd 2.x permits by enabling unprivileged low ports by default. Both are runtime-version differences, and therefore policy.
- **Cell 3 against the other runc cells** (pairs 1–3, 2–3 and 3–4) shows 15 kernel-version-explained differences (socket families, netlink protocols, mount types and masked paths that the 5.14 kernel lacks) and *no* policy-explained difference against cells 1 and 2. Its pairs with cells 5 and 6 show none in that class; the runtime-class test is applied before

Pair	Architecture	Kernel version	Runtime class	Policy	Total
1, 2	1	0	0	0	1
1, 3	1	15	0	0	16
1, 4	1	0	0	54	55
1, 5	1	0	96	19	116
1, 6	1	0	30	63	94
2, 3	0	15	0	0	15
2, 4	0	0	0	54	54
2, 5	0	0	96	19	115
2, 6	0	0	30	63	93
3, 4	0	15	0	54	69
3, 5	0	0	85	19	104
3, 6	0	0	19	63	82
4, 5	0	0	96	20	116
4, 6	0	0	30	11	41
5, 6	0	0	74	12	86
total	5	45	556	451	1057

Table 11.3: Divergent probes per cell pair and class (`fixtures/metrics/divergence-metrics.json`).

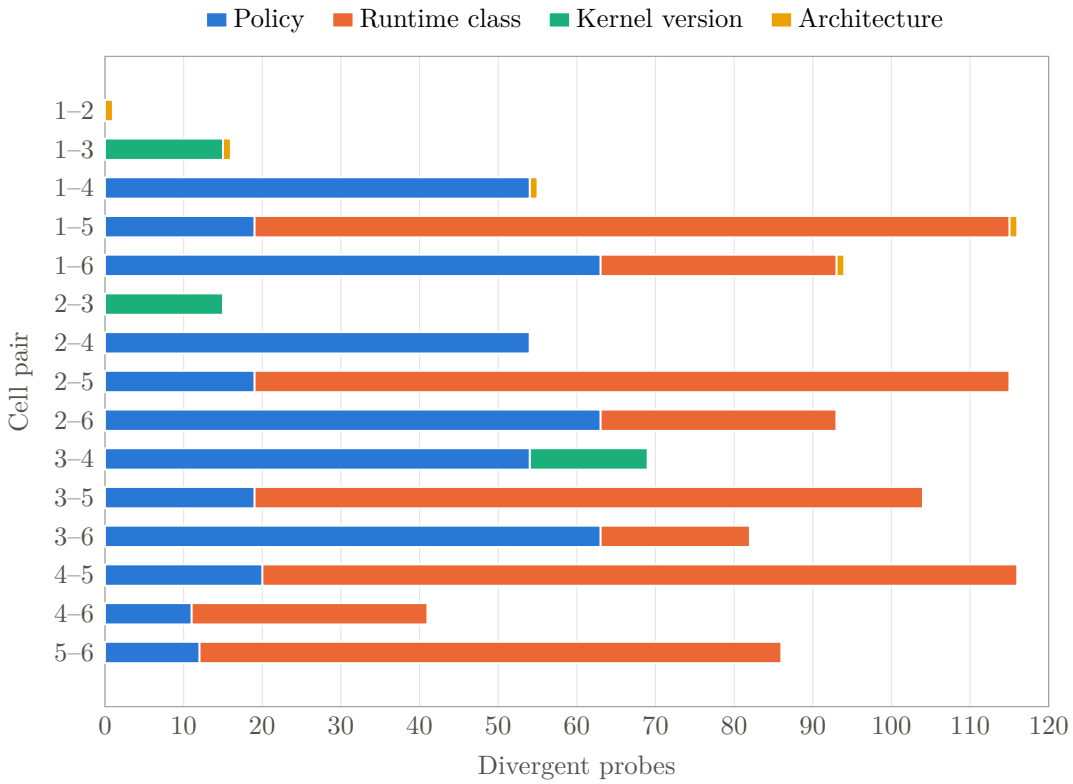


Figure 11.2: Divergence per cell pair by class (values in Table 11.3). Without the classifier, all 1,057 divergent probes would read as policy differences; 606 of them are explained by architecture, kernel version or RuntimeClass.

the kernel-version test. Replacing AppArmor with SELinux changed no verdict in this corpus that the kernel version did not already explain.

- **Pairs with cells 5 and 6** are dominated by runtime-class divergence (74 to 96 probes), which the classifier keeps out of the policy count. What remains as policy is real. Against cell 2, gVisor's 19 are eight clone and unshare probes it permits (among them new user, mount and time namespaces), six masked or read-only paths it does not apply, the FUSE and TUN devices, `capability.sys_nice`, and two netlink protocols it refuses; Kata's 63 are the 53 io_uring probes, six clone and unshare probes and `AF_VSOCK`, all permitted because no seccomp filter is applied in the guest, plus `mount.reach` and the two masked directories.

Without the classifier, the headline divergence metric would be dominated by gVisor's unimplemented interfaces and by kernel differences: 606 of the 1,057 divergent probe pairs are explained by something other than policy. The classifier's correctness in every pair rests on its unit tests and on these per-pair counts matching the axis each pair moves. Unlike attribution, it was not measured against a constructed ground truth.

11.3 Expectation and Claim Violations

Criterion: at least one expectation violation in at least three distinct cells.

At the `restricted` tier, which CI evaluates, the six cells give 108 expectation violations, 3 claim violations and 282 undeclared exposures (Table 11.4). Three cells have expectation violations, so the criterion is met, but one of the three rests on a single finding of doubtful weight. Cell 5's only violation at this tier is `device.open.fuse`. Under gVisor that device node is the sandbox's own FUSE implementation, not the host driver that the posture assertion's rationale is about. Without it two cells qualify, and the criterion is not met at this tier.

At the `untrusted-multi-tenant` tier every cell has expectation violations: 5, 5, 4, 58, 3 and 59 on cells 1 to 6. Cell 5 still has two there without the FUSE device (`clone.flags.newuser` and `unshare.flags.newuser`). The criterion is therefore met without qualification at that tier, and with the qualification above at `restricted`. Both figures are reported because the project decided to state the dependence openly rather than lean on the `restricted` number.

	`restricted`			`untrusted-multi-tenant`
Cell	**Expectation**	**Claim**	**Undeclared**	**Expectation**
1 (arm64)	0	0	53	5
2	0	0	53	5
3 (SELinux)	0	0	44	4
4 (containerd 1.7)	53	0	52	58
5 (gVisor)	1	3	33	3
6 (Kata)	54	0	47	59
total	108	3	282	134

Table 11.4: Findings at the `restricted` tier, and expectation violations at `untrusted-multi-tenant` (`fixtures/metrics/finding-metrics.json`).

The expectation violations at `restricted` are the 53 io_uring probes on cell 4 (containerd 1.7) and on cell 6 (Kata, no seccomp in the guest), `mount.reach` on cell 6, and

`device.open.fuse` on cell 5. At `untrusted-multi-tenant` the runc cells add AF_ALG and the crypto, netfilter, rdma and xfrm netlink protocols (cell 3 has no rdma finding, because its kernel lacks the protocol).

11.3.1 Undeclared exposures by family

Table 11.5 breaks the 282 undeclared exposures down by family. They are the input the proposal intended them to be: reachable surface that neither the manifest nor the posture addresses. The mount types (up to 25) and netlink protocols (up to 14, beside the oracle probe) reachable on the runc cells are the largest groups; they would be the first candidates for a posture v2.

Cell	socket	netlink	mount	clone	path	device	capab.	io_uring	Total
1	10	15	25	0	1	1	1	0	53
2	10	15	25	0	1	1	1	0	53
3	5	14	22	0	1	1	1	0	44
4	10	15	25	0	1	1	0	0	52
5	4	3	10	6	7	2	1	0	33
6	6	11	19	6	3	1	1	0	47

Table 11.5: Undeclared exposures at `restricted` by family. Path includes the control probe on every cell; device includes the `/dev/null` control; socket and netlink include their out-of-range oracle probes where permitted.

11.3.2 The three claim violations

All three claim violations are on cell 5, and they differ in kind (Section 12.2). `clone.flags.newns` is a real missing capability check in gVisor: a new mount namespace is created without CAP_SYS_ADMIN. `capability.sys_nice` is a check gVisor skips deliberately, without effect, because niceness reaches no scheduler. `unshare.flags.newtime` is a false positive: gVisor accepts the flag, does nothing with it, and has no time namespace. A `permitted` verdict records that the call succeeded; on a reimplemented kernel such as gVisor's, success does not show that the operation took place, and the corpus's oracles are written against Linux.

11.4 The AF_ALG Case

Criterion: reproduce the AF_ALG case unaided. Binary.

Met. The workload is the standard probe Job (`deploy/k8s/af-alg-demonstration.yaml`, identical to `deploy/k8s/job.yaml`): a non-root user, all capabilities dropped, RuntimeDefault, the ordinary probe image and no bespoke profile (Table 11.6). On the GKE cluster the Job was admitted by a namespace enforcing PSS Restricted; on the cells the same pod specification was run through the CRI, where there is no admission step.

containerd added an AF_ALG deny to its RuntimeDefault profile in May 2026 (containerd PR 13327), released in 2.2.4 and backported to 1.7.32. The cells' 2.2.1 and 1.7.12 predate it, so they permit AF_ALG, matching the published result that containerd's RuntimeDefault did not block it; the GKE node's 2.2.7 carries the deny. The same manifest is exposed on one node and not on another, and the manifest does not record the difference. The two sides also differ in distribution and kernel, and `socket.family.af_alg` is a seccomp-or-LSM probe, so the instrument does not itself attribute the GKE denial: that it comes from RuntimeDefault rests on containerd's change history. At the

Recording	Runtime	AF_ALG	Finding at `untrusted-multi-tenant`
Cells 1, 2, 3	containerd 2.2.1	permitted	expectation violation (CVE-2026-31431)
Cell 4	containerd 1.7.12	permitted	expectation violation
Cell 5	gVisor	unimplemented (`EAFNOSUPPORT`)	none
Cell 6	Kata	permitted	expectation violation
GKE	containerd 2.2.7	denied (`EPERM`)	none

Table 11.6: `socket.family.af_alg` across recordings.

`untrusted-multi-tenant` tier the finding is an expectation violation carrying CVE-2026-31431 from the posture; at `restricted`, which asserts nothing about AF_ALG, it is an undeclared exposure. This is the case the proposal said a manifest-only checker would miss: measured against the manifest alone, it produces no finding.

`algif_aead` was absent from the module snapshot on every recording. The probe only opens an AF_ALG socket and never binds an `aead` algorithm, so the recordings do not measure whether that module can be autoloaded. They do show that the family's own module was autoloaded by the run (`af_alg: not loaded before run, loaded by run`) on cells 1, 2 and 4, which is the precondition for reaching it.

11.5 Attribution Accuracy

Criterion: at least 90 per cent of attributed denials correct, and at most 10 per cent of all denials undecidable, against a constructed ground truth.

Table 11.7 gives the result over the single-cause environments of Table 10.2. The two cases the proposal names as ambiguous are included: `clone3.args.short` denied with `ENOSYS` by a filter, and AppArmor answering `mount` with `EACCES`.

Environment	Only control	Denials	Correct	Incorrect	Undecidable
seccomp	RuntimeDefault	54	54	0	0
seccomp-hardened	hardened profile	123	123	0	0
apparmor	AppArmor profile	3	3	0	0
capability	all dropped, UID 65534	27	24	0	3
paths	masked, read-only paths	12	12	0	0
userns	no user namespaces	4	4	0	0
total		223	220	0	3

Table 11.7: Attribution against single-cause environments (`fixtures/metrics/attribution-accuracy.json`).

Both parts of the criterion are met: all 220 attributed denials are correct (100 per cent), and 3 of 223 denials (1.3 per cent) are undecidable. The three are `capability.sys_nice`'s `EACCES` without CAP_DAC_OVERRIDE, which a file-permission check could also have produced, and the `EPERM` of `capability.sys_ptrace` and `capability.sys_rawio`, which Yama and lockdown can also return whatever the major LSM's label says. Cell elimination is what makes the difference: with the oracles and correlations alone, 140 of the 223 are

attributed correctly and 83 (37.2 per cent) are undecidable; the cell decides 80 of the 83.

11.5.1 The undecidable fraction in production

The ground truth has limits that the headline figure hides. 177 of its 223 denials come from seccomp, three from AppArmor and none from SELinux, and all of it is on one cell. More importantly, the environments are built so that the cell rules out every mechanism but one. A production cell has several controls in force at once, and there the fraction is much higher (Table 11.8, Figure 11.3). These production fractions were computed for this report by running the attributor on the committed baseline fingerprints; unlike the other figures, they are not a committed metric file.

Cell	Denials	Undecidable	Fraction
1 (arm64)	106	90	84.9%
2	107	91	85.0%
3 (SELinux)	105	38	36.2%
4 (containerd 1.7)	55	40	72.7%
5 (gVisor)	29	11	37.9%
6 (Kata)	37	8	21.6%
six cells	439	278	63.3%
5 (gVisor, seccomp applied in sandbox)	91	30	33.0%
6 (Kata, seccomp applied in guest)	98	30	30.6%
GKE (no cell-elimination fields)	106	39	36.8%

Table 11.8: Undecidable fraction on production cells, full corpus under RuntimeDefault, corpus 0.3.3 (GKE: 0.3.2).

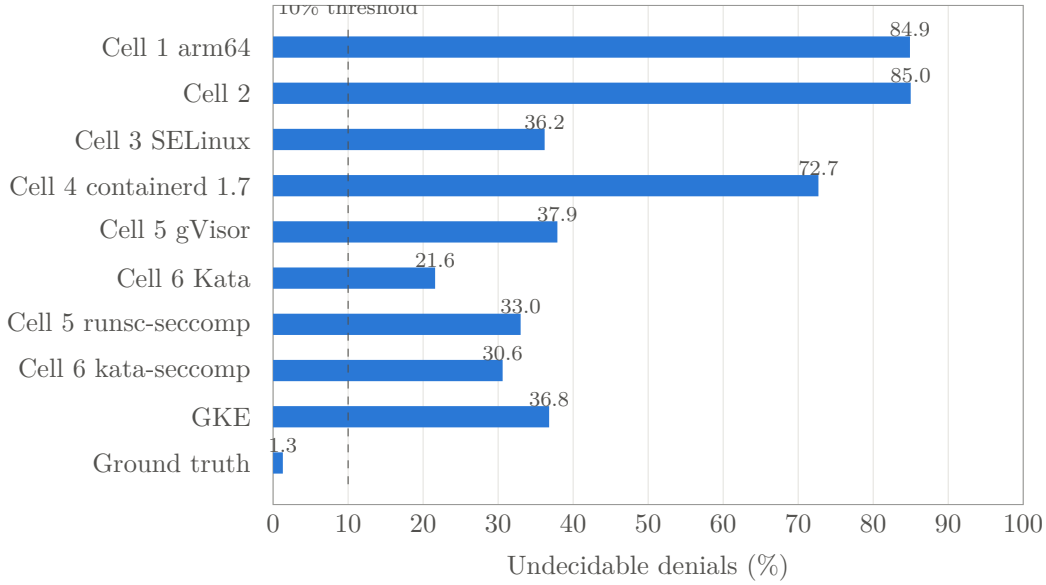


Figure 11.3: Undecidable fraction by cell, with the ground truth and the proposal's 10 per cent threshold for comparison (values in Table 11.8).

On cell 2 the 91 undecidable denials have three causes:

- **52 are io_uring `EPERM`s that could be seccomp or AppArmor.** The 7.0 kernel has the io_uring LSM hook and the AppArmor profile is enforcing, so neither can be

ruled out. Cell 3's 5.14 kernel predates the hook, which is why the same probes are decided there and cell 3's fraction is less than half of cell 2's.

- **32 are clone, unshare, socket, mount and capability calls** with every capability dropped and seccomp in force: 24 could be the filter or the missing capability, 7 the filter or the LSM (two of them the capability as well), and one is an `EACCES` that a file-permission check could also have produced.
- **7 are absent device nodes** (`ENOENT`), which none of the mechanisms the attributor names produced.

The undecidable fraction is therefore a property of the cell rather than of the attributor. On a production cell it is large, and reducing it needs the privileged helper that was not built (Chapter 13). The instrument's primary measurement, reachability, does not depend on attribution.

11.6 Remediation Closure

Criterion: 100 per cent closure of seccomp-attributed divergences, verified by re-probing; other mechanisms reported without a threshold.

Each cell was measured under RuntimeDefault, evaluated at `untrusted-multi-tenant`, given a generated Localhost profile, and measured again from a fresh boot under it (Table 11.9). A finding is seccomp-attributed when the generator writes a seccomp rule for it and the cell enforces seccomp (mode 2 at the baseline); it is closed when the re-probe shows it denied or killed.

Cell	Seccomp	Findings	Seccomp closed	Other closed
1 (arm64)	yes	5	5 of 5	
2	yes	5	5 of 5	
3 (SELinux)	yes	4	4 of 4	
4 (containerd 1.7)	yes	58	58 of 58	
5 (gVisor, `runsc`)	no	3		0 of 3
5 (gVisor, `runsc-seccomp`)	yes	1		0 of 1
6 (Kata, `kata`)	no	59		0 of 59
6 (Kata, `kata-seccomp`)	yes	3	3 of 3	
all		138	75 of 75	0 of 63

Table 11.9: Closure verified by re-probing (`fixtures/metrics/closure-metrics.json`).

The criterion is met: all 75 seccomp-attributed findings are closed, using six distinct rules. On cells 1 to 3 the findings are AF_ALG and the crypto, netfilter, rdma and xfrm netlink protocols (cell 3 has no rdma finding); cell 4 adds 53 io_uring probes, which one rule denying `io_uring_setup`, `io_uring_enter` and `io_uring_register` closes. Under `kata-seccomp` the findings are AF_ALG, netfilter and xfrm.

Under their default handlers gVisor and Kata run the workload with seccomp mode 0, so neither RuntimeDefault nor a generated profile reaches it, and none of their 62 findings closes:

- 57 are already denied by RuntimeDefault on the runc cells (io_uring on Kata, and the two user-namespace probes on both), so the generator has no allow rule to narrow and reports them as not closable;

- AF_ALG, netfilter and xfrm on Kata receive rules, and the re-probe shows them still permitted (`unenforced`);
- `device.open.fuse` on gVisor and `mount.reach` on Kata take a pathname or a filesystem type, which seccomp-BPF cannot inspect.

Under the variants that apply seccomp inside the sandbox, RuntimeDefault takes effect: on gVisor what remains is the FUSE device, which no seccomp rule can close, and on Kata the generated profile closes the remaining three. The generator's user-namespace and capability rules were not exercised, because no cell that applies seccomp permits those probes. The Security Profiles Operator accepted cell 2's generated resource on a test cluster; the other cells' resources were not installed. Closure was verified on self-managed nodes only, as the proposal anticipated.

11.7 Residual State and Runtime Cost

Criteria: zero unreclaimed objects after a full run, autoloaded modules reported apart; full corpus under 60 seconds, reduced corpus under 5 seconds.

Run	Full (ms)	Reduced (ms)	Classes	Unreclaimed	Leaks	Modules
Cell 1 (arm64)	241	56	6	0	0	28
Cell 2	585	158	6	0	0	31
Cell 3 (SELinux)	247	175	6	0	0	0
Cell 4 (containerd 1.7)	629	175	6	0	0	31
Cell 5 (gVisor)	8632	2851	3	0	0	0
Cell 6 (Kata)	109	92	6	0	0	n/a
Cell 2, controls off	899		6	0	0	36

Table 11.10: Residual state and runtime (`fixtures/metrics/residual-metrics.json`). Classes: residual classes the cell exposes. Modules: loaded by the full run.

All three criteria are met. The slowest full run is gVisor at 8.6 seconds, about seven times under budget, and the slowest reduced run gVisor at 2.9 seconds; on runc the full corpus takes a quarter to two-thirds of a second. gVisor's cost is consistent with the system call overheads reported by Young et al. [10]. Only two classes have probes that create objects: mounts (4 probes) and network namespaces (4). Under RuntimeDefault none of them is permitted on cells 1 to 4, so the zero there means nothing was created; gVisor permits two network-namespace probes and Kata one mount and two network-namespace probes. The run on cell 2 with every control off is the one in which all eight create their objects, and it ends with nothing left. No probe creates keyrings, cgroups or IPC objects; the engine's leak tests are the evidence that those classes would be caught. gVisor exposes neither keyrings nor either IPC class, so its zero covers three classes, and Kata exposes no module list. Reduced runs load no modules.

11.8 Comparison with Ancestor Tools

Reported as findings produced by each, without a threshold.

CALIPER, `amicontained` v0.4.9 and `seccomp-diff` were run against the same target on cell 2 (Table 11.11).

The counts are in different units and are not scores. Neither ancestor reports path masking (`path.masked.proc_kcore ENODATA`, `path.readonly.proc_sys EROFS`), per-filesystem mount reachability, per-family socket reachability (CALIPER: 10 permitted,

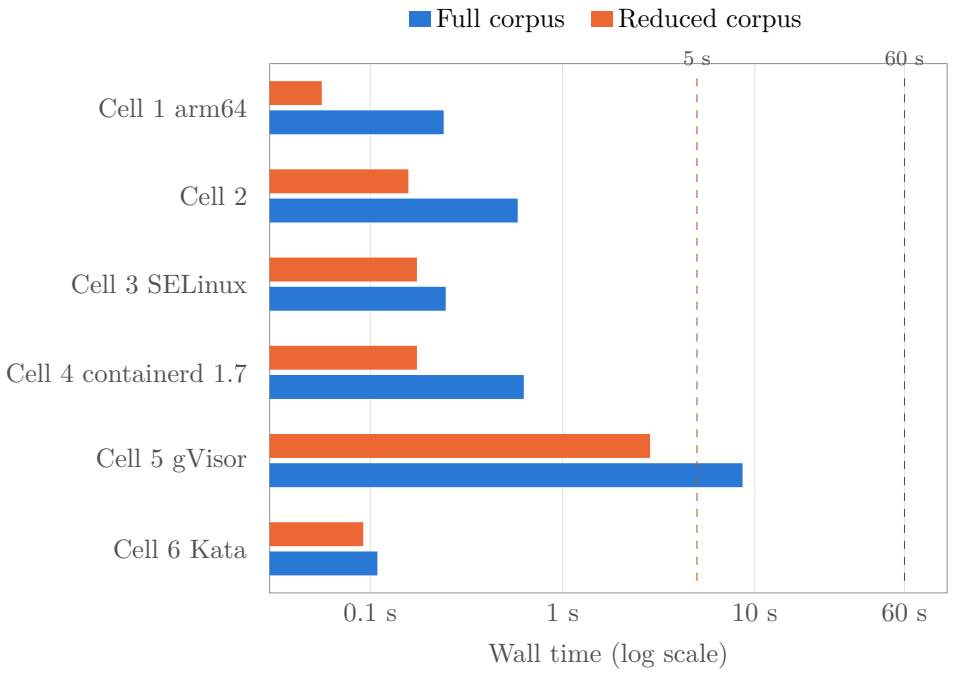


Figure 11.4: Runtime of the full and reduced corpus per cell, against the 60-second and 5-second budgets (dashed). Values in Table 11.10.

Tool	Findings on the shared target	What the output describes
CALIPER	107 denied, 53 permitted and 25 unimplemented probe results	Per-probe outcomes at argument level, including paths, mounts and kernel-dependent behaviour
`amicontained` v0.4.9	seccomp filtering; 65 blocked system call names; PID namespace present, user namespace absent; enforced AppArmor profile; empty capability list; runtime `not-found` (no runtime socket in the target)	A container summary and a coarse blocked-call list
`seccomp-diff`	native x86_64 branch, system call numbers 0 to 456: 297 `ALLOW`, 156 default `ERRNO(1)`, 3 `CONDITION` (`clone`, `personality`, `socket`), 1 explicit `ERRNO(38)` (`clone3`), 0 `KILL`	The active seccomp filter, decoded with host privilege

Table 11.11: Same-target comparison on cell 2 (`fixtures/ancestors/cell2/`).

5 denied, 9 unimplemented) or per-protocol netlink reachability (15 permitted, 3 unimplemented; both counts include the family's out-of-range oracle probe), per-flag clone and unshare outcomes, or what this kernel does not implement. `seccomp-diff` marks `socket` as conditional; decoding the condition shows it allows every family except 40 (`AF_VSOCK`), including AF_ALG (38), which CALIPER measures directly as permitted. In the other direction, `seccomp-diff` shows the filter that explains CALIPER's verdicts, which CALIPER cannot read without privilege, and `amicontained` summarises capability and namespace state in one screen. The outputs are complementary, not interchangeable. The comparison covers one target on one cell, and `seccomp-diff` captured the filter of a

fresh container with identical settings after a restart rather than of the same container.

11.9 Summary of Results

Metric	Threshold	Measured	Met
Policy divergence	≥ 4 pairs	12 of 15 pairs; 606 of 1,057 divergent probes explained by other classes	yes
Expectation violations	≥ 3 cells	3 cells at `restricted`, one of them on a single doubtful finding; 6 cells at `untrusted-multi-tenant`	yes
AF_ALG unaided	binary	reproduced with the standard Job; present under containerd 2.2.1 and 1.7.12, absent under 2.2.7	yes
Attribution accuracy	$\geq 90\%$	220 of 220	yes
Undecidable fraction	$\leq 10\%$	1.3% on the ground truth; 21.6% to 85.0% on production cells	yes
Seccomp closure	100%	75 of 75; other mechanisms 0 of 63	yes
Residual state	zero	0 unreclaimed, 0 leaks; up to 36 modules autoloaded by a full run, none by reduced runs	yes
Runtime	60 s / 5 s	8.6 s and 2.9 s (gVisor)	yes
Ancestor comparison	none	cell 2, Section 11.8	

Table 11.12: Every success criterion of the proposal (§11.2) with its measured value.

11.9.1 Answers to the research questions

RQ1 Yes. 185 argument-level probes run from an unprivileged container in 0.2 to 0.6 seconds on runc and 8.6 seconds on gVisor, survive 122 seccomp kills in one run, and leave nothing behind. The reduced corpus runs on every pull request.

RQ2 Less than half is policy. Of 1,057 divergent probe pairs, 451 are policy-explained; 556 are explained by RuntimeClass, 45 by kernel version and 5 by architecture. Swapping AppArmor for SELinux produced no policy difference that the kernel version did not explain; moving containerd from 2.2 to 1.7 produced 54.

RQ3 Accurately, but not completely. Every attribution made on the ground truth was correct. On production cells 21.6 to 85.0 per cent of denials remain undecidable without privilege, driven mainly by the io_uring LSM hook on new kernels.

RQ4 Yes. The AF_ALG exposure is invisible against the manifest and an expectation violation against the posture, as are io_uring on containerd 1.7 and on Kata.

RQ5 Where seccomp is enforced, yes; elsewhere, no. All 75 seccomp-attributable findings closed on verification, but gVisor and Kata do not apply seccomp by default, so none of their 62 default-handler findings closed.

Chapter 12
Responsible Use and Disclosure

12.1 Responsible Use

The instrument probes security boundaries, and the proposal made three commitments in consequence: the corpus contains no exploit code; the instrument is intended only for infrastructure the operator owns or has written authorisation to test; and a genuine defect found in a runtime, distribution or managed platform is reported to its vendor under coordinated disclosure before publication. The first is established by the safety review (Section 7.8). The second is stated in the documentation and printed in every report the instrument writes. Two further measures make a deliberate run distinguishable from an attack: the release image is signed with cosign, so admission controllers and runtime monitors can allowlist its digest, and the reduced corpus omits the probes most likely to raise alerts, including the AF_ALG probe on which published Copy Fail detection rules key. On a monitored cluster the motivating case is demonstrated by allowlisting the signed image and running the full corpus.

12.2 Disclosure Review: Method

The review covered the two sandboxed runtimes, where the measurements differ most from Linux and where a difference is most likely to be a defect rather than a configuration choice. It was carried out in four steps:

1. Every probe permitted under gVisor (cell 5) or Kata (cell 6) was compared with the same probe on Linux with the same credentials and no seccomp, which is exactly the ground-truth `capability` run on cell 2 (UID 65534, no capabilities). A probe permitted in a sandbox and denied there is a candidate.
2. Because that comparison has no masked paths, every path probe was compared separately with cell 2 and with the ground-truth `paths` run.
3. Each candidate was traced to the runtime's source at the exact version measured (gVisor release-20261005.0; Kata Containers 3.14.0, agent and runtime), and checked against the current development branch and the latest release.
4. For gVisor, the three claim violations were reproduced with the same release in a disposable virtual machine, as UID 65534 with no capabilities, against a native Linux control.

Upstream issue trackers were then searched for each confirmed defect. Table 12.1 gives the outcome.

12.3 Findings

12.3.1 gVisor

Of the three claim violations on cell 5, one is a genuine defect, one a deliberate design choice and one a false positive of this instrument. The `CLONE_NEWNS` defect was public before this project first recorded it on 2026-10-18: a fix had been proposed in June 2026 and the issue was filed independently on 2026-10-21. It is described here as independently observed, not discovered. The false positive (`unshare.flags.newtime`) is a limitation of the method: a `permitted` verdict records that the call succeeded, and on a reimplemented

Observation	What happens	Status upstream	Assessment
gVisor: `clone.flags.newns` permitted without CAP_SYS_ADMIN	`CLONE_NEWNS` is missing from the capability check in `Clone` (`Unshare` has it); a new mount namespace is created (namespace id 7 became 8 in the live test), though the child still cannot mount or unmount	public: google/gvisor issue 16118 [70] (2026-10-21) and fix PR 13511 [71] (open since 2026-06-19); unfixed in release-20261102.0	genuine defect; per the issue, classed by gVisor's security team as an in-sandbox emulation bug
gVisor: `capability.sys_nice` permitted	CAP_SYS_NICE deliberately not checked when a task lowers its own nice value; niceness reaches no scheduler (only procfs, `getpriority` and `sched_getattr` read it)	documented in the source	not a defect
gVisor: `unshare.flags.newtime` permitted	`unshare` ignores unknown flags and gVisor has no time namespace; nothing is created	`CLONE_NEWTIME` documented as unsupported	false positive in CALIPER
gVisor: masked and read-only paths not applied	`runsc` never reads `maskedPaths` or `readonlyPaths`; the files are synthesised by the Sentry (`/sys/firmware` is an empty directory)	known: google/gvisor issue 9245 [72], closed as not planned	not a defect
gVisor, Kata: RuntimeDefault not applied by default	seccomp is off inside the sandbox unless `oci-seccomp` (gVisor) or `disable_guest_seccomp=false` (Kata) is set	gVisor logs a warning; Kata documents the default in its configuration	documented default
Kata: masked directories not masked	the agent bind-mounts `/dev/null` over each masked path and treats `ENOTDIR` as success, so `/proc/acpi` and `/sys/firmware` stay visible	public: kata-containers issue 14188 [73] (2026-11-01), fixed by PR 14189 [74] on 2026-11-02, not yet in a release	genuine defect
Kata: AppArmor profile not applied	nothing in the 3.14.0 agent or runtime reads the profile, and the guest runs no LSM (the cell records none)	no documentation found	limitation of the design
Kata: other differences	SELinux netlink, `xfs`, `9p` and `virtiofs` differ because the guest kernel is configured differently	expected	not a defect
containerd: AF_ALG allowed by RuntimeDefault	the motivating case	public and fixed (containerd PR 13327 [22])	known before the project

Table 12.1: Disclosure review of runtime behaviour observed in the evaluation.

kernel success does not prove that the operation took place. The masked-path behaviour is a deliberate gVisor decision, recorded upstream and harmless because the files are the sandbox's own synthesis.

12.3.2 Kata Containers

The Kata agent's masking routine bind-mounts `/dev/null` over each path in the specification's `maskedPaths`. A bind of a character device over a directory fails with `ENOTDIR`, which the routine treats as success, so masked *directories* are never masked. With the kubelet's default list the effect is that the guest virtual machine's ACPI tables and

firmware information are readable from the container; a directory an operator adds to the list is left unmasked as well. The defect was reported publicly upstream on 2026-11-01 and fixed the next day by masking directories with a shared read-only tmpfs; the fix had not reached a release when this report was written. The maintainers handled it as an ordinary public bug, without an advisory.

12.4 Where the Commitment Was Not Met

Two genuine runtime defects were observed. Neither was reported by this project, and both were public upstream before this report. The gVisor case is unproblematic: the defect was public, with a fix proposed, before our first recording of it. The Kata case is the one where the commitment was not met as written. Our cell 6 fingerprint, public in the repository since 2026-10-18, showed `path.masked.proc_acpi` and `path.masked.sys_firmware` reachable two weeks before the defect was reported upstream. We did not recognise it as a defect at the time, because the evaluator records both as undeclared exposures: the posture asserts masks only for `/proc/kcore` and `/proc/keys`, and the evaluator does not treat the kubelet's default masked paths as a claim. That gap in the evaluator is listed among the limitations (Section 13.2) and is the first change it suggests.

No finding has an open disclosure window, and nothing in this report is withheld.

Chapter 13
Discussion

13.1 What the Compression Gave Up

The proposal compressed a two-semester plan into seventeen weeks by reducing scope, and named four reductions in advance so that what was given up would be visible rather than discovered later. Each is stated here as it turned out.

The deferred corpus set. Twenty-eight probes in nine families are specified in the committed format and were not implemented: ioctl request codes, bpf, keyctl, prctl, perf_event_open, userfaultfd, open_by_handle_at, module loading and seccomp user notification. Every fingerprint lists them as unmeasured. Five posture assertions concern these families and can produce no finding until they are implemented.

The privileged helper. Outlined and neither specified nor built. Attribution rests on errno oracles, correlation and cell elimination, none of which needs privilege. The undecidable fraction is reported rather than reduced, and on production cells it is 21.6 to 85.0 per cent.

The environment matrix. Six cells, all measured, with a GKE cluster as the CI target and an AF_ALG comparison point. The optional additions were not attempted: a Podman and crun cell, a Moby cell, EKS and Talos. The runtime axis of objective 8 is therefore moved by version and by RuntimeClass but not across runtime implementations.

SARIF output. Deferred. Reports are JSON and text.

Nothing in the measurement, classification or evaluation path was reduced. Three further departures from the proposal were not planned:

- The cells run on Compute Engine rather than on developer hardware. Compute Engine supplied native amd64 machines and nested virtualisation for the Kata cell, which the team's arm64 laptops could provide only under slow emulation or not at all, and it made every cell reproducible from a script, at the cost of the cloud dependency the proposal had wanted to keep off the critical path.
- The ancestor comparison covers one target on one cell rather than the six-cell study the schedule allowed for.
- The Security Profiles Operator's acceptance of a generated resource was checked for cell 2 only.

13.2 Limitations

- **The posture is a judgement.** It is ours, cited and versioned, and findings are reproducible against a stated revision, but an expectation violation is only as strong as the assertion behind it. The FUSE finding on cell 5 shows an assertion whose rationale does not carry over to a sandbox, and the three-cell criterion at `restricted` depends on that finding.
- **A verdict is the kernel's answer, not proof of effect.** On a reimplemented kernel a successful call may do nothing (`unshare.flags.newtime` under gVisor), and the oracles are written against Linux's code paths.
- **Attribution in production is mostly undecidable** on the runc cells with AppAr-

mor and a recent kernel. The ground-truth figure measures the attributor, not what an operator will see.

- **The classifier has no ground truth.** Its separation of classes was checked by unit tests and by the per-pair counts matching the axis each pair moves, not against environments with known causes.
- **Single measurements.** Each cell was measured once per configuration from a fresh boot. The verdicts are deterministic for a given cell, but the runtime figures are single runs.
- **The residual zero is narrow.** Only mounts and network namespaces are created by any probe; the other four classes are covered by the engine's tests, not by the evaluation.
- **Module autoload changes the host.** A full run loads up to 36 modules that remain loaded for every tenant. The reduced corpus avoids this, at the cost of the AF_ALG probe.
- **Masked paths are not modelled as a claim.** The kubelet's default masks are part of what every pod is promised, but the evaluator does not treat a reachable masked path as a claim violation, which is how a real Kata defect was recorded only as an undeclared exposure.
- **The corpus is argument-granular only where arguments are registers.** A pathname, a filesystem type or a structure behind a pointer cannot be filtered by seccomp, so the corresponding findings can be measured but not closed by the remediator.

13.3 Threats to Validity

Internal validity. Every verdict depends on the oracle being right about the kernel's code path. Oracles were reviewed against the kernel source and swept for consistency before the freeze, and two oracle defects were nevertheless found afterwards and corrected (the io_uring register descriptor on 6.13 and later, and the io_uring setup hook on 6.15 and later). Other version-dependent orderings may exist on kernels not measured. The cell recordings are at corpus 0.3.0 and the release is 0.3.3; the two give identical verdicts and errnos on all six cells, so the results hold for the released corpus.

Construct validity. Reachability of an entry point is a proxy for attack surface, not a measure of exploitability. A permitted AF_ALG socket is exploitable only where `algif_aead` can be loaded and the kernel is unpatched; the instrument reports the first and the module state, not the second. The undecidable fraction is defined over all denials, killed results included, as the proposal specified.

External validity. The cells cover two architectures, two LSMs, two containerd versions and three runtime classes on Ubuntu, Rocky Linux and Container-Optimized OS. They do not cover other runtimes (CRI-O, Moby, crun), other managed platforms, or kernels older than 5.14. The attribution ground truth is on a single cell and contains no SELinux denials.

Reliability. Provisioning pins package versions through an archive snapshot, holds and hashes, and every metric is regenerated from committed fixtures by a test, so the reported numbers can be reproduced exactly from the repository; re-measuring a cell requires a Compute Engine project.

13.4 Lessons Learned

- **"The same confinement as a pod" must be checked against what the kubelet actually does.** The first version of the cell measurement script omitted the kubelet's masked and read-only paths, leaving `/proc` unmasked in every recording. A review of the evaluation pull request found it before the results were used.
- **Low-level details decide survivability.** A raw `clone` under musl let an aborting child kill the parent; a core-dump pipe in the host's crash handler stalled every SIGSYS kill until the engine made itself non-dumpable; `libc::syscall`'s variadic seventh argument carried garbage in its upper bits on x86_64 until full-width casts were used. Each was found by a test written for the purpose.
- **Freeze dates are what make parallel work possible.** Freezing the probe specification at the W2 gate and the fingerprint format at the W9 gate let four people build against recorded fingerprints; every later change had to be additive.
- **Reporting what is undecidable is more useful than a confident guess.** The production undecidable fractions are uncomfortable numbers, but they point exactly at the kernel change (the io_uring LSM hook) and the missing component (the privileged helper) that explain them.

Chapter 14
Conclusion and Future Work

14.1 Conclusion

CALIPER measures what a container can reach in the kernel, from inside the container, at argument granularity, and explains the differences it finds. It runs 185 probes, each in its own forked child, in under a second on runc and under nine seconds on gVisor, survives a hardened profile that kills 122 of its children in one run, and leaves nothing behind. Its fingerprints are keyed to the environment that produced them, and its classifier keeps architecture, kernel-version and sandbox effects out of the policy count: of 1,057 divergent probe pairs across six cells, 606 are explained by something other than policy, and the 451 that remain point at real differences in enforcement, such as io_uring under containerd 1.7 and the absence of seccomp inside gVisor and Kata by default.

The instrument reproduced the motivating case without help. With the standard Job's pod specification, which satisfies PSS Restricted, AF_ALG was reachable under containerd 2.2.1 and 1.7.12 and refused under 2.2.7; the manifest was the same, and only the measurement showed the difference. Against the manifest alone the case produces no finding; against the cited reference posture it is an expectation violation carrying its CVE. Every denial the attributor decided on a constructed ground truth was decided correctly, and every seccomp-attributable finding it generated a profile for was closed and verified by measuring again. Along the way it observed two genuine runtime defects that no configuration review would have found, one in gVisor and one in Kata Containers.

The evaluation also shows where the instrument stops. In production most denials on a hardened runc cell cannot be attributed from inside the container, because modern kernels place an LSM hook in front of the very checks the oracles rely on. A successful call on a sandbox's reimplemented kernel does not prove the operation took place. The three-cell expectation criterion at `restricted` depends on one finding of doubtful weight. And the project's own disclosure commitment was not met in the Kata case, because the evaluator did not treat the kubelet's masked paths as a claim. Each of these is stated in the report as measured, because an instrument for verifying confinement is only useful if its own claims can be verified.

14.2 Future Work

The first extensions follow from the limits above.

1. **Masked paths as claims.** Treat the kubelet's default masked and read-only paths as part of every pod's declaration, so that a reachable masked path is a claim violation. This is a change to the evaluator alone, and it would have flagged the Kata defect on the day it was recorded.
2. **The privileged helper.** An optional DaemonSet with CAP_SYS_ADMIN that reads the workload's seccomp filter with `PTRACE_SECCOMP_GET_FILTER` would decide most of the production undecidable fraction, in particular the 52 io_uring denials on cell 2.
3. **The deferred corpus.** Implement the 28 specified probes, starting with the families the posture already asserts against (bpf, perf_event_open, keyctl, userfaultfd,

open_by_handle_at), so that those assertions can produce findings.

4. **Sandbox-aware oracles and posture.** Mark probes whose `permitted` verdict is not evidence of effect under a reimplemented kernel, and qualify posture assertions (such as the FUSE device) whose rationale does not carry over to a sandbox.
5. **More runtimes and platforms.** Podman with crun, Moby, CRI-O, EKS and Talos, to move the runtime axis across implementations rather than versions.
6. **Comparison across clusters on every pull request.** Measure each change on several clusters and report the classified divergence between them in the pull request, so that a node-image upgrade that weakens RuntimeDefault is caught before it ships.
7. **Posture v2 and SARIF.** Use the 282 undeclared exposures as input to the next posture revision (mount types and raw netlink protocols first), and emit SARIF so that findings appear in code-scanning interfaces.

References

[1] X. Lin, L. Lei, Y. Wang, J. Jing, K. Sun, and Q. Zhou, "A measurement study on Linux container security: Attacks and countermeasures," in *Proc. 34th Annual Computer Security Applications Conference (ACSAC)*, 2018, pp. 418–429, doi: 10.1145/3274694.3274720.

[2] X. Gao, Z. Gu, Z. Li, H. Jamjoom, and C. Wang, "Houdini's escape: Breaking the resource rein of Linux control groups," in *Proc. ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2019, pp. 1073–1086, doi: 10.1145/3319535.3354227.

[3] Linux man-pages project, `capabilities(7)`, `namespaces(7)`, `user_namespaces(7)`, `socket(2)`, `netlink(7)`, `mount(2)`, `io_uring_register(2)`. [Online]. Available: `https://man7.org/linux/man-pages/`

[4] Linux man-pages project, `seccomp(2)`, `seccomp_unotify(2)`, `ptrace(2)`. [Online]. Available: `https://man7.org/linux/man-pages/`

[5] Y. Sun, D. Safford, M. Zohar, D. Pendarakis, Z. Gu, and T. Jaeger, "Security namespace: Making Linux security frameworks available to containers," in *Proc. 27th USENIX Security Symposium*, 2018.

[6] Open Container Initiative, "Runtime Specification, version 1.1: Linux container configuration (`maskedPaths`, `readonlyPaths`, devices)." [Online]. Available: `https://github.com/opencontainers/runtime-spec`

[7] X. Gao, Z. Gu, M. Kayaalp, D. Pendarakis, and H. Wang, "ContainerLeaks: Emerging security threats of information leakages in container clouds," in *Proc. 47th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, 2017, pp. 237–248, doi: 10.1109/DSN.2017.49.

[8] Kubernetes, "Pod Security Standards." [Online]. Available: `https://kubernetes.io/docs/concepts/security/pod-security-standards/`

[9] containerd project, "containerd: an industry-standard container runtime," versions 1.7.12, 2.2.1 and 2.2.7. [Online]. Available: `https://containerd.io`

[10] E. G. Young, P. Zhu, T. Caraza-Harter, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau, "The true cost of containing: A gVisor case study," in *Proc. 11th USENIX Workshop on Hot Topics in Cloud Computing (HotCloud)*, 2019.

[11] The gVisor Authors, "gVisor: An application kernel for containers," release-20261005.0. [Online]. Available: `https://gvisor.dev`

[12] Kata Containers project, "Kata Containers," version 3.14.0. [Online]. Available: `https://katacontainers.io`

[13] Anjali, T. Caraza-Harter, and M. M. Swift, "Blending containers and virtual machines: A study of Firecracker and gVisor," in *Proc. 16th ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments (VEE)*, 2020, pp. 101–113, doi: 10.1145/3381052.3381315.

[14] MITRE, "CVE-2026-31431: Linux kernel `algif_aead` improper handling of in-place operations (Copy Fail)," 2026. [Online]. Available: `https://nvd.nist.gov/vuln/detail/CVE-2026-31431`

[15] Cybersecurity and Infrastructure Security Agency, "Known Exploited Vulnerabilities Catalog: CVE-2026-31431," added 1 May 2026. [Online]. Available: `https://www.cisa.gov/known-exploited-vulnerabilities-catalog`

[16] Microsoft Security Blog, "CVE-2026-31431: Copy Fail vulnerability enables Linux root privilege escalation across cloud environments," 1 May 2026.

[17] Sysdig, "CVE-2026-31431: Copy Fail Linux kernel flaw lets local users gain root in seconds," 2026.

[18] Juliet Security, "Copy Fail in Kubernetes: RuntimeDefault did not block AF_ALG," 2026.

[19] moby/moby pull request 46762, "seccomp: block io_uring_* syscalls in default profile," merged November 2023; released in Moby 25.0, January 2024.

[20] containerd/containerd pull request 9320, "Update RuntimeDefault seccomp profile to disallow io_uring related syscalls," merged November 2023; released in containerd 2.0, November 2024.

[21] moby/moby pull request 52501, "seccomp: block AF_ALG sockets in default profile (CVE-2026-31431)," merged May 2026; released in Moby 29.4.2.

[22] containerd/containerd pull request 13327: deny AF_ALG in the RuntimeDefault seccomp profile, May 2026; released in containerd 2.2.4 and backported to 1.7.32.

[23] H. Patel, D. Barrera, and A. Somayaji, "Houdini: Benchmarking container security confinement," in *Proc. 22nd Annual International Conference on Privacy, Security and Trust (PST)*, 2025, pp. 1–10, doi: 10.1109/PST65910.2025.11268879.

[24] X. Gao, B. Steenkamer, Z. Gu, M. Kayaalp, D. Pendarakis, and H. Wang, "A study on the security implications of information leakages in container clouds," *IEEE Transactions on Dependable and Secure Computing*, vol. 18, no. 1, pp. 174–191, 2021, doi: 10.1109/TDSC.2018.2879605.

[25] S. Ghavamnia, T. Palit, A. Benameur, and M. Polychronakis, "Confine: Automated system call policy generation for container attack surface reduction," in *Proc. 23rd International Symposium on Research in Attacks, Intrusions and Defenses (RAID)*, USENIX Association, 2020.

[26] M. Rostamipoor, S. Ghavamnia, and M. Polychronakis, "Confine: Fine-grained system call filtering for container attack surface reduction," *Computers & Security*, vol. 132, Art. no. 103325, 2023, doi: 10.1016/j.cose.2023.103325.

[27] N. DeMarinis, K. Williams-King, D. Jin, R. Fonseca, and V. P. Kemerlis, "sysfilter: Automated system call filtering for commodity software," in *Proc. 23rd International Symposium on Research in Attacks, Intrusions and Defenses (RAID)*, USENIX Association, 2020, pp. 459–474.

[28] S. Ghavamnia, T. Palit, S. Mishra, and M. Polychronakis, "Temporal system call specialization for attack surface reduction," in *Proc. 29th USENIX Security Symposium*, 2020, pp. 1749–1766.

[29] S. Ghavamnia, T. Palit, and M. Polychronakis, "C2C: Fine-grained configuration-driven system call filtering," in *Proc. ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2022, pp. 1243–1257, doi: 10.1145/3548606.3559366.

[30] C. Canella, M. Werner, D. Gruss, and M. Schwarz, "Automating seccomp filter generation for Linux applications," in *Proc. Cloud Computing Security Workshop (CCSW)*, 2021, pp. 139–151, doi: 10.1145/3474123.3486762.

[31] Z. Wan, D. Lo, X. Xia, L. Cai, and S. Li, "Mining sandboxes for Linux containers," in *Proc. IEEE International Conference on Software Testing, Verification and Validation (ICST)*, 2017, pp. 92–102, doi: 10.1109/ICST.2017.16.

[32] Z. Wan, D. Lo, X. Xia, and L. Cai, "Practical and effective sandboxing for Linux containers," *Empirical Software Engineering*, vol. 24, no. 6, pp. 4034–4070, 2019, doi: 10.1007/s10664-019-09737-2.

[33] L. Lei, J. Sun, K. Sun, C. Shenefiel, R. Ma, Y. Wang, and Q. Li, "SPEAKER: Split-phase execution of application containers," in *Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA)*, LNCS, Springer, 2017, pp. 230–251, doi: 10.1007/978-3-319-60876-1_11.

[34] H. Zhu and C. Gehrmann, "Lic-Sec: An enhanced AppArmor Docker security profile generator," *Journal of Information Security and Applications*, vol. 61, Art. no. 102924, 2021, doi: 10.1016/j.jisa.2021.102924.

[35] A. Rahman, S. I. Shamim, D. B. Bose, and R. Pandita, "Security misconfigurations in open source Kubernetes manifests: An empirical study," *ACM Transactions on Software Engineering and Methodology*, vol. 32, no. 4, pp. 1–36, 2023, doi: 10.1145/3579639.

[36] F. Minna, A. Blaise, F. Rebecchi, B. Chandrasekaran, and F. Massacci, "Understanding the security implications of Kubernetes networking," *IEEE Security & Privacy*, vol. 19, no. 5, pp. 46–56, 2021, doi: 10.1109/MSEC.2021.3094726.

[37] A. Agache, M. Brooker, A. Florescu, A. Iordache, A. Liguori, R. Neugebauer, P. Piwonka, and D.-M. Popa, "Firecracker: Lightweight virtualization for serverless applications," in *Proc. 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2020, pp. 419–434.

[38] A. Kurmus, S. Dechand, and R. Kapitza, "Quantifiable run-time kernel attack surface reduction," in *Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA)*, LNCS, Springer, 2014, pp. 212–234, doi: 10.1007/978-3-319-08509-8_12.

[39] S. Sultan, I. Ahmad, and T. Dimitriou, "Container security: Issues, challenges, and the road ahead," *IEEE Access*, vol. 7, pp. 52976–52996, 2019, doi: 10.1109/ACCESS.2019.2911732.

[40] C. Pahl, A. Brogi, J. Soldani, and P. Jamshidi, "Cloud container technologies: A state-of-the-art review," *IEEE Transactions on Cloud Computing*, vol. 7, no. 3, pp. 677–692, 2019, doi: 10.1109/TCC.2017.2702586.

[41] E. Casalicchio and S. Iannucci, "The state-of-the-art in container technologies: Application, orchestration and security," *Concurrency and Computation: Practice and Experience*, vol. 32, no. 17, Art. no. e5668, 2020, doi: 10.1002/cpe.5668.

[42] J. Watada, A. Roy, R. Kadikar, H. Pham, and B. Xu, "Emerging trends, techniques and open issues of containerization: A review," *IEEE Access*, vol. 7, pp. 152443–152472, 2019, doi: 10.1109/ACCESS.2019.2945930.

[43] O. Jarkas, R. Ko, N. Dong, and R. Mahmud, "A container security survey: Exploits, attacks, and defenses," *ACM Computing Surveys*, vol. 57, no. 7, pp. 1–36, 2025, doi: 10.1145/3715001.

[44] A. Bhardwaj and C. R. Krishna, "Virtualization in cloud computing: Moving from hypervisor to containerization, a survey," *Arabian Journal for Science and Engineering*, vol. 46, no. 9, pp. 8585–8601, 2021, doi: 10.1007/s13369-021-05553-3.

[45] V. S. Devi Priya, S. C. Sethuraman, and M. K. Khan, "Container security: Precaution levels, mitigation strategies, and research perspectives," *Computers & Security*, vol. 135, Art. no. 103490, 2023, doi: 10.1016/j.cose.2023.103490.

[46] T. Combe, A. Martin, and R. Di Pietro, "To Docker or not to Docker: A security perspective," *IEEE Cloud Computing*, vol. 3, no. 5, pp. 54–62, 2016, doi: 10.1109/MCC.2016.100.

[47] A. Martin, S. Raponi, T. Combe, and R. Di Pietro, "Docker ecosystem – Vulnerability analysis," *Computer Communications*, vol. 122, pp. 30–43, 2018, doi: 10.1016/j.comcom.2018.03.011.

[48] M. Souppaya, J. Morello, and K. Scarfone, "Application container security guide," NIST Special Publication 800-190, National Institute of Standards and Technology, 2017, doi: 10.6028/NIST.SP.800-190.

[49] Aqua Security, "kube-bench: CIS Kubernetes Benchmark checks." [Online]. Available: `https://github.com/aquasecurity/kube-bench`

[50] Kubescape project, "Kubescape." [Online]. Available: `https://kubescape.io`

[51] Aqua Security, "Trivy." [Online]. Available: `https://trivy.dev`

[52] The Falco Project, "Falco: cloud-native runtime security." [Online]. Available: `https://falco.org`

[53] Cilium project, "Tetragon: eBPF-based security observability and runtime enforcement." [Online]. Available: `https://tetragon.io`

[54] containers/oci-seccomp-bpf-hook, "OCI hook to trace syscalls and generate a seccomp profile." [Online]. Available: `https://github.com/containers/oci-seccomp-bpf-hook`

[55] Kubernetes SIGs, "Security Profiles Operator," v1.1.1. [Online]. Available: `https://github.com/kubernetes-sigs/security-profiles-operator`

[56] Inspektor Gadget, "advise seccomp-profile." [Online]. Available: `https://inspektor-gadget.io`

[57] genuinetools/amicontained, "Container introspection tool," release v0.4.9, November 2019. [Online]. Available: `https://github.com/genuinetools/amicontained`

[58] antitree/seccomp-diff, "Extract and compare seccomp-BPF filters from running containers." [Online]. Available: `https://github.com/antitree/seccomp-diff`

[59] Linux kernel documentation, "io_uring." [Online]. Available: `https://docs.kernel.org/io_uring/index.html`

[60] MITRE, "CVE-2017-2636: race condition in the Linux kernel n_hdlc line discipline," 2017. [Online]. Available: `https://nvd.nist.gov/vuln/detail/CVE-2017-2636`

[61] MITRE, "CVE-2024-1086: use-after-free in the Linux kernel netfilter nf_tables component," 2024. [Online]. Available: `https://nvd.nist.gov/vuln/detail/CVE-2024-1086`

[62] MITRE, "CVE-2022-0185: heap overflow in the Linux kernel filesystem context, reachable through user namespaces," 2022. [Online]. Available: `https://nvd.nist.gov/vuln/detail/CVE-2022-0185`

[63] MITRE, "CVE-2013-2094: Linux kernel perf_swevent_init privilege escalation," 2013. [Online]. Available: `https://nvd.nist.gov/vuln/detail/CVE-2013-2094`

[64] MITRE, "CVE-2017-16995: Linux kernel BPF verifier sign-extension flaw," 2017. [Online]. Available: `https://nvd.nist.gov/vuln/detail/CVE-2017-16995`

[65] MITRE, "CVE-2016-0728: Linux kernel keyring reference-count overflow," 2016. [Online]. Available: `https://nvd.nist.gov/vuln/detail/CVE-2016-0728`

[66] Linux kernel documentation, "Userfaultfd: unprivileged userfaultfd." [Online]. Available: `https://docs.kernel.org/admin-guide/mm/userfaultfd.html`

[67] S. Krahmer, "Shocker: Docker container breakout using `open_by_handle_at`," 2014. [Online]. Available: `https://github.com/earthquake/shocker`

[68] Sigstore project, "cosign: container signing, verification and storage in an OCI registry," v3.1.3. [Online]. Available: `https://github.com/sigstore/cosign`

[69] A. A. Madake, Y. B. Palve, R. A. Saindane, and S. T. Waje, "CALIPER: Empirical measurement of container confinement," release v1.0.0, 2026. [Online]. Available: `https://github.com/Abhinavmadake/caliper`

[70] google/gvisor issue 16118: `CLONE_NEWNS` permitted without CAP_SYS_ADMIN, opened 21 October 2026.

[71] google/gvisor pull request 13511: add `CLONE_NEWNS` to the namespace capability check in `Clone`, opened 19 June 2026.

[72] google/gvisor issue 9245: `maskedPaths` and `readonlyPaths` from the OCI specification, closed as not planned.

[73] kata-containers/kata-containers issue 14188: directories in `maskedPaths` are not masked, opened 1 November 2026.

[74] kata-containers/kata-containers pull request 14189: mask directories in `maskedPaths` with a shared read-only tmpfs, merged 2 November 2026.

Appendix A

Reproducing the Results

Every metric in Chapter 11 is regenerated from committed fingerprints by a command in `control/cmd`, and a test compares each command's output with the committed metric file, so a change that alters a reported number fails CI. Run from `control/`:

```
# divergence per cell pair and class (Table 11.3)
go run ./cmd/caliper-divergence-metrics \
    -out ../fixtures/metrics/divergence-metrics.json
# expectation, claim and undeclared findings at the restricted tier (Table 11.4)
go run ./cmd/caliper-ci-report -cell-fixtures ../fixtures/cells \
    -metrics-out ../fixtures/metrics/finding-metrics.json
# attribution against single-cause environments (Table 11.7)
go run ./cmd/caliper-attribution-accuracy \
    -ground-truth ../fixtures/ground-truth/attribution.json \
    -out ../fixtures/metrics/attribution-accuracy.json
# closure verified by re-probing (Table 11.9)
go run ./cmd/caliper-closure-metrics \
    -out ../fixtures/metrics/closure-metrics.json
# residual state and runtime (Table 11.10)
go run ./cmd/caliper-residual-metrics \
    -out ../fixtures/metrics/residual-metrics.json
```

The measurements themselves come from the cells, with `deploy/gce/cells.sh`:

```
# a cell: create (VM from a pinned image, then provisioning), measure, stop
deploy/gce/cells.sh create  cellN
deploy/gce/cells.sh measure cellN <image> > cellN.raw   # reboots first
deploy/gce/cells.sh stop    cellN                       # a running VM is billed

# ground-truth modes and the hand-hardened profile on cell 2
deploy/gce/cells.sh measure cell2 <image> "" <mode>
deploy/gce/cells.sh measure cell2 <image> \
    engine/fixtures/seccomp/hardened.json

# reduced corpus; closure pipeline; sandbox handlers with seccomp
CALIPER_CORPUS_SET=reduced deploy/gce/cells.sh measure cellN <image>
CALIPER_POSTURE_TIER=untrusted-multi-tenant \
    deploy/gce/run-pipeline.sh cellN
CALIPER_HANDLER=runsc-seccomp deploy/gce/run-pipeline.sh cell5
CALIPER_HANDLER=kata-seccomp  deploy/gce/run-pipeline.sh cell6

# complete a raw measurement into a fingerprint (from control/)
go run ./cmd/caliper-complete -distribution "Ubuntu 24.04" \
    -runtime containerd -runtime-version 2.2.1 -runtime-class runc \
    -node-kernel 7.0.0-1014-gcp < ../cell2.raw > ../fixtures/cells/cell2.json
```

The released image is verified with cosign against the release workflow's identity (`RELEASE.md`):

```
ghcr.io/abhinavmadake/caliper@sha256:
    61b918d61aae4cec0b9df70526555c65ff42f9a33f6381ebb99eb1f83a174db7
```

The production undecidable fractions of Table 11.8 are not a committed metric. They were computed for this report by calling `control.Attribute` and `control.BuildReport` on the baseline fingerprints in `fixtures/pipeline/` and on `fixtures/gke-cos-containerd-2.2.7.json`; for a single fingerprint this is the value the text report prints on its “Undecidable” line.

Appendix B
Project Record

B.1 Timeline

The repository's first commit is dated 2026-09-04 and week 1 began on Monday 2026-09-07. Figure B.1 shows the main periods of work, taken from the merge dates in the repository's history, and Figure B.2 the commits on `main` per calendar week (each merged pull request is one squashed commit).

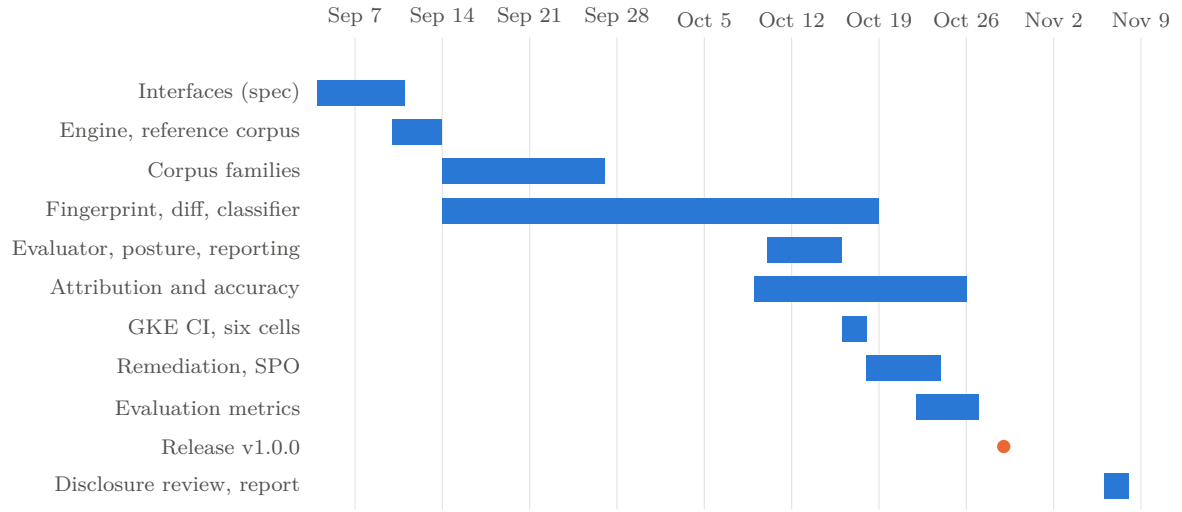


Figure B.1: Main periods of work (from merge dates). The week-17 gate, with the demonstration, is 2026-11-29.

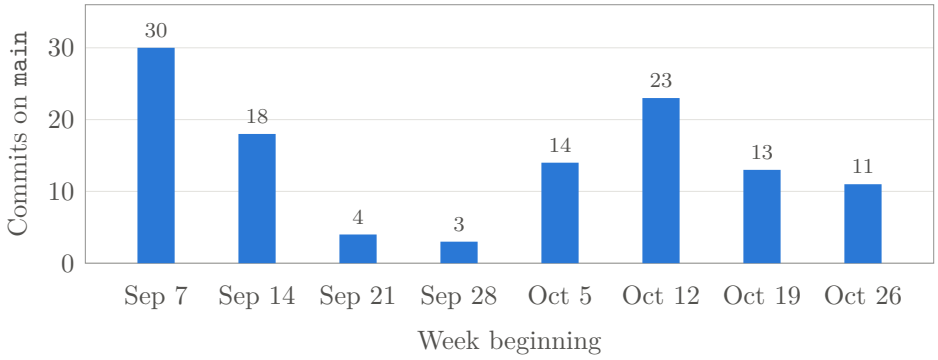


Figure B.2: Commits on `main` per week, 116 in all (the first week includes the three commits of 2026-09-04).

B.2 Milestones

The proposal's schedule was tracked as one GitHub milestone per freeze gate (Table B.1). Week 16 was kept empty as declared slack; Table 5.4 in Chapter 5 compares each gate's due date with the date it was met.

Milestone	Due	Issues
W2: interfaces frozen	2026-09-11	2
W3: engine and reference corpus	2026-09-18	10
W8: corpus frozen	2026-09-27	14
W9: fingerprint frozen	2026-10-11	14
W12: attribution, evaluator, remediation	2026-10-25	21
W15: evaluation complete	2026-11-15	9
W16: integration buffer (kept empty)	2026-11-22	0
W17: release	2026-11-29	3

Table B.1: Milestones and their issues. The backlog was seeded from the proposal by `scripts/seed-backlog.sh`.

B.3 Statistics

At the time of writing `main` has 116 commits from 87 merged pull requests, and 71 of 73 issues are closed; the two open ones are this report and the demonstration. The code is about 9,900 lines of Rust, 8,900 lines of Go and 2,400 lines of shell, tests included. The repository holds 8 fingerprints for the cells and the GKE node, 7 ground-truth fingerprints, 8 pipeline runs, 13 runtime runs and 5 metric files. Workstreams B and D exchanged their work on 2026-09-14, after the week-3 gate; the letters stayed with the work.

Appendix C

The Complete Probe Index

Table C.1 lists every probe in `corpus/index.json`: the 185 committed probes by family, then the 28 deferred probes. "Isolates" is the oracle's isolation (`seccomp`; `sec./LSM` for seccomp or an LSM; `undec.` for undecidable from the errno alone). "G" is the errno the kernel returns if the call reaches the oracle's point, and "A" the errno that absence of the entry point produces, with the encoded absence after a slash for io_uring opcodes. Errnos are raw numbers, as in the fingerprint: 2 `ENOENT`, 7 `E2BIG`, 8 `ENOEXEC`, 9 `EBADF`, 14 `EFAULT`, 19 `ENODEV`, 22 `EINVAL`, 38 `ENOSYS`, 61 `ENODATA`, 93 `EPROTONOSUPPORT`, 95 `EOPNOTSUPP`, 97 `EAFNOSUPPORT`, 126 `ENOKEY`. Every committed probe has a 1,000 ms timeout and runs in a forked child.

Table C.1: The complete probe index (`corpus/index.json`). G: guaranteed errno; A: absence errno (encoded absence after a slash); †: may autoload a module.

Probe	Isolates	G	A	Operation
`socket` (24)				
`socket.family.af_inet`	sec./LSM	–	–	socket(AF_INET, SOCK_STREAM, 0): the family every workload uses
`socket.family.af_alg`†	sec./LSM	–	97	socket(AF_ALG, SOCK_SEQPACKET, 0): the crypto user API is reachable
`socket.family.af_packet`	undec.	–	97	socket(AF_PACKET, SOCK_RAW, 0): raw link-layer access, CAP_NET_RAW
`socket.family.out_of_range`	seccomp	97	–	socket(1000, SOCK_STREAM, 0): a family beyond NPROTO, guaranteed EAFNOSUPPORT
`socket.family.af_unix`	sec./LSM	–	–	socket(AF_UNIX, SOCK_STREAM, 0): the local family every runtime needs
`socket.family.af_inet6`†	sec./LSM	–	97	socket(AF_INET6, SOCK_STREAM, 0): IPv6 is reachable
`socket.family.af_key`†	undec.	–	97	socket(AF_KEY, SOCK_RAW, PF_KEY_V2): the IPsec key management interface, CAP_NET_ADMIN
`socket.family.af_bluetooth`†	sec./LSM	–	97	socket(AF_BLUETOOTH, SOCK_STREAM, BTPROTO_RFCOMM): the Bluetooth stack is reachable. Only from the host network namespace: in any other, bt_sock_create answers EAFNOSUPPORT whatever the policy
`socket.family.af_can`†	sec./LSM	–	97	socket(AF_CAN, SOCK_RAW, CAN_RAW): the CAN bus stack is reachable
`socket.family.af_tipc`†	sec./LSM	–	97	socket(AF_TIPC, SOCK_RDM, 0): the TIPC cluster stack is reachable
`socket.family.af_rds`†	sec./LSM	–	97	socket(AF_RDS, SOCK_SEQPACKET, 0): Reliable Datagram Sockets are reachable
`socket.family.af_xdp`	undec.	–	97	socket(AF_XDP, SOCK_RAW, 0): AF_XDP zero-copy sockets, CAP_NET_RAW
`socket.family.af_rxrpc`†	sec./LSM	–	97	socket(AF_RXRPC, SOCK_DGRAM, PF_INET): the RxRPC transport is reachable
`socket.family.af_phonet`†	undec.	–	97	socket(AF_PHONET, SOCK_DGRAM, 0): the Phonet stack, CAP_SYS_ADMIN
`socket.family.af_ieee802154`†	sec./LSM	–	97	socket(AF_IEEE802154, SOCK_DGRAM, 0): the 802.15.4 stack is reachable
`socket.family.af_nfc`†	sec./LSM	–	97	socket(AF_NFC, SOCK_STREAM, NFC_SOCKPROTO_LLCP): the NFC stack is reachable. Only from the host network namespace: in any other, nfc_sock_create answers EAFNOSUPPORT whatever the policy
`socket.family.af_vsock`†	sec./LSM	–	97	socket(AF_VSOCK, SOCK_STREAM, 0): the host-guest vsock family is reachable
`socket.family.af_kcm`†	sec./LSM	–	97	socket(AF_KCM, SOCK_DGRAM, KCMPROTO_CONNECTED): kernel connection multiplexor
`socket.family.af_qipcrtr`†	sec./LSM	–	97	socket(AF_QIPCRTR, SOCK_DGRAM, PF_QIPCRTR): the Qualcomm IPC router is reachable

Table C.1 continued

Probe	Isolates	G	A	Operation
socket.family.af_smc†	sec./LSM	–	97	socket(AF_SMC, SOCK_STREAM, SMCPROTO_SMC): shared memory communications
socket.family.af_mctp†	sec./LSM	–	97	socket(AF_MCTP, SOCK_DGRAM, 0): the Management Component Transport Protocol
socket.family.af_ax25†	sec./LSM	–	97	socket(AF_AX25, SOCK_DGRAM, 0): the amateur-radio AX.25 stack is reachable
socket.family.af_appletalk†	sec./LSM	–	97	socket(AF_APPLETALK, SOCK_DGRAM, 0): the AppleTalk stack is reachable
socket.family.af_x25†	sec./LSM	–	97	socket(AF_X25, SOCK_SEQPACKET, 0): the X.25 stack is reachable
netlink (18)				
netlink.protocol.route	sec./LSM	–	–	socket(AF_NETLINK, SOCK_RAW, NETLINK_ROUTE): rtnetlink is reachable
netlink.protocol.usersock	sec./LSM	–	–	socket(AF_NETLINK, SOCK_RAW, NETLINK_USERSOCK): the user-space-to-user-space protocol
netlink.protocol.sock_diag	sec./LSM	–	93	socket(AF_NETLINK, SOCK_RAW, NETLINK_SOCK_DIAG): socket enumeration (ss, inet_diag)
netlink.protocol.nflog†	sec./LSM	–	93	socket(AF_NETLINK, SOCK_RAW, NETLINK_NFLOG): the legacy netfilter log protocol
netlink.protocol.xfrm†	sec./LSM	–	93	socket(AF_NETLINK, SOCK_RAW, NETLINK_XFRM): IPsec transform configuration
netlink.protocol.selinux	sec./LSM	–	93	socket(AF_NETLINK, SOCK_RAW, NETLINK_SELINUX): SELinux event notifications
netlink.protocol.iscsi†	sec./LSM	–	93	socket(AF_NETLINK, SOCK_RAW, NETLINK_ISCSI): the open-iSCSI transport
netlink.protocol.audit	sec./LSM	–	93	socket(AF_NETLINK, SOCK_RAW, NETLINK_AUDIT): the audit subsystem
netlink.protocol.fib_lookup	sec./LSM	–	93	socket(AF_NETLINK, SOCK_RAW, NETLINK_FIB_LOOKUP): route lookups from user space
netlink.protocol.connector†	sec./LSM	–	93	socket(AF_NETLINK, SOCK_RAW, NETLINK_CONNECTOR): the kernel connector (proc events)
netlink.protocol.netfilter†	sec./LSM	–	93	socket(AF_NETLINK, SOCK_RAW, NETLINK_NETFILTER): nfnetlink, the nf_tables entry point
netlink.protocol.kobject_uevent	sec./LSM	–	93	socket(AF_NETLINK, SOCK_RAW, NETLINK_KOBJECT_UEVENT): device hotplug events
netlink.protocol.generic	sec./LSM	–	–	socket(AF_NETLINK, SOCK_RAW, NETLINK_GENERIC): generic netlink, every modern family's bus
netlink.protocol.scsitransport	sec./LSM	–	93	socket(AF_NETLINK, SOCK_RAW, NETLINK_SCSITRANSPORT): the SCSI transport netlink
netlink.protocol.rdma†	sec./LSM	–	93	socket(AF_NETLINK, SOCK_RAW, NETLINK_RDMA): the RDMA subsystem's netlink
netlink.protocol.crypto†	sec./LSM	–	93	socket(AF_NETLINK, SOCK_RAW, NETLINK_CRYPTO): the crypto user configuration API
netlink.protocol.smc†	sec./LSM	–	93	socket(AF_NETLINK, SOCK_RAW, NETLINK_SMC): the SMC protocol's netlink
netlink.protocol.out_of_range	sec./LSM	93	–	socket(AF_NETLINK, SOCK_RAW, 32): a protocol at MAX_LINKS, guaranteed EPROTONOSUPPORT
io_uring (53)				
io_uring.register.probe	seccomp	9	38	io_uring_register(INT_MAX, IORING_REGISTER_PROBE, NULL, 0): guaranteed EBADF
io_uring.setup.zero_entries	seccomp	22	38	io_uring_setup(0, &params): guaranteed EINVAL, no ring is created
io_uring.opcode.nop	seccomp	–	38/61	IORING_REGISTER_PROBE reports IORING_OP_NOP (0, since 5.1): the control: a ring that can be probed at all supports it
io_uring.opcode.readv	seccomp	–	38/61	IORING_REGISTER_PROBE reports IORING_OP_READV (1, since 5.1): vectored reads without read(2)
io_uring.opcode.writev	seccomp	–	38/61	IORING_REGISTER_PROBE reports IORING_OP_WRITEV (2, since 5.1): vectored writes without write(2)
io_uring.opcode.fsync	seccomp	–	38/61	IORING_REGISTER_PROBE reports IORING_OP_FSYNC (3, since 5.1): fsync without fsync(2)
io_uring.opcode.read_fixed	seccomp	–	38/61	IORING_REGISTER_PROBE reports IORING_OP_READ_FIXED (4, since 5.1): reads into registered buffers
io_uring.opcode.write_fixed	seccomp	–	38/61	IORING_REGISTER_PROBE reports IORING_OP_WRITE_FIXED (5, since 5.1): writes from registered buffers
io_uring.opcode.poll_add	seccomp	–	38/61	IORING_REGISTER_PROBE reports IORING_OP_POLL_ADD (6, since 5.1): poll without poll(2)

Table C.1 continued

Probe	Isolates	G	A	Operation
io_uring.opcode.sync_file_range	seccomp	–	38/61	IORING_REGISTER_PROBE reports IORING_OP_SYNC_FILE_RANGE (8, since 5.2): sync_file_range without the syscall
io_uring.opcode.sendmsg	seccomp	–	38/61	IORING_REGISTER_PROBE reports IORING_OP_SENDMSG (9, since 5.3): sendmsg without sendmsg(2)
io_uring.opcode.recvmsg	seccomp	–	38/61	IORING_REGISTER_PROBE reports IORING_OP_RECVMSG (10, since 5.3): recvmsg without recvmsg(2)
io_uring.opcode.timeout	seccomp	–	38/61	IORING_REGISTER_PROBE reports IORING_OP_TIMEOUT (11, since 5.4): timers in the ring
io_uring.opcode.accept	seccomp	–	38/61	IORING_REGISTER_PROBE reports IORING_OP_ACCEPT (13, since 5.5): accept without accept4(2)
io_uring.opcode.connect	seccomp	–	38/61	IORING_REGISTER_PROBE reports IORING_OP_CONNECT (16, since 5.5): connect without connect(2)
io_uring.opcode.fallocate	seccomp	–	38/61	IORING_REGISTER_PROBE reports IORING_OP_FALLOCATE (17, since 5.6): fallocate without the syscall
io_uring.opcode.openat	seccomp	–	38/61	IORING_REGISTER_PROBE reports IORING_OP_OPENAT (18, since 5.6): opening files without openat(2)
io_uring.opcode.close	seccomp	–	38/61	IORING_REGISTER_PROBE reports IORING_OP_CLOSE (19, since 5.6): close without close(2)
io_uring.opcode.statx	seccomp	–	38/61	IORING_REGISTER_PROBE reports IORING_OP_STATX (21, since 5.6): statx without the syscall
io_uring.opcode.read	seccomp	–	38/61	IORING_REGISTER_PROBE reports IORING_OP_READ (22, since 5.6): read without read(2)
io_uring.opcode.write	seccomp	–	38/61	IORING_REGISTER_PROBE reports IORING_OP_WRITE (23, since 5.6): write without write(2)
io_uring.opcode.fadvise	seccomp	–	38/61	IORING_REGISTER_PROBE reports IORING_OP_FADVISE (24, since 5.6): fadvise without the syscall
io_uring.opcode.madvise	seccomp	–	38/61	IORING_REGISTER_PROBE reports IORING_OP_MADVISE (25, since 5.6): madvise without madvise(2)
io_uring.opcode.send	seccomp	–	38/61	IORING_REGISTER_PROBE reports IORING_OP_SEND (26, since 5.6): send without send(2)
io_uring.opcode.recv	seccomp	–	38/61	IORING_REGISTER_PROBE reports IORING_OP_RECV (27, since 5.6): recv without recv(2)
io_uring.opcode.openat2	seccomp	–	38/61	IORING_REGISTER_PROBE reports IORING_OP_OPENAT2 (28, since 5.6): openat2 without the syscall
io_uring.opcode.epoll_ctl	seccomp	–	38/61	IORING_REGISTER_PROBE reports IORING_OP_EPOLL_CTL (29, since 5.6): epoll_ctl without the syscall
io_uring.opcode.splice	seccomp	–	38/61	IORING_REGISTER_PROBE reports IORING_OP_SPLICE (30, since 5.7): splice without splice(2)
io_uring.opcode.tee	seccomp	–	38/61	IORING_REGISTER_PROBE reports IORING_OP_TEE (33, since 5.8): tee without tee(2)
io_uring.opcode.shutdown	seccomp	–	38/61	IORING_REGISTER_PROBE reports IORING_OP_SHUTDOWN (34, since 5.11): shutdown without shutdown(2)
io_uring.opcode.renameat	seccomp	–	38/61	IORING_REGISTER_PROBE reports IORING_OP_RENAMEAT (35, since 5.11): renames without renameat2(2)
io_uring.opcode.unlinkat	seccomp	–	38/61	IORING_REGISTER_PROBE reports IORING_OP_UNLINKAT (36, since 5.11): unlinks without unlinkat(2)
io_uring.opcode.mkdirat	seccomp	–	38/61	IORING_REGISTER_PROBE reports IORING_OP_MKDIRAT (37, since 5.15): mkdir without mkdirat(2)
io_uring.opcode.symlinkat	seccomp	–	38/61	IORING_REGISTER_PROBE reports IORING_OP_SYMLINKAT (38, since 5.15): symlinks without symlinkat(2)
io_uring.opcode.linkat	seccomp	–	38/61	IORING_REGISTER_PROBE reports IORING_OP_LINKAT (39, since 5.15): hard links without linkat(2)
io_uring.opcode.msg_ring	seccomp	–	38/61	IORING_REGISTER_PROBE reports IORING_OP_MSG_RING (40, since 5.18): messages and descriptors passed between rings
io_uring.opcode.fsetxattr	seccomp	–	38/61	IORING_REGISTER_PROBE reports IORING_OP_FSETXATTR (41, since 5.19): fsetxattr without the syscall
io_uring.opcode.setxattr	seccomp	–	38/61	IORING_REGISTER_PROBE reports IORING_OP_SETXATTR (42, since 5.19): setxattr without the syscall
io_uring.opcode.fgetxattr	seccomp	–	38/61	IORING_REGISTER_PROBE reports IORING_OP_FGETXATTR (43, since 5.19): fgetxattr without the syscall
io_uring.opcode.getxattr	seccomp	–	38/61	IORING_REGISTER_PROBE reports IORING_OP_GETXATTR (44, since 5.19): getxattr without the syscall

Table C.1 continued

Probe	Isolates	G	A	Operation
`io_uring.opcode.socket`	seccomp	–	38/61	IORING_REGISTER_PROBE reports IORING_OP_SOCKET (45, since 5.19): socket creation without socket(2), any family
`io_uring.opcode.uring_cmd`	seccomp	–	38/61	IORING_REGISTER_PROBE reports IORING_OP_URING_CMD (46, since 5.19): driver passthrough commands (NVMe, ublk) without ioctl(2)
`io_uring.opcode.send_zc`	seccomp	–	38/61	IORING_REGISTER_PROBE reports IORING_OP_SEND_ZC (47, since 6.0): zero-copy send
`io_uring.opcode.sendmsg_zc`	seccomp	–	38/61	IORING_REGISTER_PROBE reports IORING_OP_SENDMSG_ZC (48, since 6.1): zero-copy sendmsg
`io_uring.opcode.read_multishot`	seccomp	–	38/61	IORING_REGISTER_PROBE reports IORING_OP_READ_MULTISHOT (49, since 6.7): multishot reads
`io_uring.opcode.waitid`	seccomp	–	38/61	IORING_REGISTER_PROBE reports IORING_OP_WAITID (50, since 6.7): waitid without the syscall
`io_uring.opcode.futex_wait`	seccomp	–	38/61	IORING_REGISTER_PROBE reports IORING_OP_FUTEX_WAIT (51, since 6.7): futex wait without futex(2)
`io_uring.opcode.futex_wake`	seccomp	–	38/61	IORING_REGISTER_PROBE reports IORING_OP_FUTEX_WAKE (52, since 6.7): futex wake without futex(2)
`io_uring.opcode.futex_waitv`	seccomp	–	38/61	IORING_REGISTER_PROBE reports IORING_OP_FUTEX_WAITV (53, since 6.7): futex_waitv without the syscall
`io_uring.opcode.fixed_fd_install`	seccomp	–	38/61	IORING_REGISTER_PROBE reports IORING_OP_FIXED_FD_INSTALL (54, since 6.8): a ring-registered descriptor installed into the process's table
`io_uring.opcode.ftruncate`	seccomp	–	38/61	IORING_REGISTER_PROBE reports IORING_OP_FTRUNCATE (55, since 6.9): ftruncate without the syscall
`io_uring.opcode.bind`	seccomp	–	38/61	IORING_REGISTER_PROBE reports IORING_OP_BIND (56, since 6.11): bind without bind(2)
`io_uring.opcode.listen`	seccomp	–	38/61	IORING_REGISTER_PROBE reports IORING_OP_LISTEN (57, since 6.11): listen without listen(2)
`mount` (38)				
`mount.reach`	seccomp	2	–	mount(NULL, "/caliper-no-such-dir", "proc", 0, NULL): guaranteed ENOENT
`mount.type.proc`	undec.	–	19	/proc/filesystems lists proc: procfs, mountable anywhere the capability allows
`mount.type.sysfs`	undec.	–	19	/proc/filesystems lists sysfs: sysfs
`mount.type.tmpfs`	undec.	–	19	/proc/filesystems lists tmpfs: tmpfs
`mount.type.devtmpfs`	undec.	–	19	/proc/filesystems lists devtmpfs: the kernel's own /dev
`mount.type.devpts`	undec.	–	19	/proc/filesystems lists devpts: pseudoterminals
`mount.type.overlay`	undec.	–	19	/proc/filesystems lists overlay: overlayfs, every container's root
`mount.type.fuse`	undec.	–	19	/proc/filesystems lists fuse: user-space filesystems
`mount.type.fusectl`	undec.	–	19	/proc/filesystems lists fusectl: the FUSE control filesystem
`mount.type.cgroup2`	undec.	–	19	/proc/filesystems lists cgroup2: the unified cgroup hierarchy
`mount.type.cgroup`	undec.	–	19	/proc/filesystems lists cgroup: the v1 cgroup hierarchy
`mount.type.bpf`	undec.	–	19	/proc/filesystems lists bpf: pinned BPF objects
`mount.type.debugfs`	undec.	–	19	/proc/filesystems lists debugfs: kernel debugging interfaces
`mount.type.tracefs`	undec.	–	19	/proc/filesystems lists tracefs: ftrace
`mount.type.configfs`	undec.	–	19	/proc/filesystems lists configfs: kernel object configuration
`mount.type.securityfs`	undec.	–	19	/proc/filesystems lists securityfs: LSM interfaces
`mount.type.efivarfs`	undec.	–	19	/proc/filesystems lists efivarfs: UEFI variables
`mount.type.pstore`	undec.	–	19	/proc/filesystems lists pstore: persistent storage for crash logs
`mount.type.binfmt_misc`	undec.	–	19	/proc/filesystems lists binfmt_misc: interpreter registration
`mount.type.hugetlbfs`	undec.	–	19	/proc/filesystems lists hugetlbfs: huge pages
`mount.type.mqueue`	undec.	–	19	/proc/filesystems lists mqueue: POSIX message queues
`mount.type.autofs`	undec.	–	19	/proc/filesystems lists autofs: automounter
`mount.type.ecryptfs`	undec.	–	19	/proc/filesystems lists ecryptfs: stacked encryption
`mount.type.squashfs`	undec.	–	19	/proc/filesystems lists squashfs: compressed read-only images
`mount.type.iso9660`	undec.	–	19	/proc/filesystems lists iso9660: optical media images
`mount.type.vfat`	undec.	–	19	/proc/filesystems lists vfat: FAT
`mount.type.ext4`	undec.	–	19	/proc/filesystems lists ext4: ext4
`mount.type.btrfs`	undec.	–	19	/proc/filesystems lists btrfs: btrfs

Table C.1 continued

Probe	Isolates	G	A	Operation
`mount.type.xfs`	undec.	–	19	/proc/filesystems lists xfs: xfs
`mount.type.nfs`	undec.	–	19	/proc/filesystems lists nfs: NFS v3
`mount.type.nfs4`	undec.	–	19	/proc/filesystems lists nfs4: NFS v4
`mount.type.cifs`	undec.	–	19	/proc/filesystems lists cifs: SMB
`mount.type.9p`	undec.	–	19	/proc/filesystems lists 9p: Plan 9 transport (virtio-9p)
`mount.type.virtiofs`	undec.	–	19	/proc/filesystems lists virtiofs: virtio-fs
`mount.type.ntfs3`	undec.	–	19	/proc/filesystems lists ntfs3: NTFS, the in-kernel driver
`mount.fs.proc`	undec.	–	–	mount(none, /tmp/caliper-<run>-mount, proc): a second procfs, unmounted by the child
`mount.fs.overlay`†	undec.	–	–	mount(none, /tmp/caliper-<run>-mount, overlay, lowerdir=a:b): a read-only overlay, unmounted by the child
`mount.fs.fuse`†	undec.	22	–	mount(none, /tmp/caliper-<run>-mount, fuse): fuse without a descriptor, guaranteed EINVAL past the capability
`clone` (19)				
`unshare.flags.invalid`	seccomp	22	–	unshare(0x1): an invalid flag set, guaranteed EINVAL
`unshare.flags.newuser`	sec./LSM	–	–	unshare(CLONE_NEWUSER): user namespace creation without host privilege
`unshare.flags.newns`	undec.	–	–	unshare(CLONE_NEWNS): mount namespace creation, CAP_SYS_ADMIN
`unshare.flags.newnet`	undec.	–	–	unshare(CLONE_NEWNET): network namespace creation, CAP_SYS_ADMIN
`unshare.flags.newpid`	undec.	–	–	unshare(CLONE_NEWPID): PID namespace creation, CAP_SYS_ADMIN
`unshare.flags.newipc`	undec.	–	–	unshare(CLONE_NEWIPC): IPC namespace creation, CAP_SYS_ADMIN
`unshare.flags.newuts`	undec.	–	–	unshare(CLONE_NEWUTS): UTS namespace creation, CAP_SYS_ADMIN
`unshare.flags.newcgroup`	undec.	–	–	unshare(CLONE_NEWCGROUP): cgroup namespace creation, CAP_SYS_ADMIN
`unshare.flags.newtime`	undec.	–	–	unshare(CLONE_NEWTIME): time namespace creation, CAP_SYS_ADMIN
`unshare.flags.newuser_newnet`	sec./LSM	–	–	unshare(CLONE_NEWUSER | CLONE_NEWNET): user+net namespace creation escalation path
`clone.flags.newuser`	sec./LSM	–	–	clone(CLONE_NEWUSER | SIGCHLD): process creation in a new user namespace
`clone.flags.newns`	undec.	–	–	clone(CLONE_NEWNS | SIGCHLD): process creation in a new mount namespace, CAP_SYS_ADMIN
`clone.flags.newnet`	undec.	–	–	clone(CLONE_NEWNET | SIGCHLD): process creation in a new net namespace, CAP_SYS_ADMIN
`clone.flags.newpid`	undec.	–	–	clone(CLONE_NEWPID | SIGCHLD): process creation in a new PID namespace, CAP_SYS_ADMIN
`clone.flags.newipc`	undec.	–	–	clone(CLONE_NEWIPC | SIGCHLD): process creation in a new IPC namespace, CAP_SYS_ADMIN
`clone.flags.newuts`	undec.	–	–	clone(CLONE_NEWUTS | SIGCHLD): process creation in a new UTS namespace, CAP_SYS_ADMIN
`clone.flags.newcgroup`	undec.	–	–	clone(CLONE_NEWCGROUP | SIGCHLD): process creation in a new cgroup namespace, CAP_SYS_ADMIN
`clone.flags.newuser_newnet`	sec./LSM	–	–	clone(CLONE_NEWUSER | CLONE_NEWNET | SIGCHLD): user+net namespace process creation escalation path
`clone3.args.short`	seccomp	22	38	clone3(NULL, 0): a size below CLONE_ARGS_SIZE_VER0, guaranteed EINVAL
`path` (17)				
`path.masked.proc_kcore`	sec./LSM	–	2	openat(O_PATH) + fstatfs: mask (ENODATA), read-only mount (EROFS), or reachable path
`path.masked.proc_keys`	sec./LSM	–	2	openat(O_PATH) + fstatfs: mask (ENODATA), read-only mount (EROFS), or reachable path
`path.masked.proc_latency_stats`	sec./LSM	–	2	openat(O_PATH) + fstatfs: mask (ENODATA), read-only mount (EROFS), or reachable path
`path.masked.proc_timer_list`	sec./LSM	–	2	openat(O_PATH) + fstatfs: mask (ENODATA), read-only mount (EROFS), or reachable path
`path.masked.proc_timer_stats`	sec./LSM	–	2	openat(O_PATH) + fstatfs: mask (ENODATA), read-only mount (EROFS), or reachable path
`path.masked.proc_sched_debug`	sec./LSM	–	2	openat(O_PATH) + fstatfs: mask (ENODATA), read-only mount (EROFS), or reachable path
`path.masked.proc_acpi`	sec./LSM	–	2	openat(O_PATH) + fstatfs: mask (ENODATA), read-only mount (EROFS), or reachable path

Table C.1 continued

Probe	Isolates	G	A	Operation
`path.masked.proc_asound`	sec./LSM	–	2	openat(O_PATH) + fstatfs: mask (ENODATA), read-only mount (EROFS), or reachable path
`path.masked.proc_scsi`	sec./LSM	–	2	openat(O_PATH) + fstatfs: mask (ENODATA), read-only mount (EROFS), or reachable path
`path.masked.sys_firmware`	sec./LSM	–	2	openat(O_PATH) + fstatfs: mask (ENODATA), read-only mount (EROFS), or reachable path
`path.masked.sys_powercap`	sec./LSM	–	2	openat(O_PATH) + fstatfs: mask (ENODATA), read-only mount (EROFS), or reachable path
`path.readonly.proc_bus`	sec./LSM	–	2	openat(O_PATH) + fstatfs: mask (ENODATA), read-only mount (EROFS), or reachable path
`path.readonly.proc_fs`	sec./LSM	–	2	openat(O_PATH) + fstatfs: mask (ENODATA), read-only mount (EROFS), or reachable path
`path.readonly.proc_irq`	sec./LSM	–	2	openat(O_PATH) + fstatfs: mask (ENODATA), read-only mount (EROFS), or reachable path
`path.readonly.proc_sys`	sec./LSM	–	2	openat(O_PATH) + fstatfs: mask (ENODATA), read-only mount (EROFS), or reachable path
`path.readonly.proc_sysrq_trigger`	sec./LSM	–	2	openat(O_PATH) + fstatfs: mask (ENODATA), read-only mount (EROFS), or reachable path
`path.control.proc_self_status`	sec./LSM	–	2	openat(O_PATH) + fstatfs: mask (ENODATA), read-only mount (EROFS), or reachable path
`device` (8)				
`device.open.fuse`†	undec.	–	–	/dev/fuse (10:229, typically 0666): read-only nonblocking open
`device.open.net_tun`†	undec.	–	–	/dev/net/tun (10:200, typically 0666): read-only nonblocking open
`device.open.kvm`†	undec.	–	–	/dev/kvm (10:232, typically 0660 root:kvm): DAC-gated read-only nonblocking open
`device.open.kmsg`	undec.	–	–	/dev/kmsg (1:11, typically 0644): read-only nonblocking open
`device.open.mem`	undec.	–	–	/dev/mem (1:1, typically 0640 root:kmem): DAC-gated read-only nonblocking open
`device.open.port`	undec.	–	–	/dev/port (1:4, typically 0640 root:kmem): DAC-gated read-only nonblocking open
`device.open.null`	undec.	–	–	/dev/null (1:3, typically 0666): control read-only nonblocking open
`device.open.sda`	undec.	–	–	/dev/sda (8:0, typically 0660 root:disk): DAC-gated read-only nonblocking open
`capability` (8)				
`capability.sys_time`	undec.	–	–	settimeofday(NULL, NULL): capability-only time check
`capability.sys_boot`	undec.	22	–	reboot(0, 0, LINUX_REBOOT_CMD_CAD_OFF, NULL): capability check
`capability.sys_module`	undec.	9	–	finit_module(-1, "", 0): module capability check
`capability.mknod`	undec.	–	–	mknodat(AT_FDCWD, /tmp/caliper-mknod, S_IFCHR|0600, makedev(1,3))
`capability.sys_ptrace`	undec.	14	–	process_vm_readv(getppid(), valid local iovec, invalid remote iovec): access check
`capability.net_bind_service`	undec.	–	–	socket(AF_INET) + bind(0.0.0.0:1): privileged-port check
`capability.sys_nice`	undec.	–	–	setpriority(PRIO_PROCESS, 0, -1): priority capability check
`capability.sys_rawio`	undec.	–	–	iopl(3): x86 I/O privilege capability check (x86_64 only)
Deferred set: specified, never run				
`ioctl` (3)				
`ioctl.request.fioclex`	seccomp	9	–	ioctl(-1, FIOCLEX, 0): the request every workload issues
`ioctl.request.tiocsti`	seccomp	9	–	ioctl(-1, TIOCSTI, NULL): terminal input injection
`ioctl.request.tioclinux`	seccomp	9	–	ioctl(-1, TIOCLINUX, NULL): virtual console control
`bpf` (5)				
`bpf.cmd.map_create`	undec.	7	38	bpf(BPF_MAP_CREATE, attr, sizeof(union bpf_attr) + 8) with a non-zero byte past the union
`bpf.cmd.prog_load`	undec.	7	38	bpf(BPF_PROG_LOAD, attr, sizeof(union bpf_attr) + 8) with a non-zero byte past the union
`bpf.cmd.btf_load`	seccomp	7	38	bpf(BPF_BTF_LOAD, attr, sizeof(union bpf_attr) + 8) with a non-zero byte past the union
`bpf.prog_type.socket_filter`	undec.	14	38	bpf(BPF_PROG_LOAD, {prog_type: SOCKET_FILTER, insn_cnt: 1, insns: NULL}, sizeof(union bpf_attr)): the type unprivileged loaders may use
`bpf.prog_type.kprobe`	undec.	14	38	bpf(BPF_PROG_LOAD, {prog_type: KPROBE, insn_cnt: 1, insns: NULL}, sizeof(union bpf_attr)): a tracing type

Table C.1 continued

Probe	Isolates	G	A	Operation
`keyctl` (4)				
`keyctl.op.read`	seccomp	126	38	keyctl(KEYCTL_READ, 0x7fffffff, NULL, 0)
`keyctl.op.out_of_range`	seccomp	95	38	keyctl(0x7fffffff, 0, 0, 0, 0): an operation no kernel defines
`keyctl.add_key`	seccomp	126	38	add_key("user", "caliper", NULL, 0, 0x7fffffff)
`keyctl.request_key`	seccomp	126	38	request_key("user", "caliper", NULL, 0x7fffffff)
`prctl` (3)				
`prctl.option.out_of_range`	sec./LSM	22	–	prctl(0x7fffffff, 0, 0, 0, 0): an option no kernel defines
`prctl.option.set_mm`	sec./LSM	22	–	prctl(PR_SET_MM, PR_SET_MM_START_CODE, 0, 0, 1): a non-zero arg5
`prctl.option.set_dumpable`	sec./LSM	22	–	prctl(PR_SET_DUMPABLE, 2, 0, 0, 0): SUID_DUMP_ROOT, which prctl refuses
`perf_event_open` (3)				
`perf_event_open.flags.invalid`	seccomp	22	38	perf_event_open(&attr, 0, -1, -1, ~0UL)
`perf_event_open.attr.bad_size`	seccomp	7	38	perf_event_open({size: 1}, 0, -1, -1, 0)
`perf_event_open.cpu_wide`	sec./LSM	–	38	perf_event_open({type: SOFTWARE, config: CPU_CLOCK}, -1, 0, -1, PERF_FLAG_FD_CLOEXEC): every task on CPU 0
`userfaultfd` (3)				
`userfaultfd.flags.invalid`	seccomp	22	38	userfaultfd(UFFD_USER_MODE_ONLY | 2)
`userfaultfd.user_mode_only`	sec./LSM	–	38	userfaultfd(UFFD_USER_MODE_ONLY | O_CLOEXEC)
`userfaultfd.kernel_faults`	undec.	–	38	userfaultfd(O_CLOEXEC): kernel-mode faults
`open_by_handle_at` (2)				
`open_by_handle_at.open`	undec.	14	38	open_by_handle_at(AT_FDCWD, NULL, O_RDONLY)
`open_by_handle_at.name_to_handle_at`	seccomp	22	38	name_to_handle_at(AT_FDCWD, "/", NULL, NULL, ~0)
`module` (2)				
`module.init_module`	undec.	8	38	init_module(NULL, 0, "")
`module.delete_module`	undec.	2	38	delete_module("caliper_absent", O_NONBLOCK)
`seccomp_notify` (3)				
`seccomp_notify.new_listener`	seccomp	14	38	seccomp(SECCOMP_SET_MODE_FILTER, SECCOMP_FILTER_FLAG_NEW_LISTENER, NULL)
`seccomp_notify.get_notif_sizes`	seccomp	14	38	seccomp(SECCOMP_GET_NOTIF_SIZES, 0, NULL)
`seccomp_notify.ioctl_recv`	seccomp	9	38	ioctl(-1, SECCOMP_IOCTL_NOTIF_RECV, NULL)